

SPCA101N

POSTGRADUATE COURSE

Master of Computer Applications

FIRST YEAR - FIRST SEMESTER

PAPER - I

C++ AND DATA STRUCTURES



**INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS**

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced as per AICTE Choice Based Credit System(CBCS) in Semester pattern from the academic year 2020-21. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty.

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

MCA
FIRST YEAR - SEMESTER - I

FIRST SEMESTER
C++ AND DATA STRUCTURES

COURSE WRITER

Mr. K. RAJA SEKARAN, M.C.A., M.Phil.,
Asst. Professor in Computer Science
D.B Jain College
Thoraipakkam, Chennai - 600 097.

EDITING

Dr. S. SASIKALA, M.C.A., M.Phil., Ph.D.,
Asst. Prof. in Computer Science
Institute of Distance Education
University of Madras
Chennai - 600 005.

MCA
FIRST YEAR
FIRST SEMESTER
C++ AND DATA STRUCTURES
SYLLABUS

Objective of the course : This course introduces the basic concepts of programming in C++ and discuss the implementation of different linear and non linear data structures.

Course Outcomes : After successful completion of this course, the students should be able to Understand the behavior of basic data structures. Analyze a problem and determine the appropriate data structure for the problem.

Unit-I: Introduction to C++; Tokens, Keywords, Identifiers, Variables, Operators, Manipulators, Expressions and Control Structures in C++; Pointers - Functions in C++ - Main Function -Function Prototyping - Parameters Passing in Functions - Values Return by Functions - Inline Functions - Friend and Virtual Functions

Unit-II: Classes and Objects; Constructors and Destructors; and Operator Overloading and Type Conversions - Type of Constructors - Function overloading. Inheritance : Single Inheritance - Multilevel Inheritance - Multiple Inheritance - Hierarchical Inheritance - Hybrid Inheritance. Pointers, Virtual Functions and Polymorphism; Managing Console I/O operations.

Unit-III: Working with Files: Classes for File Stream Operations - Opening and Closing a File - End-of-File Deduction - File Pointers - Updating a File - Error Handling during File Operations - Command-line Arguments. Data Structures: Definition of a Data structure - primitive and composite Data Types, Asymptotic notations, Arrays, Operations on Arrays, Order lists.

Unit-IV: Stacks - Applications of Stack - Infix to Postfix Conversion, Recursion, Maze Problems - Queues- Operations on Queues, Queue Applications, Circular Queue. Singly Linked List- Operations, Application - Representation of a Polynomial, Polynomial Addition; Doubly Linked List - Operations, Applications.

Unit-V: Trees and Graphs: Binary Trees - Conversion of Forest to Binary Tree, Operations -Tree Traversals; Graph - Definition, Types of Graphs, Hashing Tables and Hashing Functions, Traversal - Shortest Path; Dijkstra's Algorithm.

1. Recommended Texts :

- a. E.Horowitz, S.Sahni and Mehta, 1999, Fundamentals of Data Structures in C++, Galgotia.
- b. Herbert Schildt, 1999, C++ - The complete Reference, Third Edition, Tata McGraw – Hill.

2. Reference Books:

- a. GregoryL.Heileman, 1996, Data Structures , Algorithms and Object Oriented Programming Mc-Graw Hill International Editions.
- b. A.V.Aho, J.D. Ullman, J.E. Hopcraft: 2001, Data Structures and Algorithms-Adisson Wesley Pub.

MCA
FIRST YEAR
FIRST SEMESTER
C++ AND DATA STRUCTURES
SCHEME OF LESSONS

Sl. No.	Title	Page
1.	Introduction To OOPs	1
2.	Control Statements	21
3.	Classes	33
4.	Arrays Of Objects	63
5.	Function Overloading	82
6.	Operator Overloading	99
7.	Inheritance	120
8.	Polymorphism And Virtual Functions	137
9.	Exception Handling	156
10.	Streams And Files	179
11.	Namespaces And Conversion Functions	202
12.	Introduction to Data Structures	222
13.	Linked List	242
14.	Stacks & Queues	262
15.	Trees	293
16.	Graph	312

LESSON 1

INTRODUCTION TO OOPS

Structure

- 1.1 Introduction
- 1.2 Learning Objectives
- 1.3 OOPs Concept
- 1.4 Data Types
- 1.5 Identifier
- 1.6 Operators
- 1.7 Summary
- 1.8 Review Questions

1.1 Introduction

C++ began as an expanded version of C. The C++ extensions were first invented by Bjarne Stroustrup in 1979 at Bell Laboratories in Murray Hill, New Jersey. He initially called the new language “C with Classes.” However, in 1983 the name was changed to C++. Although C was one of the most liked and widely used professional programming languages in the world, the invention of C++ was necessitated by one major programming factor: increasing complexity. Over the years, computer programs have become larger and more complex. Even though C is an excellent programming language, it has its limits. In C, once a program exceeds from 25,000 to 100,000 lines of code, it becomes so complex that it is difficult to grasp as a totality. The purpose of C++ is to allow this barrier to be broken. The essence of C++ is to allow the programmer to comprehend and manage larger, more complex programs.

Most additions made by Stroustrup to C support object-oriented programming, sometimes referred to as OOP. (See the next section for a brief explanation of object oriented programming.) Stroustrup states that some of C++’s object-oriented features were inspired by another object-oriented language called Simula67. Therefore, C++ represents the blending of two powerful programming methods.

Since C++ was first invented, it has undergone three major revisions, with each adding to and altering the language. The first revision was in 1985 and the second in 1990. The third occurred during the standardization of C++. Several years ago, work began on a standard for C++. Toward that end, a joint ANSI (American National Standards Institute) and ISO (International Standards Organization) standardization committee was formed. The first draft of the proposed standard was created on January 25, 1994. In that draft, the ANSI/ISO C++ committee (of which I am a member) kept the features first defined by Stroustrup and added some new ones as well. But in general, this initial draft reflected the state of C++ at the time. Soon after the completion of the first draft of the C++ standard, an event occurred that caused the language to be greatly expanded: the creation of the Standard Template Library (STL) by Alexander Stepanov. The STL is a set of generic routines that you can use to manipulate data. It is both powerful and elegant, but also quite large. Subsequent to the first draft, the committee voted to include the STL in the specification for C++. The addition of the STL expanded the scope of C++ well beyond its original definition. While important, the inclusion of the STL, among other things, slowed the standardization of C++.

It is fair to say that the standardization of C++ took far longer than anyone had expected when it began. In the process, many new features were added to the language and many small changes were made. In fact, the version of C++ defined by the C++ committee is much larger and more complex than Stroustrup's original design. However, the standard is now complete. The final draft was passed out of committee on November 14, 1997. A standard for C++ is now a reality. The material in this book describes Standard C++, including all of its newest features. This is the version of C++ created by the ANSI/ISO standardization committee, and it is the one that is currently accepted by all major compilers.

1.2 Learning Objectives

After learning this lesson you should be able to know

- OOPs concepts
- Data types, Identifier
- Operators and its types
- Sample C++ Program

1.3 OOPs concepts

Since Object-Oriented Programming (OOP) drove the creation of c++, it is necessary to understand its foundational principles. OOP is a powerful way to approach the job of Programming. Programming methodologies have changed dramatically since the Invention of the computer, primarily to accommodate the increasing complexity of Programs. For example, when computers were first invented, programming was done by toggling in the binary machine instructions using the computer's front panel. As long as programs were just a few hundred instructions long, this approach worked. As Programs grew, assembly language was invented so that a programmer could deal with larger, increasingly complex programs, using symbolic representations of the Machine instructions. As programs continued to grow, high-level languages were introduced that gave the programmer more tools with which to handle complexity. The first widespread language was, of course, FORTRAN. Although FORTRAN was a very impressive first step, it is hardly a language that encourages clear, easy-understanding Programs.

The 1960s gave birth to structured programming. This is the method encouraged by Languages such as C and Pascal. The use of structured languages made it possible to write moderately complex programs fairly easily. Structured languages are characterized by their support for stand-alone subroutines, local variables, rich control constructs, and their lack of reliance upon the GOTO. Although structured languages are a powerful tool, even they reach their limit when a project becomes too large. Consider this: at each milestone in the development of programming, techniques and tools were created to allow the programmer to deal with increasingly greater Complexity. Each step of the way, the new approach took the best elements of the previous methods and moved forward. prior to the invention of OOP, many projects were nearing (or exceeding) the point where the structured approach no longer worked. Object-oriented methods were created to help programmers break through these barriers.

Object-oriented programming took the best ideas of structured programming and combined them with several new concepts. The result was a different way of organizing a program. In the most general sense, a program can be organized in one of two ways: around its code (what is happening) or around its data (who is being affected). Using only structured programming techniques, programs are typically organized around code. This approach can be thought of as "code acting on data." For example, a program written in a structured language such as C is defined by its functions, any of which may operate on any type of data used by the program. Object-oriented programs work the other way around. They are organized around data, with the key principle being "data controlling access to code." In an object-oriented language,

you define the data and the routines that are permitted to act on that data. Thus, a data type defines precisely what sort of operations can be applied to that data. To support the principles of object-oriented programming, all OOP languages have three traits in common: encapsulation, polymorphism, and inheritance.

1.3.1 Encapsulation

Encapsulation is the process of combining data and functions into a single unit called class. Using the method of encapsulation, the programmer cannot directly access the data. Data is only accessible through the functions present inside the class. Data encapsulation led to the important concept of data hiding. Data hiding is the implementation details of a class that are hidden from the user. The concept of restricted access led programmers to write specialized functions or methods for performing the operations on hidden members of the class. Attention must be paid to ensure that the class is designed properly.

Neither too much access nor too much control must be placed on the operations in order to make the class user friendly. Hiding the implementation details and providing restrictive access leads to the concept of abstract data type. Encapsulation leads to the concept of data hiding, but the concept of encapsulation must not be restricted to information hiding. Encapsulation clearly represents the ability to bundle related data and functionality within a single, autonomous entity called a class.

The key strength behind Data Encapsulation in C++ is that the keywords or the access specifiers can be placed in the class declaration as public, protected or private. A class placed after the keyword public is accessible to all the users of the class. The elements placed after the keyword private are accessible only to the methods of the class. In between the public and the private access specifiers, there exists the protected access specifier. Elements placed after the keyword protected are accessible only to the methods of the class or classes derived from that class.

The concept of encapsulation shows that a non-member function cannot access an object's private or protected data. This adds security, but in some cases the programmer might require an unrelated function to operate on an object of two different classes. The programmer is then able to utilize the concept of friend functions. Encapsulation alone is a powerful feature that leads to information hiding, abstract data type and friend functions.

Features and Advantages of the concept of Encapsulation:

1. Makes Maintenance of Application Easier:
2. Improves the Understandability of the Application
3. Enhanced Security:

1.3.2 Polymorphism

Object-oriented programming languages support polymorphism, which is characterized by the phrase “one interface, multiple methods.” In simple terms, polymorphism is the attribute that allows one interface to control access to a general class of actions. The specific action selected is determined by the exact nature of the situation. A real-world example of polymorphism is a thermostat. No matter what type of furnace your house has (gas, oil, electric, etc.), the thermostat works the same way. In this case, the thermostat (which is the interface) is the same no matter what type of furnace (method) you have. For example, if you want a 70-degree temperature, you set the thermostat to 70 degrees. It doesn't matter what type of furnace actually provides the heat. This same principle can also apply to programming. For example, you might have a program that defines three different types of stacks. One stack is used for integer values, one for character values, and one for floating-point values. Because of polymorphism, you can define one set of names, **push()** and **pop()**, that can be used for all three stacks. In your program you will create three specific versions of these functions, one for each type of stack, but names of the functions will be the same. The compiler will automatically select the right function based upon the data being stored.

Thus, the interface to a stack—the functions **push()** and **pop()**—are the same no matter which type of stack is being used. The individual versions of these functions define the specific implementations (methods) for each type of data.

Polymorphism helps reduce complexity by allowing the same interface to be used to access a general class of actions. It is the compiler's job to select the specific action (i.e., method) as it applies to each situation. You, the programmer, don't need to do this selection manually. You need only remember and utilize the general interface. The first object-oriented programming languages were interpreters, so polymorphism was, of course, supported at run time. However, C++ is a compiled language. Therefore, in C++, both run-time and compile-time polymorphism are supported.

1.3.3 Inheritance

Inheritance is the process by which one object can acquire the properties of another object. This is important because it supports the concept of classification. If you think about it, most knowledge is made manageable by hierarchical classifications. For example, a Red Delicious apple is part of the classification apple, which in turn is part of the fruit class, which is under the larger class food. Without the use of classifications, each object would have to define explicitly all of its characteristics. However, through the use of classifications, an object need only define those qualities that make it unique within its class. It is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case. As you will see, inheritance is an important aspect of object-oriented programming.

1.4 Data types

There are five atomic data types in C: character, integer, floating-point, double floating-point, and valueless (char, int, float, double, and void, respectively). As you will see, all other data types in C are based upon one of these types. The size and range of these data types may vary between processor types and compilers. However, in all cases a character is 1 byte. The size of an integer is usually the same as the word length of the execution environment of the program. For most 16-bit environments, such as DOS or Windows 3.1, an integer is 16 bits. For most 32-bit environments, such as Windows NT, an integer is 32 bits. However, you cannot make assumptions about the size of an integer if you want your programs to be portable to the widest range of environments. It is important to understand that both C and C++ only stipulate the minimal range of each data type, not its size in bytes.

Standard C++ does not specify a minimum size or range for the basic types. Instead, it simply states that they must meet certain requirements. For example, Standard C++ states that an int will “have the natural size suggested by the architecture of the execution environment.” In all cases, this will meet or exceed the minimum ranges specified by Standard C. Each C++ compiler specifies the size and range of the basic types in the header <climits>.

The bool Data Type

C++ defines a built-in Boolean type called bool. At the time of this writing, Standard C does not. Objects of type bool can store only the values true or false, which are keywords defined by C++. As explained in Part One, automatic conversions take place which allow bool values to be converted to integers, and vice versa. Specifically, any non-zero value is converted to true and zero is converted to false. The reverse also occurs; true is converted to 1 and false

is converted to zero. Thus, the fundamental concept of zero being false and non-zero being true is still fully entrenched in the C++ language.

1.5 Identifier

In C/C++, the names of variables, functions, labels, and various other user-defined objects are called identifiers. These identifiers can vary from one to several characters. The first character must be a letter or an underscore, and subsequent characters must be letters, digits, or underscores. Here are some correct and incorrect identifier names:

<u>Correct</u>	<u>Incorrect</u>
Count	1count
test23	hi!there
high_balance	high...balance

In C++, there is no limit to the length of an identifier, and at least the first 1,024 characters are significant. This difference may be important if you are converting a program from C to C++.

In an identifier, upper- and lowercase are treated as distinct. Hence, count, Count, and COUNT are three separate identifiers.

An identifier cannot be the same as a C or C++ keyword, and should not have the same name as functions that are in the C or C++ library.

1.5.1 Variables

As you probably know, a variable is a named location in memory that is used to hold a value that may be modified by the program. All variables must be declared before they can be used. The general form of a declaration is `type variable_list;`

Here, `type` must be a valid data type plus any modifiers, and `variable_list` may consist of one or more identifier names separated by commas.

Where Variables Are Declared

Variables will be declared in three basic places: inside functions, in the definition of function parameters, and outside of all functions. These are local variables, formal parameters, and global variables.

Local Variables

Variables that are declared inside a function are called local variables. In some C/C++ literature, these variables are referred to as automatic variables. This book uses the more common term, local variable. Local variables may be referenced only by statements that are inside the block in which the variables are declared. In other words, local variables are not known outside their own code block. Remember, a block of code begins with an opening curly brace and terminates with a closing curly brace. Local variables exist only while the block of code in which they are declared is executing. That is, a local variable is created upon entry into its block and destroyed upon exit. The most common code block in which local variables are declared is the function.

Global Variables

Unlike local variables, global variables are known throughout the program and may be used by any piece of code. Also, they will hold their value throughout the program's execution. You create global variables by declaring them outside of any function. Any expression may access them, regardless of what block of code that expression is in. However, it is usually best to declare global variables at the top of the program.

1.5.2 Constants

Constants refer to fixed values that the program may not alter. Constants can be of any of the basic data types. The way each constant is represented depends upon its type. Constants are also called literals. Character constants are enclosed between single quotes. For example 'a' and '%' are both character constants. Both C and C++ define wide characters (used mostly in non-English language environments), which are 16 bits long. To specify a wide character constant, precede the character with an L.

Integer constants are specified as numbers without fractional components. For example, 10 and -100 are integer constants. Floating-point constants require the decimal point followed by the number's fractional component. For example, 11.123 is a floating-point constant. C/C++ also allows you to use scientific notation for floating-point numbers.

There are two floating-point types: float and double. There are also several variations of the basic types that you can generate using the type modifiers. By default, the compiler fits a numeric constant into the smallest compatible data type that will hold it. Therefore, assuming 16-bit integers, 10 is int by default, but 103,000 is a long. Even though the value 10 could fit into

a character type, the compiler will not cross type boundaries. The only exceptions to the smallest type rule are floating-point constants, which are assumed to be doubles.

For most programs you will write, the compiler defaults are adequate. However, you can specify precisely the type of numeric constant you want by using a suffix. For floating-point types, if you follow the number with an F, the number is treated as a float. If you follow it with an L, the number becomes a long double. For integer types, the U suffix stands for unsigned and the L for long.

Hexadecimal and Octal Constants

It is sometimes easier to use a number system based on 8 or 16 rather than 10 (our standard decimal system). The number system based on 8 is called octal and uses the digits 0 through 7. In octal, the number 10 is the same as 8 in decimal. The base 16 number system is called hexadecimal and uses the digits 0 through 9 plus the letters A through F, which stand for 10, 11, 12, 13, 14, and 15, respectively. For example, the hexadecimal number 10 is 16 in decimal. Because these two number systems are used frequently, C/C++ allows you to specify integer constants in hexadecimal or octal instead of decimal. A hexadecimal constant must consist of a 0x followed by the constant in hexadecimal form. An octal constant begins with a 0.

String Constants

C/C++ supports one other type of constant: the string. A string is a set of characters enclosed in double quotes. For example, "this is a test" is a string. You have seen examples of strings in some of the printf() statements in the sample programs.

Although C allows you to define string constants, it does not formally have a string data type. (C++ does define a string class, however.)

You must not confuse strings with characters. A single character constant is enclosed in single quotes, as in 'a'. However, "a" is a string containing only one letter.

Backslash Character Constants

Enclosing character constants in single quotes works for most printing characters. A few, however, such as the carriage return, are impossible to enter into a string from the keyboard.

For this reason, C/C++ includes the special backslash character constants. These are also referred to as escape sequences. You should use the backslash codes instead of their ASCII equivalents to help ensure portability.

1.6 Operators

C/C++ is very rich in built-in operators. In fact, it places more significance on operators than do most other computer languages. There are four main classes of operators: arithmetic, relational, logical, and bitwise. In addition, there are some special operators for particular tasks.

1.6.1 The Assignment Operator

You can use the assignment operator within any valid expression. This is not the case with most computer languages (including Pascal, BASIC, and FORTRAN), which treat the assignment operator as a special case statement. The general form of the assignment operator is

variable_name = expression;

where an expression may be as simple as a single constant or as complex as you require. C/C++ uses a single equal sign to indicate assignment (unlike Pascal or Modula-2, which use the `:=` construct). The target, or left part, of the assignment must be a variable or a pointer, not a function or a constant. Frequently in literature on C/C++ and in compiler error messages you will see these two terms: lvalue and rvalue. Simply put, an lvalue is any object that can occur on the left side of an assignment statement. For all practical purposes, “lvalue” means “variable.” The term rvalue refers to expressions on the right side of an assignment and simply means the value of an expression.

1.6.2 Arithmetic Operators

The operators `+`, `-`, `*`, and `/` work as they do in most other computer languages. You can apply them to almost any built-in data type. When you apply `/` to an integer or character, any remainder will be truncated. For example, `5/2` will equal 2 in integer division. The modulus operator `%` also works in C/C++ as it does in other languages, yielding the remainder of an integer division. However, you cannot use it on floating-point types.

1.6.3 Increment and Decrement

C/C++ includes two useful operators not generally found in other computer languages. These are the increment and decrement operators, ++ and --. The operator ++ adds 1 to its operand, and -- subtracts one.

1.6.4 Relational and Logical Operators

In the term relational operator, relational refers to the relationships that values can have with one another. In the term logical operator, logical refers to the ways these relationships can be connected. Because the relational and logical operators often work together, they are discussed together here.

The idea of true and false underlies the concepts of relational and logical operators. In C, true is any value other than zero. False is zero. Expressions that use relational or logical operators return 0 for false and 1 for true. C++ fully supports the zero/non-zero concept of true and false. However, it also defines the bool data type and the Boolean constants true and false. In C++, a 0 value is automatically converted into false, and a non-zero value is automatically converted into true. The reverse also applies: true converts to 1 and false converts to 0. In C++, the outcome of a relational or logical operation is **true** or **false**. But since this automatically converts into 1 or 0, the distinction between C and C++ on this issue is mostly academic.

Relational Operators

<u>Operator</u>	<u>Action</u>
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
= =	Equal
!=	Not equal

Logical Operators

<u>Operator</u>	<u>Action</u>
&&	AND
	OR
!	NOT

1.6.5 Bitwise Operators

Unlike many other languages, C/C++ supports a full complement of bitwise operators. Since C was designed to take the place of assembly language for most programming tasks, it needed to be able to support many operations that can be done in assembler, including operations on bits. Bitwise operation refers to testing, setting, or shifting the actual bits in a byte or word, which correspond to the char and int data types and variants. You cannot use bitwise operations on float, double, long double, void, bool, or other, more complex types. These bitwise operations are applied to the individual bits of the operands.

<u>Operator</u>	<u>Action</u>
&	AND
	OR
^	Exclusive OR (XOR)
~	One's complement (NOT)
>>	Shift right
<<	Shift left

1.6.6 The? Operator

C/C++ contains a very powerful and convenient operator that replaces certain statements of the if-then-else form. The ternary operator ? takes the general form

Exp1? Exp2: Exp3;

where Exp1, Exp2, and Exp3 are expressions. Notice the use and placement of the colon. The? Operator works like this: Exp1 is evaluated. If it is true, Exp2 is evaluated and becomes

the value of the expression. If Exp1 is false, Exp3 is evaluated and its value becomes the value of the expression.

1.6.7 The Comma Operator

The comma operator strings together several expressions. The left side of the comma operator is always evaluated as **void**. This means that the expression on the right side becomes the value of the total comma-separated expression.

1.6.8 The Dot (.) and Arrow (→) Operators

In C, the . (dot) and the → (arrow) operators access individual elements of structures and unions. Structures and unions are compound (also called aggregate) data types that may be referenced under a single name. In C++, the dot and arrow operators are also used to access the members of a class.

1.6.9 sizeof () Operator

This operator accepts one parameter, which can be either a type or a variable itself and returns the size in bytes of that type or object:

```
a = sizeof (char);
```

This will assign the value 1 to a, because char is a one-byte long type.

The value returned by sizeof is a constant, so it is always determined before program execution.

Some of the new operators in C++ are-

1. :: Scope resolution operators.
2. ::* pointer to member decelerator.
3. à* pointer to member operator.
4. .* pointer to member operator.
5. delete memory release operator.
6. endl Line feed operator
7. new Memory allocation operator.
8. stew Field width operator.

Scope resolution Operator

- 1) C++ is a block - structured language. The scope of the variable extends from the point of its declaration till the end of the block containing the declaration.
- 2) Consider following program.

```

.....
{ int x = 1;
===
}
=====
{ int x = 2;
} =====

```

The two declaration of x refer to two different memory locations containing different values. Blocks in C++ are often nested. Declaration in a inner block hides a declaration of the same variable in an outer block.

- 3) In C, the global version of a variable cannot be accessed from within the inner block. C++ resolves this problem by using scope resolution operator (::), because this operator allows access to the global version of a variable.²⁷
- 4) Consider following program.

```

# include<iostream.h >

int m = 10;

int main ( )
{
  int m = 20;
  {
    int k = m;
    int m = 30;
    cout << .K = . <<k;
    cout << .m = .<<m;
  }
}

```



```

cout << ::m = . << ::m;
}
cout << .m. = .<< m;
cout << ::m = <<::m;
return 0;
}

```

The output of program is

```

k = 20
m = 30
:: m = 10
m = 20
:: m =10

```

In the above program m is declared at three places. And ::m will always refer to global m.

- 5) A major application of the scope resolution operator is in the classes to identify the class to which a member functions belongs.

Member dereferencing operators

C++ permits us to define a class containing various types of data & functions as members. C++ also permits us to access the class members through pointers. C++ provides a set of three pointers. C++ provides a set of three pointers to member operators.

- a. ::* - To access a pointer to a member of a class.
- b. .* - To access a member using object name & a pointer to that member.
- c. à* - To access a member using a pointer in the object & a pointer to the member.

Memory management operators

- 1) We use dynamic allocation techniques when it is not known in advance how much of memory space is needed. C++ supports two unary operators new and delete that perform the task of allocating & freeing the memory.

- 2) An object can be created by using new and destroyed by using delete. A data object created inside a block with new, will remain existence until it is explicitly destroyed by using delete.

- 3) It takes following form.

variable = new data type

The new operator allocated sufficient memory to hold a data object of type data-type & returns the address of the object.

EX p = new int.

Where p is a pointer of type int. Here p must have already been declared as pointer of appropriate types.

- 4) New can be used to create a memory space for any data type including user defined type such all array, classes etc.

Ex int * p = new int [10]

Creates a memory space for an array of 10 integers

- 5) When a data object is no longer needed, it is destroyed to release the memory space for reuse. The general form is delete variable. If we want to free a dynamically allocated array, we must use following form.

delete [size] variable

The size specifies the no of elements in the array to be freed.

- 6) The new operator has following advantages over the function malloc() in c -.

- I. It automatically computes the size of the data object. No need to use sizeof()
- II. It automatically returns the correct pointer type, so that there is no need to use a type cast.
- III. new and delete operators can be overloaded.
- IV. It is possible to initialize the object while creating the memory space.

Manipulators:

Manipulators are operators that are used to format the data display. There are two important manipulators.

- endl
- setw

endl : -

This manipulator is used to insert a linefeed into an output. It has same effect as using `.\n` for newline.

Example `cout<< .a. << a << endl <<.n=. <<n;
<< endl<<.p=.<<p<<endl;`

The output is

a = 2568

n = 34

p = 275

Setw : -

With the `setw`, we can specify a common field width for all the numbers and force them to print with right alignment.

Example `cout<<setw (5) <<sum<<endl;`

The manipulator `setw(5)` specifies a field width of 5 for printing the value of variable `sum` the value is right justified.

3 5 6

Type cast operator.

C++ permits explicit type conversion of variables or expressions using the type cast operator.

Syntax

type name (expression)

Example

```
avg = sum/float(i)
```

Here a type name behaves as if it is a function for converting values to a designated type.

A Closer Look at the I/O Operators

As stated, when used for I/O, the << and >> operators are capable of handling any of C++'s built-in data types. For example, this program inputs a float, a double, and a string and then outputs them:

```
#include <iostream.h>
using namespace std;
int main()
{
    float f;
    char str[80];
    double d;
    cout << "Enter two floating point numbers: ";
    cin >> f >> d;
    cout << "Enter a string: ";
    cin >> str;
    cout << f << " " << d << " " << str;
    return 0;
}
```

When you run this program, try entering This is a test. when prompted for the string. When the program redisplay the information you entered, only the word "This" will be displayed. The rest of the string is not shown because the >> operator stops reading input when the first

white-space character is encountered. Thus, “is a test” is never read by the program. This program also illustrates that you can string together several input operations in a single statement.

The New C++ Headers

As you know, when you use a library function in a program, you must include its header file. This is done using the `#include` statement. For example, in C, to include the header file for the I/O functions, you include `stdio.h` with a statement like this:

```
#include <stdio.h>
```

Here, `stdio.h` is the name of the file used by the I/O functions, and the preceding statement causes that file to be included in your program. The key point is that this `#include` statement includes a file.

When C++ was first invented and for several years after that, it used the same style of headers as did C. That is, it used header files. In fact, Standard C++ still supports C-style headers for header files that you create and for backward compatibility.

However, Standard C++ created a new kind of header that is used by the Standard C++ library. The new-style headers *do not* specify filenames. Instead, they simply specify standard identifiers that may be mapped to files by the compiler, although they need not be. The new-style C++ headers are an abstraction that simply guarantee that the appropriate prototypes and definitions required by the C++ library have been declared.

Since the new-style headers are not filenames, they do not have a `.h` extension. They consist solely of the header name contained between angle brackets. For example, here are some of the new-style headers supported by Standard C++.

```
<iostream> <fstream> <vector> <string>
```

The new-style headers are included using the `#include` statement. The only difference is that the new-style headers do not necessarily represent filenames. Because C++ includes the entire C function library, it still supports the standard C-style header files associated with that library. That is, header files such as `stdio.h` or `ctype.h` are still available. However, Standard C++ also defines new-style headers that you can use in place of these header files. The C++ versions of the C standard headers simply add a “c” prefix to the filename and drop the `.h`. For example, the C++ new-style header for `math.h` is `<cmath>`. The one for `string.h` is `<cstring>`.

Although it is currently permissible to include a C-style header file when using C library functions, this approach is deprecated by Standard C++ (that is, it is not recommended). For this reason, from this point forward, this book will use new-style C++ headers in all `#include` statements. If your compiler does not support new-style headers for the C function library, then simply substitute the old-style, C-like headers. Since the new-style header is a recent addition to C++, you will still find many, many older programs that don't use it. These programs employ C-style headers, in which a filename is specified. As the old-style skeletal program shows, the traditional way to include the I/O header is as shown here.

```
#include <iostream.h>
```

This causes the file `iostream.h` to be included in your program. In general, an old-style header file will use the same name as its corresponding new-style header with a `.h` appended.

1.7 Summary

Object-Oriented Programming (OOP) drove the creation of c++, it is necessary to understand its foundational principles. OOP is a powerful way to approach the job of Programming. Programming methodologies have changed dramatically since the Invention of the computer, primarily to accommodate the increasing complexity of Programs.

Object-oriented programming took the best ideas of structured programming and combined them with several new concepts. The result was a different way of organizing a program. In the most general sense, a program can be organized in one of two ways: around its code (what is happening) or around its data (who is being affected). Using only structured programming techniques, programs are typically organized around code. This approach can be thought of as “code acting on data.” For example, a program written in a structured language such as C is defined by its functions, any of which may operate on any type of data used by the program. Object-oriented programs work the other way around. They are organized around data, with the key principle being “data controlling access to code.” In an object-oriented language, you define the data and the routines that are permitted to act on that data. Thus, a data type defines precisely what sort of operations can be applied to that data. To support the principles of object-oriented programming, all OOP languages have three traits in common: encapsulation, polymorphism, and inheritance.

1.8 Review Questions

1. Explain the basic concepts of OOPs.
2. Define Encapsulation.
3. Explain about data types in C++.
4. Define constant. And explain.
5. Explain the different types of operators available in C++.
6. Define variable.
7. Explain identifiers.

LESSON 2

CONTROL STATEMENTS

Structure

- 2.1 Introduction**
- 2.2 Learning Objectives**
- 2.3 if – else Statement**
- 2.4 The switch Statement**
- 2.5 The for Loop**
- 2.6 The while Loop**
- 2.7 The do-while Loop**
- 2.8 Using break to Exit a Loop**
- 2.9 Using the goto Statement**
- 2.10 Summary**
- 2.11 Review Questions**

2.1 Introduction

This lesson discusses the statements that control a program's flow of execution. There are three categories of selection statements, which include the 'if' and the switch; iteration statements, which include the 'for', while, and do-while loops; and jump statements, which include break, continue, return, and goto.

2.2 Learning Objectives

After learning this lesson you should be able to know

- if – else Statement
- The switch Statement
- The for Loop

- The while Loop
- The do-while Loop
- Using break to Exit a Loop
- Using the goto Statement

2.3 IF – ELSE STATEMENT

The complete form of the if statement is

```
if(expression) statement
```

```
else statement
```

where the targets of the if and else are single statements.

The else clause is optional. The targets of both the 'if' and else can also be blocks of statements. The general form of the if using blocks of statements is

```
if(expression)
```

```
{
```

```
statement sequence
```

```
}
```

```
else
```

```
{
```

```
statement sequence
```

```
s}
```

If the conditional expression is true, the target of the 'if' will be executed; otherwise, the target of the else, if it exists, will be executed. At no time will both be executed. The conditional expression controlling the 'if' may be any type of valid C++ expression that produces a true or false result.

This program uses the 'if' statement to determine whether the user's guess matches the magic number. If it does, the message is printed on the screen. Taking the Magic Number program further, the next version uses the else to print a message when the wrong number is picked:

```

#include<iostream.h>
#include<cstdlib>

int main()
{
    int magic;// magic number
    int guess;// users guess
    magic = rand(); // get a random number
    cout << "Enter your guess"
    cin >> guess;
    if (guess == magic) cout << "Right";
    else cout<< " sorry , your are wrong";
    return 0;
}

```

2.3.1 NESTED IF

A nested if is an 'if' statement that is the target of another if or else. Nested ifs are very common in programming. The main thing to remember about nested ifs in C++ is that an else statement always refers to the nearest if statement that is within the same block as the else and not already associated with an else.

The general form of the nested if using blocks of statements is

```

if(expression)
{
    statement sequence
    if(expression)
    {
        statement sequence
    }
    else
    {

```

```
statement sequence
```

```
}
```

```
}
```

```
else
```

```
{
```

```
statement sequence
```

```
}
```

Here is an example

```
if(i)
```

```
{
```

```
if(j) result =1;
```

```
if(k) result =2;
```

```
else result = 3;
```

```
}
```

```
else result = 4;
```

The final else is not associated with if(j) (even though it is the closest if without an else), because it is not in the same block.

Rather, the final else is associated with if(i). The inner else is associated with if(k) because that is the nearest if. You can use a nested if to add a further improvement to the Magic Number program. This addition provides the player with feedback about a wrong guess.

2.4 The switch statement

The second of C++'s selection statements is the switch. The switch provides for a multiway branch. Thus, it enables a program to select among several alternatives. Although a series of nested if statements can perform multiway tests, for many situations the switch is a more efficient approach. It works like this: the value of an expression is successively tested against a list of constants. When a match is found, the statement sequence associated with that match is executed. The general form of the switch statement is

```
switch (expression)
```

```
{ case constant 1 :
```

Statement sequence

break;

case constant 2 :

Statement sequence

break;

case constant 3 :

Statement sequence

break;

.

.

.

default:

Statement sequence

}

The switch expression must evaluate to either a character or an integer value. (Floating point expressions, for example, are not allowed.) Frequently, the expression controlling the switch is simply a variable. The case constants must be integer or character literals.

The default statement sequence is performed if no matches are found. The default is optional; if it is not present, no action takes place if all matches fail. When a match is found, the statements associated with that case are executed until the break is encountered or, in a concluding case or default statement, until the end of the switch is reached.

There are four important things to know about the switch statement:

The switch differs from the 'if' in that switch can test only for equality (that is, for matches between the switch expression and the case constants), whereas the if conditional expression can be of any type.

No two case constants in the same switch can have identical values. Of course, a switch statement enclosed by an outer switch may have case constants that are the same. A switch statement is usually more efficient than nested ifs.

The statement sequences associated with each case are not blocks. However, the entire switch statement does define a block. The importance of this will become apparent as you learn more about C++.

The following program demonstrates the switch. It asks for a number between 1 and 3, inclusive. It then displays a proverb linked to that number. Any other number causes an error message to be displayed.

```
#include <iostream.h>
using namespace std;
int main()
{
    int num;
    cout<< "Enter a number from 1 to 3";
    cin >>num;
    switch(num)
    {
        case 1:
            cout << " A rolling stone gathers no moss.\n";
            break;
        case 2:
            cout<< " A bird in hand is worth two in the bush.\n";
            break;
        case 3:
            cout<< " A full and his money soon parted.\n";
            break;
        default :
            cout<< " you must enter 1 , 2 or 3.\n";
            break;
    }
    return 0;
```

```
}
```

Here is a sample run

Enter a number from 1 to 32

A bird in hand is worth two in the bush.35

2.5 The for loop

You have been using a simple form of the ‘for’ loop since Module 1. You might be surprised at just how powerful and flexible the ‘for’ loop is. Let’s begin by reviewing the basics, starting with the most traditional forms of the ‘for’.

The general form of the for loop for repeating a single statement is

```
for(initialization; expression; increment) statement;
```

For repeating a block, the general form is

```
for(initialization; expression; increment)
```

```
{
```

```
statement sequence
```

```
}
```

The initialization is usually an assignment statement that sets the initial value of the loop control variable, which acts as the counter that controls the loop. The expression is a conditional expression that determines whether the loop will repeat. The increment defines the amount by which the loop control variable will change each time the loop is repeated. Notice that these three major sections of the loop must be separated by semicolons. The ‘for’ loop will continue to execute as long as the conditional expression tests true. Once the condition becomes false, the loop will exit, and program execution will resume on the statement following the ‘for’ block.

The following program uses a ‘for’ loop to print the square roots of the numbers between 1 and 99. Notice that in this example, the loop control variable is called num.

```
#include <iostream.h>
```

```
#include<cmath>
```

```
int main()
```

```
{
```

```
int num;
```

```

double sq_root;
for(num=1;num<100;num++)
{
    sq_root=sqrt((double)num);
    cout<<num<<" "<<sq_root<<"\n";
}
return 0;
}

```

This program uses the standard function `sqrt( )`. As explained in Module 2, the `sqrt( )` function returns the square root of its argument. The argument must be of type `double`, and the function returns a value of type `double`. The header `<cmath>` is required.

2.6 The while loop

Another loop is the `while`. The general form of the `while` loop is `while (expression) statement`; where `statement` may be a single statement or a block of statements. The `expression` defines the condition that controls the loop, and it can be any valid expression.

The statement is performed while the condition is true. When the condition becomes false, program control passes to the line immediately following the loop.

The following program illustrates this characteristic of the `while` loop. It displays a line of periods. The number of periods displayed is equal to the value entered by the user. The program does not allow lines longer than 80 characters. The test for a permissible number of periods is performed inside the loop's conditional expression, not outside of it.

```

#include<iostream.h>
using namespace std;
int main()
{
    int len;
    cout<<"Enter length(1 to 79)";
    cin>>len;
    while(len>0 && len<80)
    {

```

```

    cout<<'.';
    len--;
}
return 0;
}

```

If len is out of range, then the while loop will not execute even once. Otherwise, the loop executes until len reaches zero.

There need not be any statements at all in the body of the while loop. Here is an example: `while(rand() != 100);` This loop iterates until the random number generated by `rand()` equals 100.

2.7 The do-while loop

The last of C++'s loops is the do-while. Unlike the 'for' and the while loops, in which the condition is tested at the top of the loop, the do-while loop checks its condition at the bottom of the loop. This means that a do-while loop will always execute at least once.

The general form of the do-while loop is

```

do
{ statements; }
while(condition);

```

Although the braces are not necessary when only one statement is present, they are often used to improve readability of the do-while construct, thus preventing confusion with the while. The do-while loop executes as long as the conditional expression is true.

The following program loops until the number 100 is entered:

```

#include<iostream.h>
using namespace std;
int main()
{
    int num;
    do

```



```

{
    cout << "Enter a number (100 to stop).";
    cin>> num;
}
while(num!=100);
return 0;
}

```

2.8 using break to exit a loop

It is possible to force an immediate exit from a loop, bypassing the loop's conditional test, by using the break statement. When the break statement is encountered inside a loop, the loop is immediately terminated, and program control resumes at the next statement following the loop. Here is a simple example:

```

#include<iostream.h>
using namespace std;
int main()
{
    int t;38
    for(t=0;t<100;t++)
    {
        if(t==10) break;
        cout<<t<< ' ';
    }
    return 0;
}

```

The output from the program is shown here: 0 1 2 3 4 5 6 7 8 9

As the output illustrates, this program prints only the numbers 0 through 9 on the screen before ending. It will not go to 100, because the break statement will cause it to terminate early.

Using `continue` it is possible to force an early iteration of a loop, bypassing the loop's normal control structure. This is accomplished using `continue`. The `continue` statement forces the next iteration of the loop to take place, skipping any code between itself and the conditional expression that controls the loop.

For example, the following program prints the even numbers between 0 and 100:

```
#include<iostream.h>
using namespace std;
int main()
{
    int x;
    for(x=0;x<100;x++)
    {
        if(x%2) continue;
        cout<<x<< ' ';
    }
    return 0;
}
```

Only even numbers are printed, because an odd number will cause the `continue` statement to execute, resulting in early iteration of the loop, bypassing the `cout` statement. Remember that the `%` operator produces the remainder of an integer division. Thus, when `x` is odd, the remainder is 1, which is true. When `x` is even, the remainder is 0, which is false. In `while` and `do-while` loops, a `continue` statement will cause control to go directly to the conditional expression and then continue the looping process. In the case of the `for`, the increment part of the loop is performed, then the conditional expression is executed, and then the loop continues.

2.9 using the `goto` statement

The `goto` is C++'s unconditional jump statement. Thus, when encountered, program flow jumps to the location specified by the `goto`. There are times when the use of the `goto` can clarify program flow rather than confuse it. The `goto` requires a label for operation. A label is a valid C++ identifier followed by a colon. Furthermore, the label must be in the same function as the `goto` that uses it.

For example, a loop from 1 to 100 could be written using a goto and a label, as shown here:

```
x=1;
loop1:
x++;
if(x<100) goto loop1;
```

One good use for the goto is to exit from a deeply nested routine.

For example, consider the following code fragment:

```
for(...){
for(...){
while(...){
if(...)goto stop
.
.
.
}
}
}
stop:
cout<<"Enter in program.\n";
```

Eliminating the goto would force a number of additional tests to be performed. A simple break statement would not work here, because it would only cause the program to exit from the innermost loop.

2.10 Summary

Statements that specify the order in which actions are executed are called control statements. C++ statements in a program execute one after the other in the order they are written (also known as sequential execution) unless there is a direct instruction to act otherwise. All the programs we have used so far included sequential execution which is one of types of

control statements. There are however other ways in which a program can execute a statement other than the next in the line. Enabling this to happen is known as transfer of control. Transfer of control includes other two forms of control statements. They include selection statements and repetition statements.

2.11 Review Questions

1. What is meant by control statements?
2. Explain If and Nested If statements with example.
3. Discuss about Switch statement with an example.
4. Explain for loop with an example.
5. Differentiate while and do... while statements.
6. Explain looping control statements in detail.
7. Explain the use of goto and break statements with example.

LESSON 3

CLASSES

Structure

- 3.1 Introduction
- 3.2 Learning Objectives
- 3.3 Classes
- 3.4 Friend Functions
- 3.5 Inline Function
- 3.6 Constructor and Destructor
- 3.7 Static Class Members
- 3.8 Summary
- 3.9 Review Questions

3.1 Introduction

Classes (also known as composite data types) are a collection of different data types grouped together to form one new self-defined type. The variable types we have used so far have been primitive types because they are declared in a very simplified manner. Each variable represented an object in its whole (i.e int car) but at times we want a group of values to be considered as one entity to represent an object (car). For example, we may need to use one or more data types to represent the car object. In such a situation we can group the different data types into one data type of our liking and then use it the same we have used other data types. In C++, there are three techniques of defining a new self-defined data type: a structure, a class, and a union.

3.2 Learning Objectives

After learning this Lesson you should be able to know

- Classes
- Friend Functions

- Inline Function
- Constructor and Destructor
- Static Class Members

3.3 Classes

Classes are created using the keyword `class`. A class declaration defines a new type that links code and data. This new type is then used to declare objects of that class. Thus, a class is a logical abstraction, but an object has physical existence. In other words, an object is an instance of a class.

A class declaration is similar syntactically to a structure. A simplified general form of a class declaration was shown. Here is the entire general form of a class declaration that does not inherit any other class.

```
class class-name {  
    private data and functions  
    access-specifier:  
    data and functions  
    access-specifier:  
    data and functions  
    // ...  
    access-specifier:  
    data and functions  
} object-list;
```

The object-list is optional. If present, it declares objects of the class. Here, access specifier is one of these three C++ keywords:

public

private

protected

By default, functions and data declared within a class are private to that class and may be accessed only by other members of the class. The public access specifier allows functions or data to be accessible to other parts of your program. The protected access specifier is needed only when inheritance is involved. Once an access specifier has been used, it remains in effect until either another access specifier is encountered or the end of the class declaration is reached.

You may change access specifications as often as you like within a class declaration.

For example, you may switch to public for some declarations and then switch back to private again. The class declaration in the following example illustrates this feature:

```
#include <iostream.h>
#include <cstring>
using namespace std;
class employee {
    char name[80]; // private by default
public:
    void putname(char *n); // these are public
    void getname(char *n);
private:
    double wage; // now, private again
public:
    void putwage(double w); // back to public
    double getwage();
};
void employee::putname(char *n)
{
    strcpy(name, n);
}
void employee::getname(char *n)
{
```

```

    strcpy(n, name);
}

void employee::putwage(double w)
{
    wage = w;
}

double employee::getwage()
{
    return wage;
}

int main()
{
    employee ted;
    char name[80];
    ted.putname("Ted Jones");
    ted.putwage(75000);
    ted.getname(name);
    cout << name << " makes $";
    cout << ted.getwage() << " per year.";
    return 0;
}

```

Here, `employee` is a simple class that is used to store an employee's name and wage. Notice that the public access specifier is used twice. Although you may use the access specifiers as often as you like within a class declaration, the only advantage of doing so is that by visually grouping various parts of a class, you may make it easier for someone else reading the program to understand it. However, to the compiler, using multiple access specifiers makes no difference.

Actually, most programmers find it easier to have only one private, protected, and public section within each class. For example, most programmers would code the `employee` class

as shown here, with all private elements grouped together and all public elements grouped together:

```
class employee {
    char name[80];
    double wage;
    public:
    void putname(char *n);
    void getname(char *n);
    void putwage(double w);
    double getwage();
};
```

Functions that are declared within a class are called member functions. Member functions may access any element of the class of which they are a part. This includes all private elements. Variables that are elements of a class are called member variables or data members. Collectively, any element of a class can be referred to as a member of that class.

There are a few restrictions that apply to class members. A non-static member variable cannot have an initializer. No member can be an object of the class that is being declared. No member can be declared as `auto`, `extern`, or `register`. In general, you should make all data members of a class private to that class. This is part of the way that encapsulation is achieved. However, there may be situations in which you will need to make one or more variables public.

When a variable is public, it may be accessed directly by any other part of your program. The syntax for accessing a public data member is the same as for calling a member function: Specify the object's name, the dot operator, and the variable name. This simple program illustrates the use of a public variable:

```
#include <iostream.h>

using namespace std;

class myclass {
    public:
    int i, j, k; // accessible to entire program
```

```

};

int main()
{
    myclass a, b;
    a.i = 100; // access to i, j, and k is OK
    a.j = 4;
    a.k = a.i * a.j;
    b.k = 12; // remember, a.k and b.k are different
    cout << a.k << " " << b.k;
    return 0;
}

```

3.4 Friend Functions

It is possible to grant a nonmember function access to the private members of a class by using a friend. A friend function has access to all private and protected members of the class for which it is a friend. To declare a friend function, include its prototype within the class, preceding it with the keyword `friend`. Consider this program:

```

#include <iostream.h>

using namespace std;

class myclass {
    int a, b;
public:
    friend int sum(myclass x);
    void set_ab(int i, int j);
};

void myclass::set_ab(int i, int j)
{
    a = i;
    b = j;
}

```

```

}
// Note: sum() is not a member function of any class.
int sum(myclass x)
{
/* Because sum() is a friend of myclass, it can
directly access a and b. */
return x.a + x.b;
}
int main()
{
myclass n;
n.set_ab(3, 4);
cout << sum(n);
return 0;
}

```

In this example, the `sum ()` function is not a member of `myclass`. However, it still has full access to its private members. Also, notice that `sum ()` is called without the use of the dot operator. Because it is not a member function, it does not need to be (indeed, it may not be) qualified with an object's name.

Although there is nothing gained by making `sum ()` a friend rather than a member function of `myclass`, there are some circumstances in which friend functions are quite valuable. First, friends can be useful when you are overloading certain types of operators. Second, friend functions make the creation of some types of I/O functions easier. The third reason that friend functions may be desirable is that in some cases, two or more classes may contain members that are interrelated relative to other parts of your program. Let's examine this third usage now.

To begin, imagine two different classes, each of which displays a pop-up message on the screen when error conditions occur. Other parts of your program may wish to know if a pop-up message is currently being displayed before writing to the screen so that no message is accidentally overwritten. Although you can create member functions in each class that return a value indicating whether a message is active, this means additional overhead when the condition

is checked (that is, two function calls, not just one). If the condition needs to be checked frequently, this additional overhead may not be acceptable. However, using a function that is a friend of each class, it is possible to check the status of each object by calling only this one function. Thus, in situations like this, a friend function allows you to generate more efficient code. The following program illustrates this concept:

```
#include <iostream.h>
using namespace std;
const int IDLE = 0;
const int INUSE = 1;
class C2; // forward declaration
class C1 {
int status; // IDLE if off, INUSE if on screen
// ...
public:
void set_status(int state);
friend int idle(C1 a, C2 b);
};
class C2 {
int status; // IDLE if off, INUSE if on screen
// ...
public:
void set_status(int state);
friend int idle(C1 a, C2 b);
};
void C1::set_status(int state)
{
status = state;
}
void C2::set_status(int state)
```

```

{
    status = state;
}

int idle(C1 a, C2 b)
{
    if(a.status || b.status) return 0;
    else return 1;
}

int main()
{
    C1 x;
    C2 y;
    x.set_status(IDLE);
    y.set_status(IDLE);
    if(idle(x, y)) cout << "Screen can be used.\n";
    else cout << "In use.\n";
    x.set_status(INUSE);
    if(idle(x, y)) cout << "Screen can be used.\n";
    else cout << "In use.\n";
    return 0;
}

```

Notice that this program uses a forward declaration (also called a forward reference) for the class C2. This is necessary because the declaration of `idle()` inside C1 refers to C2 before it is declared. To create a forward declaration to a class, simply use the form shown in this program.

A friend of one class may be a member of another. For example, here is the preceding program rewritten so that `idle()` is a member of C1:

```

#include <iostream.h>

using namespace std;

const int IDLE = 0;
const int INUSE = 1;

class C2; // forward declaration

class C1 {
int status; // IDLE if off, INUSE if on screen
// ...
public:
void set_status(int state);
int idle(C2 b); // now a member of C1
};

class C2 {
int status; // IDLE if off, INUSE if on screen
// ...
public:
void set_status(int state);
friend int C1::idle(C2 b);
};

void C1::set_status(int state)
{
status = state;
}

void C2::set_status(int state)
{
status = state;
}

// idle() is member of C1, but friend of C2

```

```

int C1::idle(C2 b)
{
    if(status || b.status) return 0;
    else return 1;
}

int main()
{
    C1 x;
    C2 y;
    x.set_status(IDLE);
    y.set_status(IDLE);
    if(x.idle(y)) cout << "Screen can be used.\n";
    else cout << "In use.\n";
    x.set_status(INUSE);
    if(x.idle(y)) cout << "Screen can be used.\n";
    else cout << "In use.\n";
    return 0;
}

```

Because `idle ()` is a member of `C1`, it can access the status variable of objects of type `C1` directly. Thus, only objects of type `C2` need be passed to `idle ()`. There are two important restrictions that apply to friend functions. First, a derived class does not inherit friend functions. Second, friend functions may not have a storage-class specifier. That is, they may not be declared as `static` or `extern`.

3.4.1 Friend Classes

It is possible for one class to be a friend of another class. When this is the case, the friend class and all of its member functions have access to the private members defined within the other class. For example,

```

// Using a friend class.
#include <iostream.h>
using namespace std;
class TwoValues {
int a;
int b;
public:
TwoValues(int i, int j) { a = i; b = j; }
friend class Min;
};
class Min {
public:
int min(TwoValues x);
};
int Min::min(TwoValues x)
{
return x.a < x.b ? x.a : x.b;
}
int main()
{
TwoValues ob(10, 20);
Min m;
cout << m.min(ob);
return 0;
}

```

In this example, class Min has access to the private variables a and b declared within the Two Values class. It is critical to understand that when one class is a friend of another, it only has access to names defined within the other class. It does not inherit the other class. Specifically,

the members of the first class do not become members of the friend class. Friend classes are seldom used. They are supported to allow certain special case situations to be handled.

3.5 Inline Functions

There is an important feature in C++, called an inline function that is commonly used with classes. Since the rest of this chapter (and the rest of the book) will make heavy use of it, inline functions are examined here.

In C++, you can create short functions that are not actually called; rather, their code is expanded in line at the point of each invocation. This process is similar to using a function-like macro. To cause a function to be expanded in line rather than called, precede its definition with the inline keyword. For example, in this program, the function max() is expanded in line instead of called:

```
#include <iostream.h>

using namespace std;

inline int max(int a, int b)
{
    return a>b ? a : b;
}

int main()
{
    cout << max(10, 20);
    cout << " " << max(99, 88);
    return 0;
}
```

As far as the compiler is concerned, the preceding program is equivalent to this one:

```
#include <iostream>

using namespace std;

int main()
{
```

```

cout << (10>20 ? 10 : 20);
cout << " " << (99>88 ? 99 : 88);
return 0;
}

```

The reason that inline functions are an important addition to C++ is that they allow you to create very efficient code. Since classes typically require several frequently executed interface functions (which provide access to private data), the efficiency of these functions is of critical concern. As you probably know, each time a function is called, a significant amount of overhead is generated by the calling and return mechanism. Typically, arguments are pushed onto the stack and various registers are saved when a function is called, and then restored when the function returns. The trouble is that these instructions take time. However, when a function is expanded in line, none of those operations occur. Although expanding function calls in line can produce faster run times, it can also result in larger code size because of duplicated code. For this reason, it is best to inline only very small functions. Further, it is also a good idea to inline only those functions that will have significant impact on the performance of your program.

Like the register specifier, inline is actually just a request, not a command, to the compiler. The compiler can choose to ignore it. Also, some compilers may not inline all types of functions. For example, it is common for a compiler not to inline a recursive function. You will need to check your compiler's user manual for any restrictions to inline. Remember, if a function cannot be inlined, it will simply be called as a normal function. Inline functions may be class member functions. For example, this is a perfectly

```

valid C++ program:
#include <iostream.h>
using namespace std;
class myclass {
int a, b;
public:
void init(int i, int j);
void show();
};

```

```

// Create an inline function.
inline void myclass::init(int i, int j)
{
    a = i;
    b = j;
}

// Create another inline function.
inline void myclass::show()
{
    cout << a << " " << b << "\n";
}

int main()
{
    myclass x;
    x.init(10, 20);
    x.show();
    return 0;
}

```

3.5.1 Defining Inline Functions within a Class

It is possible to define short functions completely within a class declaration. When a function is defined inside a class declaration, it is automatically made into an inline function (if possible). It is not necessary (but not an error) to precede its declaration with the inline keyword. For example, the preceding program is rewritten here with the definitions of `init()` and `show()` contained within the declaration of `myclass`:

```

#include <iostream.h>

using namespace std;

class myclass {
    int a, b;

```

```

public:
// automatic inline
void init(int i, int j)
{ a=i; b=j; }
void show() { cout << a << " " << b << "\n"; }
};

int main()
{
myclass x;
x.init(10, 20);
x.show();
return 0;
}

```

Notice the format of the function code within myclass. Because inline functions are short, this style of coding within a class is fairly typical. However, you are free to use any format you like. For example, this is a perfectly valid way to rewrite the myclass declaration:

```

#include <iostream.h>
using namespace std;
class myclass {
int a, b;
public:
// automatic inline
void init(int i, int j)
{
a = i;
b = j;
}
void show()

```

```

{
    cout << a << " " << b << "\n";
}
};

```

Technically, the inlining of the `show()` function is of limited value because (in general) the amount of time the I/O statement will take far exceeds the overhead of a function call. However, it is extremely common to see all short member functions defined inside their class in C++ programs. Constructor and destructor functions may also be inlined, either by default, if defined within their class, or explicitly.

3.6 Constructors and Destructors

It is very common for some part of an object to require initialization before it can be used. Before the stack could be used, `tos` had to be set to zero. This was performed by using the function `init()`. Because the requirement for initialization is so common, C++ allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a constructor function.

A constructor function is a special function that is a member of a class and has the same name as that class. For example, here is how the stack class looks when converted to use a constructor function for initialization:

```

// This creates the class stack.
class stack {
    int stck[SIZE];
    int tos;
public:
    stack(); // constructor
    void push(int i);
    int pop();
};

```

Notice that the constructor `stack ()` has no return type specified. In C++, constructor functions cannot return values and, thus, have no return type. The `stack ()` function is coded like this:

```
// stack's constructor function

stack::stack ()
{
    tos = 0;
    cout << "Stack Initialized\n";
}
```

Keep in mind that the message Stack Initialized is output as a way to illustrate the constructor. In actual practice, most constructor functions will not output or input anything. They will simply perform various initializations. An object's constructor is automatically called when the object is created. This means that it is called when the object's declaration is executed. If you are accustomed to thinking of a declaration statement as being passive, this is not the case for C++. In C++, a declaration statement is a statement that is executed. This distinction is not just academic. The code executed to construct an object may be quite significant. An object's constructor is called once for global or static local objects. For local objects, the constructor is called each time the object declaration is encountered. The complement of the constructor is the destructor. In many circumstances, an object will need to perform some action or actions when it is destroyed. Local objects are created when their block is entered, and destroyed when the block is left. Global objects are destroyed when the program terminates. When an object is destroyed, its destructor (if it has one) is automatically called. There are many reasons why a destructor function may be needed. For example, an object may need to deallocate memory that it had previously allocated or it may need to close a file that it had opened. In C++, it is the destructor function that handles deactivation events. The destructor has the same name as the constructor, but it is preceded by a ~. For example, here is the stack class and its constructor and destructor functions.

```
// This creates the class stack.

class stack {
    int stck[SIZE];
    int tos;
public:
    stack(); // constructor
    ~stack(); // destructor
```

```

void push(int i);
int pop();
};

// stack's constructor function
stack::stack()
{
    tos = 0;
    cout << "Stack Initialized\n";
}

// stack's destructor function
stack::~~stack()
{
    cout << "Stack Destroyed\n";
}

```

Notice that, like constructor functions, destructor functions do not have return values. To see how constructors and destructors work, here is a new version of the stack program examined earlier in this chapter. Observe that `init ()` is no longer needed.

```

public:
    stack(); // constructor
    ~stack(); // destructor
    void push(int i);
    int pop();
};

// stack's constructor function
stack::stack()
{
    tos = 0;
    cout << "Stack Initialized\n";
}

```

```

}
// stack's destructor function
stack::~~stack()
{
    cout << "Stack Destroyed\n";
}

```

Notice that, like constructor functions, destructor functions do not have return values. To see how constructors and destructors work, here is a new version of the stack program examined earlier in this chapter. Observe that `init()` is no longer needed.

```

{
    tos = 0;
    cout << "Stack Initialized\n";
}
// stack's destructor function
stack::~~stack()
{
    cout << "Stack Destroyed\n";
}
void stack::push(int i)
{
    if(tos==SIZE) {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = i;
    tos++;
}
int stack::pop()

```



```

{
if(tos==0) {
cout << "Stack underflow.\n";
return 0;
}
tos--;
return stck[tos];
}
int main()
{
stack a, b; // create two stack objects
a.push(1);
b.push(2);
a.push(3);
b.push(4);
cout << a.pop() << " ";
cout << a.pop() << " ";
cout << b.pop() << " ";
cout << b.pop() << "\n";
return 0;
}

```

This program displays the following:

Stack Initialized

Stack Initialized

3 1 4 2

Stack Destroyed

Stack Destroyed

3.6.1 Parameterized Constructors

It is possible to pass arguments to constructor functions. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object. For example, here is a simple class that includes a parameterized constructor:

```
#include <iostream.h>

using namespace std;

class myclass {
    int a, b;
public:
    myclass(int i, int j) {a=i; b=j;}
    void show() {cout << a << " " << b;}
};

int main()
{
    myclass ob(3, 5);
    ob.show();
    return 0;
}
```

Notice that in the definition of `myclass()`, the parameters `i` and `j` are used to give initial values to `a` and `b`.

The program illustrates the most common way to specify arguments when you declare an object that uses a parameterized constructor function. Specifically, this statement

```
myclass ob(3, 4);
```

causes an object called `ob` to be created and passes the arguments 3 and 4 to the `i` and `j` parameters of `myclass()`. You may also pass arguments using this type of declaration statement:

```
myclass ob = myclass(3, 4);
```

However, the first method is the one generally used, and this is the approach taken by most of the examples in this book. Actually, there is a small technical difference between the two types of declarations that relates to copy constructors. Here is another example that uses a parameterized constructor function. It creates a class that stores information about library books.

```
#include <iostream.h>
#include <cstring>
using namespace std;
const int IN = 1;
const int CHECKED_OUT = 0;
class book {
char author[40];
char title[40];
int status;
public:
book(char *n, char *t, int s);
int get_status() {return status;}
void set_status(int s) {status = s;}
void show();
};
book::book(char *n, char *t, int s)
{
strcpy(author, n);
strcpy(title, t);
status = s;
}
void book::show()
```

```

{
    cout << title << " by " << author;
    cout << " is ";
    if(status==IN) cout << "in.\n";
    else cout << "out.\n";
}

int main()
{
    book b1("Twain", "Tom Sawyer", IN);
    book b2("Melville", "Moby Dick", CHECKED_OUT);
    b1.show();
    b2.show();
    return 0;
}

```

Parameterized constructor functions are very useful because they allow you to avoid having to make an additional function call simply to initialize one or more variables in an object. Each function call you can avoid makes your program more efficient. Also, notice that the short `get_status()` and `set_status()` functions are defined in line, within the `book` class. This is a common practice when writing C++ programs.

3.6.2 Constructors with One Parameter: A Special Case

If a constructor only has one parameter, there is a third way to pass an initial value to that constructor. For example, consider the following short program.

```

#include <iostream.h>

using namespace std;

class X {
    int a;
public:
    X(int j) { a = j; }
}

```

```

int geta() { return a; }

};

int main()
{
    X ob = 99; // passes 99 to j
    cout << ob.geta(); // outputs 99
    return 0;
}

```

Here, the constructor for X takes one parameter. Pay special attention to how ob is declared in main(). In this form of initialization, 99 is automatically passed to the j parameter in the X() constructor. That is, the declaration statement is handled by the compiler as if it were written like this:

```
X ob = X(99);
```

In general, any time you have a constructor that requires only one argument, you can use either ob(i) or ob = i to initialize an object. The reason for this is that whenever you create a constructor that takes one argument, you are also implicitly creating a conversion from the type of that argument to the type of the class. Remember that the alternative shown here applies only to constructors that have exactly one parameter.

3.7 Static Class Members

Both function and data members of a class can be made static. This section explains the consequences of each.

3.7.1 Static Data Members

When you precede a member variable's declaration with static, you are telling the compiler that only one copy of that variable will exist and that all objects of the class will share that variable. Unlike regular data members, individual copies of a static member variable are not made for each object. No matter how many objects of a class are created, only one copy of a static data member exists. Thus, all objects of that class use that same variable. All static variables are initialized to zero before the first object is created.

When you declare a static data member within a class, you are not defining it. (That is, you are not allocating storage for it.) Instead, you must provide a global definition for it elsewhere, outside the class. This is done by redeclaring the static variable using the scope resolution operator to identify the class to which it belongs. This causes storage for the variable to be allocated. To understand the usage and effect of a static data member, consider this program:

```
#include <iostream.h>
using namespace std;
class shared {
static int a;
int b;
public:
void set(int i, int j) {a=i; b=j;}
void show();
};
int shared::a; // define a
void shared::show()
{
cout << "This is static a: " << a;
cout << "\nThis is non-static b: " << b;
cout << "\n";
}
int main()
{
shared x, y;
x.set(1, 1); // set a to 1
x.show();
y.set(2, 2); // change a to 2
y.show();
```

```

x.show(); /* Here, a has been changed for both x and y
because a is shared by both objects. */
return 0;
}

```

This program displays the following output when run.

This is static a: 1

This is non-static b: 1

This is static a: 2

This is non-static b: 2

This is static a: 2

This is non-static b: 1

3.7.2 Static Member Functions

Member functions may also be declared as static. There are several restrictions placed on static member functions. They may only directly refer to other static members of the class. (Of course, global functions and data may be accessed by static member functions.) A static member function does not have a `this` pointer. There cannot be a static and a non-static version of the same function. A static member function may not be virtual. Finally, they cannot be declared as `const` or `volatile`.

Following is a slightly reworked version of the shared-resource program from the previous section. Notice that `get_resource ()` is now declared as static. As the program illustrates, `get_resource ()` may be called either by itself, independent of any object, by using the class name and the scope resolution operator, or in connection with an object.

```

#include <iostream.h>

using namespace std;

class cl {
    static int resource;

public:
    static int get_resource();
    void free_resource() { resource = 0; }
}

```

```

};

int cl::resource; // define resource

int cl::get_resource()
{
    if(resource) return 0; // resource already in use
    else {
        resource = 1;
        return 1; // resource allocated to this object
    }
}

int main()
{
    cl ob1, ob2;

    /* get_resource() is static so may be called independent
    of any object. */

    if(cl::get_resource()) cout << "ob1 has resource\n";
    if(!cl::get_resource()) cout << "ob2 denied resource\n";
    ob1.free_resource();
    if(ob2.get_resource()) // can still call using object syntax
        cout << "ob2 can now use resource\n";
    return 0;
}

```

Actually, static member functions have limited applications, but one good use for them is to “preinitialize” private static data before any object is actually created. For example, this is a perfectly valid C++ program:

```

#include <iostream.h>

using namespace std;

class static_type {

```



```

static int i;

public:

static void init(int x) {i = x;}

void show() {cout << i;}

};

int static_type::i; // define i

int main()

{

// init static data before object creation

static_type::init(100);

static_type x;

x.show(); // displays 100

return 0;

}

```

3.7.3 When Constructors and Destructors Are Executed?

As a general rule, an object's constructor is called when the object comes into existence, and an object's destructor is called when the object is destroyed. Precisely when these events occur is discussed here.

A local object's constructor function is executed when the object's declaration statement is encountered. The destructor functions for local objects are executed in the reverse order of the constructor functions.

Global objects have their constructor functions execute before main () begins execution. Global constructors are executed in order of their declaration, within the same file. You cannot know the order of execution of global constructors spread among several files. Global destructors execute in reverse order after main() has terminated.

3.7.4 Nested Classes

It is possible to define one class within another. Doing so creates a nested class. Since a class declaration does, in fact, define a scope, a nested class is valid only within the scope of

the enclosing class. Frankly, nested classes are seldom used. Because of C++'s flexible and powerful inheritance mechanism, the need for nested classes is virtually nonexistent.

3.7.5 Passing Objects to Functions

Objects may be passed to functions in just the same way that any other type of variable can. Objects are passed to functions through the use of the standard call-by value mechanism. This means that a copy of an object is made when it is passed to a function. However, the fact that a copy is created means, in essence, that another object is created. This raises the question of whether the object's constructor function is executed when the copy is made and whether the destructor function is executed when the copy is destroyed. The answer to these two questions may surprise you. To begin, here is an example:

```
// Passing an object to a function.

#include <iostream.h>
using namespace std;
class myclass {
int i;
public:
myclass(int n);
~myclass();
void set_i(int n) { i=n; }
int get_i() { return i; }
};
myclass::myclass(int n)
{
i = n;
cout << "Constructing " << i << "\n";
}
myclass::~~myclass()
{
```

```

    cout << "Destroying " << i << "\n";
}
void f(myclass ob);
int main()
{
    myclass o(1);
    f(o);
    cout << "This is i in main: ";
    cout << o.get_i() << "\n";
    return 0;
}
void f(myclass ob)
{
    ob.set_i(2);
    cout << "This is local i: " << ob.get_i();
    cout << "\n";
}

```

This program produces this output:

Constructing 1

This is local i: 2

Destroying 2

This is i in main: 1

Destroying 1

3.8 Summary

Classes are created using the keyword `class`. A class declaration defines a new type that links code and data. This new type is then used to declare objects of that class. Thus, a class is a logical abstraction, but an object has physical existence. In other words, an object is an instance of a class. A class declaration is similar syntactically to a structure.

In object-oriented programming, a constructor (sometimes shortened to ctor) in a class is a special type of subroutine called to create an object. It prepares the new object for use, often accepting arguments that the constructor uses to set member variables required.

3.9 Review Questions

1. Define class.
2. Explain about classes in detail.
3. What is friend function?
4. What is inline function?
5. Explain about constructor with example.
6. Discuss about parameterized constructor with example.
7. Explain about static member class with an example.

LESSON 4

ARRAYS OF OBJECTS

Structure

- 4.1 Introduction**
- 4.2 Learning Objectives**
- 4.3 Array of Objects**
- 4.4 Pointer to Object**
- 4.5 The this Pointer**
- 4.6 References**
- 4.7 Dynamic Allocation Operators**
- 4.8 Summary**
- 4.9 Review Questions**

4.1 Introduction

Arrays of variables of type “class” is known as “Array of objects”. The “identifier” used to refer the array of objects is an user defined data type. In this lesson we discuss about arrays of object and references in detail.

4.2 Learning Objectives

After learning this lesson you should be able know

- Array of Objects
- Pointer to Object
- The this Pointer
- References
- Dynamic Allocation Operators

4.3 Array of Objects

In C++, it is possible to have arrays of objects. The syntax for declaring and using an object array is exactly the same as it is for any other type of array. For example, this program uses a three-element array of objects:

```
#include <iostream.h>

using namespace std;

class cl {
    int i;
public:
    void set_i(int j) { i=j; }
    int get_i() { return i; }
};

int main()
{
    cl ob[3];
    int i;
    for(i=0; i<3; i++) ob[i].set_i(i+1);
    for(i=0; i<3; i++)
        cout << ob[i].get_i() << "\n";
    return 0;
}
```

This program displays the numbers 1, 2, and 3 on the screen. If a class defines a parameterized constructor, you may initialize each object in an array by specifying an initialization list, just like you do for other types of arrays. However, the exact form of the initialization list will be decided by the number of parameters required by the object's constructor function. For objects whose constructors have only one parameter, you can simply specify a list of initial values, using the normal array-initialization syntax. As each element in the array is created, a value from the list is passed to the constructor's parameter. For example, here is a slightly different version of the preceding program that uses an initialization:

```

#include <iostream.h>

using namespace std;

class cl {
    int i;
public:
    cl(int j) { i=j; } // constructor
    int get_i() { return i; }
};

int main()
{
    cl ob[3] = {1, 2, 3}; // initializers
    int i;
    for(i=0; i<3; i++)
        cout << ob[i].get_i() << "\n";
    return 0;
}

```

As before, this program displays the numbers 1, 2, and 3 on the screen. Actually, the initialization syntax shown in the preceding program is shorthand for this longer form:

```
cl ob[3] = { cl(1), cl(2), cl(3) };
```

Here, the constructor for `cl` is invoked explicitly. Of course, the short form used in the program is more common. The short form works because of the automatic conversion that applies to constructors taking only one argument. Thus, the short form can only be used to initialize object arrays whose constructors only require one argument.

If an object's constructor requires two or more arguments, you will have to use the longer initialization form. For example,

```

#include <iostream.h>

using namespace std;

class cl {

```

```

int h;
int i;
public:
cl(int j, int k) { h=j; i=k; } // constructor with 2 parameters
int get_i() {return i;}
int get_h() {return h;}
};
int main()
{
cl ob[3] = {
cl(1, 2), // initialize
cl(3, 4),
cl(5, 6)
};
int i;
for(i=0; i<3; i++) {
cout << ob[i].get_h();
cout << ", ";
cout << ob[i].get_i() << "\n";
}
return 0;
}

```

Here, `cl`'s constructor has two parameters and, therefore, requires two arguments. This means that the shorthand initialization format cannot be used and the long form, shown in the example, must be employed.

4.3.1 Creating Initialized vs. Uninitialized Arrays

A special case situation occurs if you intend to create both initialized and uninitialized arrays of objects. Consider the following class.


```

class cl {
    int i;
public:
    cl(int j) { i=j; }
    int get_i() { return i; }
};

```

Here, the constructor function defined by `cl` requires one parameter. This implies that any array declared of this type must be initialized. That is, it precludes this array declaration:

```
cl a[9]; // error, constructor requires initializers
```

The reason that this statement isn't valid (as `cl` is currently defined) is that it implies that `cl` has a parameterless constructor because no initializers are specified. However, as it stands, `cl` does not have a parameterless constructor. Because there is no valid constructor that corresponds to this declaration, the compiler will report an error. To solve this problem, you need to overload the constructor function, adding one that takes no parameters. In this way, arrays that are initialized and those that are not are both allowed.

```

class cl {
    int i;
public:
    cl() { i=0; } // called for non-initialized arrays
    cl(int j) { i=j; } // called for initialized arrays
    int get_i() { return i; }
};

```

Given this class, both of the following statements are permissible:

```

cl a1[3] = {3, 5, 6}; // initialized
cl a2[34]; // uninitialized

```

4.4 Pointers to Objects

Just as you can have pointers to other types of variables, you can have pointers to objects. When accessing members of a class given a pointer to an object, use the arrow (`->`) operator

instead of the dot operator. The next program illustrates how to access an object given a pointer to it:

```
#include <iostream.h>
using namespace std;
class cl {
int i;
public:
cl(int j) { i=j; }
int get_i() { return i; }
};
int main()
{
cl ob(88), *p;
p = &ob; // get address of ob
cout << p->get_i(); // use -> to call get_i()
return 0;
}
```

As you know, when a pointer is incremented, it points to the next element of its type. For example, an integer pointer will point to the next integer. In general, all pointer arithmetic is relative to the base type of the pointer. (That is, it is relative to the type of data that the pointer is declared as pointing to.) The same is true of pointers to objects. For example, this program uses a pointer to access all three elements of array ob after being assigned ob's starting address:

```
#include <iostream.h>
using namespace std;
class cl {
int i;
public:
cl() { i=0; }
```

```

cl(int j) { i=j; }

int get_i() { return i; }

};

int main()
{
cl ob[3] = {1, 2, 3};

cl *p;

int i;

p = ob; // get start of array
for(i=0; i<3; i++) {
cout << p->get_i() << "\n";
p++; // point to next object
}

return 0;

}

```

You can assign the address of a public member of an object to a pointer and then access that member by using the pointer. For example, this is a valid C++ program that displays the number 1 on the screen:

```

#include <iostream.h>

using namespace std;

class cl {
public:
int i;

cl(int j) { i=j; }

};

int main()
{
cl ob(1);

```

```

int *p;

p = &ob.i; // get address of ob.i

cout << *p; // access ob.i via p

return 0;

}

```

Because `p` is pointing to an integer, it is declared as an integer pointer. It is irrelevant that `i` is a member of `ob` in this situation.

4.5 The `this` Pointer

When a member function is called, it is automatically passed an implicit argument that is a pointer to the invoking object (that is, the object on which the function is called). This pointer is called `this`. To understand this, first consider a program that creates a class called `pwr` that computes the result of a number raised to some power:

```

#include <iostream.h>

using namespace std;

class pwr {
    double b;
    int e;
    double val;
public:
    pwr(double base, int exp);
    double get_pwr() { return val; }
};

pwr::pwr(double base, int exp)
{
    b = base;
    e = exp;
    val = 1;
    if(exp==0) return;
}

```

```

for( ; exp>0; exp--) val = val * b;
}
int main()
{
    pwr x(4.0, 2), y(2.5, 1), z(5.7, 0);
    cout << x.get_pwr() << " ";
    cout << y.get_pwr() << " ";
    cout << z.get_pwr() << "\n";
    return 0;
}

```

Within a member function, the members of a class can be accessed directly, without any object or class qualification. Thus, inside `pwr()`, the statement

```
b = base;
```

means that the copy of `b` associated with the invoking object will be assigned the value contained in `base`. However, the same statement can also be written like this:

```
this->b = base;
```

The `this` pointer points to the object that invoked `pwr()`. Thus, `this->b` refers to that object's copy of `b`. For example, if `pwr()` had been invoked by `x` (as in `x(4.0, 2)`), then `this` in the preceding statement would have been pointing to `x`. Writing the statement without using `this` is really just shorthand.

Here is the entire `pwr()` function written using the `this` pointer:

```

pwr::pwr(double base, int exp)
{
    this->b = base;
    this->e = exp;
    this->val = 1;
    if(exp==0) return;
}

```

```

for( ; exp>0; exp—)
    this->val = this->val * this->b;
}

```

Actually, no C++ programmer would write `pwr()` as just shown because nothing is gained, and the standard form is easier. However, the 'this' pointer is very important when operators are overloaded and whenever a member function must utilize a pointer to the object that invoked it.

Remember that the 'this' pointer is automatically passed to all member functions. Therefore, `get_pwr()` could also be rewritten as shown here:

```

double get_pwr()
{ return this->val; }

```

In this case, if `get_pwr()` is invoked like this:

```
y.get_pwr();
```

then this will point to object `y`.

Two final points about 'this'. First, friend functions are not members of a class and, therefore, are not passed a 'this' pointer. Second, static member functions do not have a 'this' pointer.

4.6 References

C++ contains a feature that is related to the pointer called a reference. A reference is essentially an implicit pointer. There are three ways that a reference can be used: as a function parameter, as a function return value, or as a stand-alone reference. Each is examined here.

4.6.1 Reference Parameters

Probably the most important use for a reference is to allow you to create functions that automatically use call-by-reference parameter passing. Arguments can be passed to functions in one of two ways: using call-by-value or call-by-reference. When using call-by-value, a copy of the argument is passed to the function. Call-by-reference passes the address of the argument to the function. By default, C++ uses call-by-value, but it provides two ways to achieve call-by-reference parameter passing. First, you can explicitly pass a pointer to the argument. Second, you can use a reference parameter. For most circumstances the best way is to use a reference parameter.

To fully understand what a reference parameter is and why it is valuable, we will begin by reviewing how a call-by-reference can be generated using a pointer parameter. The following program manually creates a call-by-reference parameter using a pointer in the function called `neg()`, which reverses the sign of the integer variable pointed to by its argument.

```
// Manually create a call-by-reference using a pointer.
```

```
#include <iostream.h>
```

```
using namespace std;
```

```
void neg(int *i);
```

```
int main()
```

```
{
```

```
int x;
```

```
x = 10;
```

```
cout << x << " negated is ";
```

```
neg(&x);
```

```
cout << x << "\n";
```

```
return 0;
```

```
}
```

```
void neg(int *i)
```

```
{
```

```
*i = -*i;
```

```
}
```

In this program, `neg()` takes as a parameter a pointer to the integer whose sign it will reverse. Therefore, `neg()` must be explicitly called with the address of `x`. Further, inside `neg()` the `*` operator must be used to access the variable pointed to by `i`. This is how you generate a “manual” call-by-reference in C++, and it is the only way to obtain a call-by-reference using the C subset. Fortunately, in C++ you can automate this feature by using a reference parameter.

To create a reference parameter, precede the parameter’s name with an `&`. For example, here is how to declare `neg()` with `i` declared as a reference parameter:

```
void neg(int &i);
```

For all practical purposes, this causes `i` to become another name for whatever argument `neg()` is called with. Any operations that are applied to `i` actually affect the calling argument. In technical terms, `i` is an implicit pointer that automatically refers to the argument used in the call to `neg()`. Once `i` has been made into a reference, it is no longer necessary (or even legal) to apply the `*` operator. Instead, each time `i` is used, it is implicitly a reference to the argument and any changes made to `i` affect the argument. Further, when calling `neg()`, it is no longer necessary (or legal) to precede the argument's name with the `&` operator. Instead, the compiler does this automatically. Here is the reference version of the preceding program:

```
// Use a reference parameter.
#include <iostream.h>
using namespace std;
void neg(int &i); // i now a reference
int main()
{
    int x;
    x = 10;
    cout << x << " negated is ";
    neg(x); // no longer need the & operator
    cout << x << "\n";
    return 0;
}
void neg(int &i)
{
    i = -i; // i is now a reference, don't need *
}
```

To review: When you create a reference parameter, it automatically refers to (implicitly points to) the argument used to call the function. Therefore, in the preceding program, the statement


```
i = -i ;
```

actually operates on *x*, not on a copy of *x*. There is no need to apply the `&` operator to an argument. Also, inside the function, the reference parameter is used directly without the need to apply the `*` operator. In general, when you assign a value to a reference, you are actually assigning that value to the variable that the reference points to. Inside the function, it is not possible to change what the reference parameter is pointing to. That is, a statement like

```
i++:
```

inside `neg()` increments the value of the variable used in the call. It does not cause *i* to point to some new location.

Here is another example. This program uses reference parameters to swap the values of the variables it is called with. The `swap()` function is the classic example of call-by-reference parameter passing.

```
#include <iostream.h>
using namespace std;
void swap(int &i, int &j);
int main()
{
    int a, b, c, d;
    a = 1;
    b = 2;
    c = 3;
    d = 4;
    cout << "a and b: " << a << " " << b << "\n";
    swap(a, b); // no & operator needed
    cout << "a and b: " << a << " " << b << "\n";
    cout << "c and d: " << c << " " << d << "\n";
    swap(c, d);
    cout << "c and d: " << c << " " << d << "\n";
```

```

return 0;

}

void swap(int &i, int &j)

{
int t;
t = i; // no * operator needed
i = j;
j = t;
}

```

This program displays the following:

a and b: 1 2

a and b: 2 1

c and d: 3 4

c and d: 4 3

4.6.2 Returning References

A function may return a reference. This has the rather startling effect of allowing a function to be used on the left side of an assignment statement! For example, consider this simple program:

```

#include <iostream.h>

using namespace std;

char &replace(int i); // return a reference

char s[80] = "Hello There";

int main()

{
replace(5) = 'X'; // assign X to space after Hello
cout << s;
return 0;
}

```

```

}
char &replace(int i)
{
    return s[i];
}

```

This program replaces the space between Hello and There with an X. That is, the program displays HelloXthere. Take a look at how this is accomplished. First, `replace()` is declared as returning a reference to a character. As `replace()` is coded, it returns a reference to the element of `s` that is specified by its argument `i`. The reference returned by `replace()` is then used in `main()` to assign to that element the character X.

One thing to beware of when returning references is that the object being referred to does not go out of scope after the function terminates.

4.7 Dynamic Allocation Operators

C++ provides two dynamic allocation operators: `new` and `delete`. These operators are used to allocate and free memory at run time. Dynamic allocation is an important part of almost all real-world programs. C++ supports dynamic memory allocation functions, called `malloc()` and `free()`. These are included for the sake of compatibility with C. However, for C++ code, you should use the `new` and `delete` operators because they have several advantages.

The `new` operator allocates memory and returns a pointer to the start of it. The `delete` operator frees memory previously allocated using `new`. The general forms of `new` and `delete` are shown here:

```

p_var = new type;
delete p_var;

```

Here, `p_var` is a pointer variable that receives a pointer to memory that is large enough to hold an item of type `type`.

Since the heap is finite, it can become exhausted. If there is insufficient available memory to fill an allocation request, then `new` will fail and a `bad_alloc` exception will be generated. This exception is defined in the header `<new>`. Your program should handle this exception and take appropriate action if a failure occurs. If this exception is not handled by your program, then your program will be terminated.

The actions of `new` on failure as just described are specified by Standard C++. The trouble is that not all compilers, especially older ones, will have implemented `new` in compliance with Standard C++. When C++ was first invented, `new` returned null on failure. Later, this was changed such that `new` caused an exception on failure. Finally, it was decided that a `new` failure will generate an exception by default, but that a null pointer could be returned instead, as an option. Thus, `new` has been implemented differently, at different times, by compiler manufacturers. Although all compilers will eventually implement `new` in compliance with Standard C++, currently the only way to know the precise action of `new` on failure is to check your compiler's documentation.

Since Standard C++ specifies that `new` generates an exception on failure, this is the way the code in this book is written. If your compiler handles an allocation failure differently, you will need to make the appropriate changes.

Here is a program that allocates memory to hold an integer:

```
#include <iostream.h>
#include <new>
using namespace std;
int main()
{
    int *p;
    try {
        p = new int; // allocate space for an int
    } catch (bad_alloc xa) {
        cout << "Allocation Failure\n";
        return 1;
    }
    *p = 100;
    cout << "At " << p << " ";
    cout << "is the value " << *p << "\n";
    delete p;
```

```

return 0;
}

```

This program assigns to `p` an address in the heap that is large enough to hold an integer. It then assigns that memory the value 100 and displays the contents of the memory on the screen. Finally, it frees the dynamically allocated memory. Remember, if your compiler implements `new` such that it returns null on failure, you must change the preceding program appropriately.

The `delete` operator must be used only with a valid pointer previously allocated by using `new`. Using any other type of pointer with `delete` is undefined and will almost certainly cause serious problems, such as a system crash.

Although `new` and `delete` perform functions similar to `malloc()` and `free()`, they have several advantages. First, `new` automatically allocates enough memory to hold an object of the specified type. You do not need to use the `sizeof` operator. Because the size is computed automatically, it eliminates any possibility for error in this regard. Second, `new` automatically returns a pointer of the specified type. You don't need to use an explicit type cast as you do when allocating memory by using `malloc()`. Finally, both `new` and `delete` can be overloaded, allowing you to create customized allocation systems.

Although there is no formal rule that states this, it is best not to mix `new` and `delete` with `malloc()` and `free()` in the same program. There is no guarantee that they are mutually compatible.

4.7.1 Initializing Allocated Memory

You can initialize allocated memory to some known value by putting an initializer after the type name in the `new` statement. Here is the general form of `new` when an initialization is included:

```
p_var = new var_type (initializer);
```

Of course, the type of the initializer must be compatible with the type of data for which memory is being allocated.

This program gives the allocated integer an initial value of 87:

```

#include <iostream.h>
#include <new>
using namespace std;
int main()
{
    int *p;
    try {
        p = new int (87); // initialize to 87
    } catch (bad_alloc xa) {
        cout << "Allocation Failure\n";
        return 1;
    }
    cout << "At " << p << " ";
    cout << "is the value " << *p << "\n";
    delete p;
    return 0;
}

```

4.7.2 Allocating Arrays

You can allocate arrays using new by using this general form:

```
p_var = new array_type [size];
```

Here, size specifies the number of elements in the array. To free an array, use this form of delete:

```
delete [] p_var;
```

Here, the [] informs delete that an array is being released. For example, the next program allocates a 10-element integer array.

```

#include <iostream.h>
#include <new>

```

```

using namespace std;

int main()
{
    int *p, i;

    try {
        p = new int [10]; // allocate 10 integer array
    } catch (bad_alloc xa) {
        cout << "Allocation Failure\n";
        return 1;
    }

    for(i=0; i<10; i++ )
        p[i] = i;

    for(i=0; i<10; i++)
        cout << p[i] << " ";

    delete [] p; // release the array

    return 0;
}

```

Notice the delete statement. As just mentioned, when an array allocated by new is released, delete must be made aware that an array is being freed by using the []. One restriction applies to allocating arrays: They may not be given initial values. That is, you may not specify an initializer when allocating arrays.

4.8 Summary

Arrays of variables of type “class” is known as “Array of objects”. The “identifier” used to refer the array of objects is an user defined data type. C++ provides two dynamic allocation operators: new and delete. These operators are used to allocate and free memory at run time. Dynamic allocation is an important part of almost all real-world programs. C++ supports dynamic memory allocation functions, called malloc() and free(). These are included for the sake of compatibility with C. However, for C++ code, you should use the new and delete operators because they have several advantages.

4.9 Review Questions

1. Define array.
2. What is meant by object?
3. Explain about array of object in detail.
4. Discuss about pointer to object in detail.
5. What is 'this' pointer? Explain with example.
6. What is meant by references? Explain in detail.
7. Discuss about Dynamic memory allocation.

LESSON 5

FUNCTION OVERLOADING

Structure

- 5.1 Introduction
- 5.2 Learning Objectives
- 5.3 Function Overloading
- 5.4 Overloading Constrcutor function
- 5.5 Copy constructor
- 5.6 Default function arguments
- 5.7 Summary
- 5.8 Review Questions

5.1 Introduction

Function overloading or method overloading is a feature found in various programming languages such as Ada, C++, C#, D, and Java, that allows creating several methods with the same name which differ from each other in the type of the input and the output of the function. It is simply defined as the ability of one function to perform different tasks.

For example, `doTask()` and `doTask (object O)` are overloaded methods. To call the latter, an object must be passed as a parameter, whereas the former does not require a parameter, and is called with an empty parameter field. A common error would be to assign a default value to the object in the second method, which would result in an ambiguous call error, as the compiler wouldn't know which of the two methods to use.

One of the more important forms of an overloaded constructor is the copy constructor. Defining a copy constructor can help you prevent problems that might occur when one object is used to initialize another.

5.2 Learning Objectives

After learning this lesson you should be able to know

- Function Overloading
- Overloading Constructor function
- Copy constructor
- Default function arguments

5.3 Function Overloading

Function overloading is the process of using the same name for two or more functions. The secret to overloading is that each redefinition of the function must use either different types of parameters or a different number of parameters. It is only through these differences that the compiler knows which function to call in any given situation. For example, this program overloads myfunc() by using different types of parameters.

```
#include <iostream.h>

using namespace std;

int myfunc(int i); // these differ in types of parameters
double myfunc(double i);

int main()
{
    cout << myfunc(10) << " "; // calls myfunc(int i)
    cout << myfunc(5.4); // calls myfunc(double i)
    return 0;
}

double myfunc(double i)
{
    return i;
}

int myfunc(int i)
```

```
{
return i;
}
```

The next program overloads myfunc() using a different number of parameters:

```
#include <iostream.h>
using namespace std;
int myfunc(int i); // these differ in number of parameters
int myfunc(int i, int j);
int main()
{
cout << myfunc(10) << " "; // calls myfunc(int i)
cout << myfunc(4, 5); // calls myfunc(int i, int j)
return 0;
}
int myfunc(int i)
{
return i;
}
int myfunc(int i, int j)
{
return i*j;
}
```

As mentioned, the key point about function overloading is that the functions must differ in regard to the types and/or number of parameters. Two functions differing only in their return types cannot be overloaded. For example, this is an invalid attempt to overload myfunc():

```
int myfunc(int i); // Error: differing return types are
float myfunc(int i); // insufficient when overloading.
```

Sometimes, two function declarations will appear to differ, when in fact they do not. For example, consider the following declarations.

```
void f(int *p);  
void f(int p[]); // error, *p is same as p[]
```

Remember, to the compiler `*p` is the same as `p[ ]`. Therefore, although the two prototypes appear to differ in the types of their parameter, in actuality they do not.

5.4 Overloading Constructor Functions

Constructor functions can be overloaded; in fact, overloaded constructors are very common. There are three main reasons why you will want to overload a constructor function: to gain flexibility, to allow both initialized and uninitialized objects to be created, and to define copy constructors. In this section, the first two of these are examined. The following section describes the copy constructor.

5.4.1 Overloading a Constructor to Gain Flexibility

Many times you will create a class for which there are two or more possible ways to construct an object. In these cases, you will want to provide an overloaded constructor function for each way. This is a self-enforcing rule because if you attempt to create an object for which there is no matching constructor, a compile-time error results.

By providing a constructor for each way that a user of your class may plausibly want to construct an object, you increase the flexibility of your class. The user is free to choose the best way to construct an object given the specific circumstance. Consider this program that creates a class called `date`, which holds a calendar date. Notice that the constructor is overloaded two ways:

```
#include <iostream.h>  
#include <cstdio>  
using namespace std;  
class date {  
    int day, month, year;  
public:  
    date(char *d);
```

```

date(int m, int d, int y);
void show_date();
};

// Initialize using string.
date::date(char *d)
{
    sscanf(d, "%d%*c%d%*c%d", &month, &day, &year);
}

// Initialize using integers.
date::date(int m, int d, int y)
{
    day = d;
    month = m;
    year = y;
}

void date::show_date()
{
    cout << month << "/" << day;
    cout << "/" << year << "\n";
}

int main()
{
    date ob1(12, 4, 2001), ob2("10/22/2001");
    ob1.show_date();
    ob2.show_date();
    return 0;
}

```

In this program, you can initialize an object of type `date`, either by specifying the date using three integers to represent the month, day, and year, or by using a string that contains the date in this general form:

`mm/dd/yyyy`

Since both are common ways to represent a date, it makes sense that `date` allows both when constructing an object.

As the `date` class illustrates, perhaps the most common reason to overload a constructor is to allow an object to be created by using the most appropriate and natural means for each particular circumstance. For example, in the following `main()`, the user is prompted for the date, which is input to array `s`. This string can then be used directly to create `d`. There is no need for it to be converted to any other form. However, if `date()` were not overloaded to accept the string form, you would have to manually convert it into three integers.

```
int main()
{
    char s[80];
    cout << "Enter new date: ";
    cin >> s;
    date d(s);
    d.show_date();
    return 0;
}
```

In another situation, initializing an object of type `date` by using three integers may be more convenient. For example, if the date is generated by some sort of computational method, then creating a date object using `date(int, int, int)` is the most natural and appropriate constructor to employ. The point here is that by overloading `date`'s constructor, you have made it more flexible and easier to use. This increased flexibility and ease of use are especially important if you are creating class libraries that will be used by other programmers.

5.5 Copy Constructors

One of the more important forms of an overloaded constructor is the copy constructor. Defining a copy constructor can help you prevent problems that might occur when one object is

used to initialize another. Let's begin by restating the problem that the copy constructor is designed to solve. By default, when one object is used to initialize another, C++ performs a bitwise copy. That is, an identical copy of the initializing object is created in the target object. Although this is perfectly adequate for many cases—and generally exactly what you want to happen—there are situations in which a bitwise copy should not be used. One of the most common is when an object allocates memory when it is created. For example, assume a class called `MyClass` that allocates memory for each object when it is created, and an object `A` of that class. This means that `A` has already allocated its memory. Further, assume that `A` is used to initialize `B`, as shown here:

```
MyClass B= A;
```

If a bitwise copy is performed, then `B` will be an exact copy of `A`. This means that `B` will be using the same piece of allocated memory that `A` is using, instead of allocating its own. Clearly, this is not the desired outcome. For example, if `MyClass` includes a destructor that frees the memory, then the same piece of memory will be freed twice when `A` and `B` are destroyed!

The same type of problem can occur in two additional ways: first, when a copy of an object is made when it is passed as an argument to a function; second, when a temporary object is created as a return value from a function. Remember, temporary objects are automatically created to hold the return value of a function and they may also be created in certain other circumstances.

To solve the type of problem just described, C++ allows you to create a copy constructor, which the compiler uses when one object initializes another. When a copy constructor exists, the default, bitwise copy is bypassed. The most common general form of a copy constructor is

```
classname (const classname &o) {  
    // body of constructor  
}
```

Here, `o` is a reference to the object on the right side of the initialization. It is permissible for a copy constructor to have additional parameters as long as they have default arguments defined for them. However, in all cases the first parameter must be a reference to the object doing the initializing.

It is important to understand that C++ defines two distinct types of situations in which the value of one object is given to another. The first is assignment. The second is initialization, which can occur any of three ways:

1. When one object explicitly initializes another, such as in a declaration
2. When a copy of an object is made to be passed to a function
3. When a temporary object is generated (most commonly, as a return value)

The copy constructor applies only to initializations. For example, assuming a class called `myclass`, and that `y` is an object of type `myclass`, each of the following statements involves initialization.

```
myclass x = y; // y explicitly initializing x
func(y); // y passed as a parameter
y = func(); // y receiving a temporary, return object
```

Following is an example where an explicit copy constructor function is needed. This program creates a very limited “safe” integer array type that prevents array boundaries from being overrun. Storage for each array is allocated by the use of `new`, and a pointer to the memory is maintained within each array object.

`/* This program creates a “safe” array class. Since space for the array is allocated using new, a copy constructor is provided to allocate memory when one array object is used to initialize another.*/`

```
#include <iostream.h>
#include <new>
#include <cstdlib>
using namespace std;
class array {
int *p;
int size;
public:
array(int sz) {
```



```

try {
p = new int[sz];
} catch (bad_alloc xa) {
cout << "Allocation Failure\n";
exit(EXIT_FAILURE);
}
size = sz;
}
~array() { delete [] p; }
// copy constructor
array(const array &a);
void put(int i, int j) {
if(i>=0 && i<size) p[i] = j;
}
int get(int i) {
return p[i];
}
};
// Copy Constructor
array::array(const array &a) {
int i;
try {
p = new int[a.size];
} catch (bad_alloc xa) {
cout << "Allocation Failure\n";
exit(EXIT_FAILURE);
}
for(i=0; i<a.size; i++) p[i] = a.p[i];

```

```

    }
    int main()
    {
        array num(10);
        int i;
        for(i=0; i<10; i++) num.put(i, i);
        for(i=9; i>=0; i--) cout << num.get(i);
        cout << "\n";
        // create another array and initialize with num
        array x(num); // invokes copy constructor
        for(i=0; i<10; i++) cout << x.get(i);
        return 0;
    }

```

Let's look closely at what happens when num is used to initialize x in the statement

```
array x(num); // invokes copy constructor
```

The copy constructor is called, memory for the new array is allocated and stored in x.p, and the contents of num are copied to x's array. In this way, x and num have arrays that contain the same values, but each array is separate and distinct. (That is, num.p and x.p do not point to the same piece of memory.) If the copy constructor had not been created, the default bitwise initialization would have resulted in x and num sharing the same memory for their arrays. (That is, num.p and x.p would have indeed pointed to the same location.)

Remember that the copy constructor is called only for initializations. For example, this sequence does not call the copy constructor defined in the preceding program:

```

array a(10);
// ...
array b(10);
b = a; // does not call copy constructor

```

In this case, `b = a` performs the assignment operation. If `=` is not overloaded (as it is not here), a bitwise copy will be made. Therefore, in some cases, you may need to overload the `=` operator as well as create a copy constructor to avoid certain types of problems.

5.6 Default Function Arguments

C++ allows a function to assign a parameter a default value when no argument corresponding to that parameter is specified in a call to that function. The default value is specified in a manner syntactically similar to a variable initialization. For example, this declares `myfunc()` as taking one double argument with a default value of 0.0:

```
void myfunc(double d = 0.0)
{
    // ...
}
```

Now, `myfunc()` can be called one of two ways, as the following examples show:

```
myfunc(198.234); // pass an explicit value
myfunc();        // let function use default
```

The first call passes the value 198.234 to `d`. The second call automatically gives `d` the default value zero.

One reason that default arguments are included in C++ is because they provide another method for the programmer to manage greater complexity. To handle the widest variety of situations, quite frequently a function contains more parameters than are required for its most common usage. Thus, when the default arguments apply, you need specify only the arguments that are meaningful to the exact situation, not all those needed by the most general case. For example, many of the C++ I/O functions make use of default arguments for just this reason.

A simple illustration of how useful a default function argument can be is shown by the `clrscr()` function in the following program. The `clrscr()` function clears the screen by outputting a series of linefeeds (not the most efficient way, but sufficient for this example). Because a very common video mode displays 25 lines of text, the default argument of 25 is provided. However, because some terminals can display more or less than 25 lines (often depending upon what type of video mode is used), you can override the default argument by specifying one explicitly.

```

#include <iostream.h>
using namespace std;
void clrscr(int size=25);
int main()
{
    register int i;
    for(i=0; i<30; i++ ) cout << i << endl;
    cin.get();
    clrscr(); // clears 25 lines
    for(i=0; i<30; i++ ) cout << i << endl;
    cin.get();
    clrscr(10); // clears 10 lines
    return 0;
}

void clrscr(int size)
{
    for(; size; size--) cout << endl;
}

```

As this program illustrates, when the default value is appropriate to the situation, no argument need be specified when `clrscr()` is called. However, it is still possible to override the default and give `size` a different value when needed.

A default argument can also be used as a flag telling the function to reuse a previous argument. To illustrate this usage, a function called `iputs()` is developed here that automatically indents a string by a specified amount. To begin, here is a version of this function that does not use a default argument:

```

void iputs(char *str, int indent)
{
    if(indent < 0) indent = 0;

```

```

for( ; indent; indent--) cout << " ";
cout << str << "\n";
}

```

This version of `iputs()` is called with the string to output as the first argument and the amount to indent as the second. Although there is nothing wrong with writing `iputs()` this way, you can improve its usability by providing a default argument for the indent parameter that tells `iputs()` to indent to the previously specified level. It is quite common to display a block of text with each line indented the same amount. In this situation, instead of having to supply the same indent argument over and over, you can give indent a default value that tells `iputs()` to indent to the level of the previous call. This approach is illustrated in the following program:

```

#include <iostream.h>
using namespace std;
/* Default indent to -1. This value tells the function
to reuse the previous value. */
void iputs(char *str, int indent = -1);
int main()
{
    iputs("Hello there", 10);
    iputs("This will be indented 10 spaces by default");
    iputs("This will be indented 5 spaces", 5);
    iputs("This is not indented", 0);
    return 0;
}
void iputs(char *str, int indent)
{
    static i = 0; // holds previous indent value
    if(indent >= 0)
        i = indent;
    else // reuse old indent value

```

```

indent = i;
for( ; indent; indent—) cout << “ “;
cout << str << “\n”;
}

```

This program displays this output:

Hello there

This will be indented 10 spaces by default

This will be indented 5 spaces

This is not indented

5.6.1 Function Overloading and Ambiguity

You can create a situation in which the compiler is unable to choose between two (or more) overloaded functions. When this happens, the situation is said to be ambiguous. Ambiguous statements are errors, and programs containing ambiguity will not compile. By far the main cause of ambiguity involves C++’s automatic type conversions. As you know, C++ automatically attempts to convert the arguments used to call a function into the type of arguments expected by the function. For example, consider this fragment:

```

int myfunc(double d);

// ...

cout << myfunc('c'); // not an error, conversion applied

```

As the comment indicates, this is not an error because C++ automatically converts the character `c` into its double equivalent. In C++, very few type conversions of this sort are actually disallowed. Although automatic type conversions are convenient, they are also a prime cause of ambiguity. For example, consider the following program:

```

#include <iostream.h>

using namespace std;

float myfunc(float i);

double myfunc(double i);

int main()

```

```

{
    cout << myfunc(10.1) << " "; // unambiguous, calls myfunc(double)
    cout << myfunc(10); // ambiguous
    return 0;
}

float myfunc(float i)
{
    return i;
}

double myfunc(double i)
{
    return -i;
}

```

Here, `myfunc()` is overloaded so that it can take arguments of either type `float` or type `double`. In the unambiguous line, `myfunc(double)` is called because, unless explicitly specified as `float`, all floating-point constants in C++ are automatically of type `double`. Hence, that call is unambiguous. However, when `myfunc()` is called by using the integer `10`, ambiguity is introduced because the compiler has no way of knowing whether it should be converted to a `float` or to a `double`. This causes an error message to be displayed, and the program will not compile.

As the preceding example illustrates, it is not the overloading of `myfunc()` relative to `double` and `float` that causes the ambiguity. Rather, it is the specific call to `myfunc()` using an indeterminate type of argument that causes the confusion. Put differently, the error is not caused by the overloading of `myfunc()`, but by the specific invocation. Here is another example of ambiguity caused by C++'s automatic type conversions:

```

#include <iostream.h>

using namespace std;

char myfunc(unsigned char ch);
char myfunc(char ch);

int main()

```

```

{
    cout << myfunc('c'); // this calls myfunc(char)
    cout << myfunc(88) << " "; // ambiguous
    return 0;
}

char myfunc(unsigned char ch)
{
    return ch-1;
}

char myfunc(char ch)
{
    return ch+1;
}

```

In C++, unsigned char and char are not inherently ambiguous. However, when myfunc() is called by using the integer 88, the compiler does not know which function to call. That is, should 88 be converted into a char or an unsigned char?

Another way you can cause ambiguity is by using default arguments in overloaded functions. To see how, examine this program:

```

#include <iostream.h>
using namespace std;
int myfunc(int i);
int myfunc(int i, int j=1);
int main()
{
    cout << myfunc(4, 5) << " "; // unambiguous
    cout << myfunc(10); // ambiguous
    return 0;
}

```



```

int myfunc(int i)
{
    return i;
}

int myfunc(int i, int j)
{
    return i*j;
}

```

Here, in the first call to `myfunc()`, two arguments are specified; therefore, no ambiguity is introduced and `myfunc(int i, int j)` is called. However, when the second call to `myfunc()` is made, ambiguity occurs because the compiler does not know whether to call the version of `myfunc()` that takes one argument or to apply the default to the version that takes two arguments. Some types of overloaded functions are simply inherently ambiguous even if, at first, they may not seem so. For example, consider this program.

```

// This program contains an error.

#include <iostream.h>
using namespace std;
void f(int x);
void f(int &x); // error
int main()
{
    int a=10;
    f(a); // error, which f()?
    return 0;
}

void f(int x)
{
    cout << "In f(int)\n";
}

```

```

    }
    void f(int &x)
    {
        cout << "In f(int &)\n";
    }

```

As the comments in the program describe, two functions cannot be overloaded when the only difference is that one takes a reference parameter and the other takes a normal, call-by-value parameter. In this situation, the compiler has no way of knowing which version of the function is intended when it is called. Remember, there is no syntactical difference in the way that an argument is specified when it will be received by a reference parameter or by a value parameter.

5.7 Summary

Function overloading is the process of using the same name for two or more functions. The secret to overloading is that each redefinition of the function must use either different types of parameters or a different number of parameters. It is only through these differences that the compiler knows which function to call in any given situation.

One of the more important forms of an overloaded constructor is the copy constructor. Defining a copy constructor can help you prevent problems that might occur when one object is used to initialize another. C++ allows a function to assign a parameter a default value when no argument corresponding to that parameter is specified in a call to that function. The default value is specified in a manner syntactically similar to a variable initialization.

5.8 Review Questions

1. Define function overloading.
2. Explain about function overloading with example.
3. Discuss about overloading constructor function.
4. Explain about copy constructor.
5. Explain about default function arguments.
6. Discuss about Function Overloading and Ambiguity.

LESSON 6

OPERATOR OVERLOADING

Structure

- 6.1 Introduction**
- 6.2 Learning Objectives**
- 6.3 Operator overloading**
- 6.4 Operator overloading using a friend function**
- 6.5 Overloading some special operators**
- 6.6 Summary**
- 6.7 Review Questions**

6.1 Introduction

In C++ the overloading principle applies not only to functions, but to operators too. That is, operators can be extended to work not just with built-in types but also classes. A programmer can provide his or her own operator to a class by overloading the built-in operator to perform some specific computation when the operator is used on objects of that class. Is operator overloading really useful in real world implementations? It certainly can be, making it very easy to write code that feels natural. On the other hand, operator overloading, like any advanced C++ feature, makes the language more complicated. In addition, operators tend to have very specific meaning, and most programmers don't expect operators to do a lot of work, so overloading operators can be abused to make code unreadable. But we won't do that.

Ada supports overloading of operators from its inception, with the publication of the Ada 83 language standard. However, the designers of the language chose not to permit the definition of new operators: only the existing operators in the language may be overloaded (by defining new functions with identifiers such as "+", "*", "and" etc.). Subsequent revisions of the language (in 1995 and 2005) maintain the restriction to overloading of existing operators.

6.2 Learning Objectives

After learning this lesson you should be able to know as

- Operator overloading
- Operator overloading using a friend function
- Overloading some special operators

-[], () and comma operator

6.3 Operator Overloading

Closely related to function overloading is operator overloading. In C++, you can overload most operators so that they perform special operations relative to classes that you create. For example, a class that maintains a stack might overload + to perform a push operation and – – to perform a pop. When an operator is overloaded, none of its original meanings are lost. Instead, the type of objects it can be applied to is expanded.

The ability to overload operators is one of C++'s most powerful features. It allows the full integration of new class types into the programming environment. After overloading the appropriate operators, you can use objects in expressions in just the same way that you use C++'s built-in data types. Operator overloading also forms the basis of C++'s approach to I/O.

You overload operators by creating operator functions. An operator function defines the operations that the overloaded operator will perform relative to the class upon which it will work. An operator function is created using the keyword operator. Operator functions can be either members or nonmembers of a class. Nonmember operator functions are almost always friend functions of the class, however. The way operator functions are written differs between member and nonmember functions. Therefore, each will be examined separately, beginning with member operator functions.

6.3.1 Creating a Member Operator Function

A member operator function takes this general form:

```
ret-type class-name::operator#(arg-list)
{
    // operations
```

```
}
```

Often, operator functions return an object of the class they operate on, but ret-type can be any valid type. The # is a placeholder. When you create an operator function, substitute the operator for the #. For example, if you are overloading the / operator, use operator/. When you are overloading a unary operator, arg-list will be empty. When you are overloading binary operators, arg-list will contain one parameter. Here is a simple first example of operator overloading. This program creates a class called loc, which stores longitude and latitude values. It overloads the + operator relative to this class. Examine this program carefully, paying special attention to the definition of operator+() :

```
#include <iostream.h>

using namespace std;

class loc {
    int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt) {
        longitude = lg;
        latitude = lt;
    }
    void show() {
        cout << longitude << " ";
        cout << latitude << "\n";
    }
    loc operator+(loc op2);
};

// Overload + for loc.
loc loc::operator+(loc op2)
{
    loc temp;
```

```

temp.longitude = op2.longitude + longitude;
temp.latitude = op2.latitude + latitude;
return temp;
}

int main()
{
loc ob1(10, 20), ob2( 5, 30);
ob1.show(); // displays 10 20
ob2.show(); // displays 5 30
ob1 = ob1 + ob2;
ob1.show(); // displays 15 50
return 0;
}

```

As you can see, `operator+()` has only one parameter even though it overloads the binary `+` operator. (You might expect two parameters corresponding to the two operands of a binary operator.) The reason that `operator+()` takes only one parameter is that the operand on the left side of the `+` is passed implicitly to the function through the `this` pointer. The operand on the right is passed in the parameter `op2`. The fact that the left operand is passed using `this` also implies one important point: When binary operators are overloaded, it is the object on the left that generates the call to the operator function.

As mentioned, it is common for an overloaded operator function to return an object of the class it operates upon. By doing so, it allows the operator to be used in larger expressions. For example, if the `operator+()` function returned some other type, this expression would not have been valid:

```
ob1 = ob1 + ob2;
```

In order for the sum of `ob1` and `ob2` to be assigned to `ob1`, the outcome of that operation must be an object of type `loc`. Further, having `operator+()` return an object of type `loc` makes possible the following statement:

```
(ob1+ob2).show(); // displays outcome of ob1+ob2
```

In this situation, `ob1+ob2` generates a temporary object that ceases to exist after the call to `show()` terminates.

It is important to understand that an operator function can return any type and that the type returned depends solely upon your specific application. It is just that, often, an operator function will return an object of the class upon which it operates. One last point about the `operator+()` function: It does not modify either operand. Because the traditional use of the `+` operator does not modify either operand, it makes sense for the overloaded version not to do so either. (For example, `5+7` yields 12, but neither 5 nor 7 is changed.) Although you are free to perform any operation you want inside an operator function, it is usually best to stay within the context of the normal use of the operator.

The next program adds three additional overloaded operators to the `loc` class: the `-`, the `=`, and the unary `++`. Pay special attention to how these functions are defined.

```
#include <iostream.h>

using namespace std;

class loc {
    int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) {
        longitude = lg;
        latitude = lt;
    }
    void show() {
        cout << longitude << " ";
        cout << latitude << "\n";
    }
    loc operator+(loc op2);
    loc operator-(loc op2);
    loc operator=(loc op2);
```

```

loc operator++();
};
// Overload + for loc.
loc loc::operator+(loc op2)
{
loc temp;
temp.longitude = op2.longitude + longitude;
temp.latitude = op2.latitude + latitude;
return temp;
}
// Overload - for loc.
loc loc::operator-(loc op2)
{
loc temp;
// notice order of operands
temp.longitude = longitude - op2.longitude;
temp.latitude = latitude - op2.latitude;
return temp;
}
// Overload assignment for loc.
loc loc::operator=(loc op2)
{
longitude = op2.longitude;
latitude = op2.latitude;
return *this; // i.e., return object that generated call
}
// Overload prefix ++ for loc.
loc loc::operator++()

```



```

{
    longitude++;
    latitude++;
    return *this;
}

int main()
{
    loc ob1(10, 20), ob2( 5, 30), ob3(90, 90);
    ob1.show();
    ob2.show();
    ++ob1;
    ob1.show(); // displays 11 21
    ob2 = ++ob1;
    ob1.show(); // displays 12 22
    ob2.show(); // displays 12 22
    ob1 = ob2 = ob3; // multiple assignment
    ob1.show(); // displays 90 90
    ob2.show(); // displays 90 90
    return 0;
}

```

First, examine the operator–() function. Notice the order of the operands in the subtraction. In keeping with the meaning of subtraction, the operand on the right side of the minus sign is subtracted from the operand on the left. Because it is the object on the left that generates the call to the operator–() function, op2’s data must be subtracted from the data pointed to by this. It is important to remember which operand generates the call to the function.

In C++, if the = is not overloaded, a default assignment operation is created automatically for any class you define. The default assignment is simply a member-by-member, bitwise copy. By overloading the =, you can define explicitly what the assignment does relative to a class. In this example, the overloaded = does exactly the same thing as the default, but in other

situations, it could perform other operations. Notice that the `operator=()` function returns `*this`, which is the object that generated the call. This arrangement is necessary if you want to be able to use multiple assignment operations such as this:

```
ob1 = ob2 = ob3; // multiple assignment
```

Now, look at the definition of `operator++()`. As you can see, it takes no parameters. Since `++` is a unary operator, its only operand is implicitly passed by using the `'this'` pointer.

Notice that both `operator=()` and `operator++()` alter the value of an operand. In the case of assignment, the operand on the left (the one generating the call to the `operator=()` function) is assigned a new value. In the case of the `++`, the operand is incremented. As stated previously, although you are free to make these functions do anything you please, it is almost always wisest to stay consistent with their original meanings.

6.4 Operator Overloading Using a Friend Function

You can overload an operator for a class by using a nonmember function, which is usually a friend of the class. Since a friend function is not a member of the class, it does not have a `'this'` pointer. Therefore, an overloaded friend operator function is passed the operands explicitly. This means that a friend function that overloads a binary operator has two parameters, and a friend function that overloads a unary operator has one parameter. When overloading a binary operator using a friend function, the left operand is passed in the first parameter and the right operand is passed in the second parameter.

In this program, the `operator+()` function is made into a friend:

```
#include <iostream.h>

using namespace std;

class loc {
    int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) {
        longitude = lg;
        latitude = lt;
```

```

}

void show() {
    cout << longitude << " ";
    cout << latitude << "\n";
}

friend loc operator+(loc op1, loc op2); // now a friend
loc operator-(loc op2);
loc operator=(loc op2);
loc operator++();
};

// Now, + is overloaded using friend function.
loc operator+(loc op1, loc op2)
{
    loc temp;
    temp.longitude = op1.longitude + op2.longitude;
    temp.latitude = op1.latitude + op2.latitude;
    return temp;
}

// Overload - for loc.
loc loc::operator-(loc op2)
{
    loc temp;

    // notice order of operands
    temp.longitude = longitude - op2.longitude;
    temp.latitude = latitude - op2.latitude;
    return temp;
}

// Overload assignment for loc.

```

```

loc loc::operator=(loc op2)
{
    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; // i.e., return object that generated call
}

// Overload ++ for loc.
loc loc::operator++()
{
    longitude++;
    latitude++;
    return *this;
}

int main()
{
    loc ob1(10, 20), ob2( 5, 30);
    ob1 = ob1 + ob2;
    ob1.show();
    return 0;
}

```

There are some restrictions that apply to friend operator functions. First, you may not overload the `=`, `()`, `[]`, or `->` operators by using a friend function. Second, as explained in the next section, when overloading the increment or decrement operators, you will need to use a reference parameter when using a friend function.

6.4.1 Using a Friend to Overload `++` or `--`

If you want to use a friend function to overload the increment or decrement operators, you must pass the operand as a reference parameter. This is because friend functions do not have 'this' pointers. Assuming that you stay true to the original meaning of the `++` and `--` operators, these operations imply the modification of the operand they operate upon. However, if you

overload these operators by using a friend, then the operand is passed by value as a parameter. This means that a friend operator function has no way to modify the operand. Since the friend operator function is not passed a 'this' pointer to the operand, but rather a copy of the operand, no changes made to that parameter affect the operand that generated the call. However, you can remedy this situation by specifying the parameter to the friend operator function as a reference parameter. This causes any changes made to the parameter inside the function to affect the operand that generated the call. For example, this program uses friend functions to overload the prefix versions of ++ and -- operators relative to the loc class:

```
#include <iostream.h>
using namespace std;
class loc {
int longitude, latitude;
public:
loc() {}
loc(int lg, int lt) {
longitude = lg;
latitude = lt;
}
void show() {
cout << longitude << " ";
cout << latitude << "\n";
}
loc operator=(loc op2);
friend loc operator++(loc &op);
friend loc operator--(loc &op);
};
// Overload assignment for loc.
loc loc::operator=(loc op2)
{
```

```

    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; // i.e., return object that generated call
}

// Now a friend; use a reference parameter.
loc operator++(loc &op)
{
    op.longitude++;
    op.latitude++;
    return op;
}

// Make op— a friend; use reference.
loc operator--(loc &op)
{
    op.longitude--;
    op.latitude--;
    return op;
}

int main()
{
    loc ob1(10, 20), ob2;
    ob1.show();
    ++ob1;
    ob1.show(); // displays 11 21
    ob2 = ++ob1;
    ob2.show(); // displays 12 22
    --ob2;
    ob2.show(); // displays 11 21

```

```
return 0;
}
```

If you want to overload the postfix versions of the increment and decrement operators using a friend, simply specify a second, dummy integer parameter. For example, this shows the prototype for the friend, postfix version of the increment operator relative to `loc`.

```
// friend, postfix version of ++
friend loc operator++(loc &op, int x);
```

6.5 Overloading Some Special Operators

C++ defines array subscripting, function calling, and class member access as operations. The operators that perform these functions are the `[]`, `()`, and `->`, respectively. These rather exotic operators may be overloaded in C++, opening up some very interesting uses.

One important restriction applies to overloading these three operators: They must be nonstatic member functions. They cannot be friends.

6.5.1 Overloading `[]`

In C++, the `[]` is considered a binary operator when you are overloading it. Therefore, the general form of a member operator`[]()` function is as shown here:

```
type class-name::operator[](int i)
{
    // ...
}
```

Technically, the parameter does not have to be of type `int`, but an `operator[]()` function is typically used to provide array subscripting, and as such, an integer value is generally used.

Given an object called `O`, the expression `O[3]` translates into this call to the `operator[]()` function:

```
O.operator[] (3)
```

That is, the value of the expression within the subscripting operators is passed to the `operator[]()` function in its explicit parameter. The `this` pointer will point to `O`, the object that

generated the call.

In the following program, `atype` declares an array of three integers. Its constructor function initializes each member of the array to the specified values. The overloaded operator `[]()` function returns the value of the array as indexed by the value of its parameter.

```
#include <iostream.h>
using namespace std;
class atype {
    int a[3];
public:
    atype(int i, int j, int k) {
        a[0] = i;
        a[1] = j;
        a[2] = k;
    }
    int operator[](int i) { return a[i]; }
};
int main()
{
    atype ob(1, 2, 3);
    cout << ob[1]; // displays 2
    return 0;
}
```

You can design the `operator[]()` function in such a way that the `[]` can be used on both the left and right sides of an assignment statement. To do this, simply specify the return value of `operator[]()` as a reference. The following program makes this change and shows its use:

```
#include <iostream.h>
using namespace std;
class atype {
```



```

int a[3];
public:
atype(int i, int j, int k) {
a[0] = i;
a[1] = j;
a[2] = k;
}
int &operator[](int i) { return a[i]; }
};
int main()
{
atype ob(1, 2, 3);
cout << ob[1]; // displays 2
cout << " ";
ob[1] = 25; // [] on left of =
cout << ob[1]; // now displays 25
return 0;
}

```

Because `operator[]()` now returns a reference to the array element indexed by `i`, it can be used on the left side of an assignment to modify an element of the array. One advantage of being able to overload the `[]` operator is that it allows a means of implementing safe array indexing in C++. As you know, in C++, it is possible to overrun (or underrun) an array boundary at run time without generating a run-time error message. However, if you create a class that contains the array, and allow access to that array only through the overloaded `[]` subscripting operator, then you can intercept an out-of-range index. For example, this program adds a range check to the preceding program and proves that it works:

```

// A safe array example.
#include <iostream.h>
#include <cstdlib>

```

```

using namespace std;
class atype {
int a[3];
public:
atype(int i, int j, int k) {
a[0] = i;
a[1] = j;
a[2] = k;
}
int &operator[](int i);
};
// Provide range checking for atype.
int &atype::operator[](int i)
{
if(i<0 || i> 2) {
cout << "Boundary Error\n";
exit(1);
}
return a[i];
}
int main()
{
atype ob(1, 2, 3);
cout << ob[1]; // displays 2
cout << " ";
ob[1] = 25; // [] appears on left
cout << ob[1]; // displays 25
ob[3] = 44; // generates runtime error, 3 out-of-range

```

```
return 0;
}
```

In this program, when the statement

```
ob[3] = 44;
```

executes, the boundary error is intercepted by `operator[]()`, and the program is terminated before any damage can be done.

6.5.2 Overloading ()

When you overload the () function call operator, you are not, per se, creating a new way to call a function. Rather, you are creating an operator function that can be passed an arbitrary number of parameters. Let's begin with an example. Given the overloaded operator function declaration `double operator()(int a, float f, char*s);` and an object `O` of its class, then the statement

```
O(10, 23.34, "hi");
```

Translates into this call to the `operator()` function

```
O.operator()(10, 23.34, "hi");
```

In general, when you overload the () operator, you define the parameters that you want to pass to that function. When you use the () operator in your program, the arguments you specify are copied to those parameters. As always, the object that generates the call (`O` in this example) is pointed to by the 'this' pointer. Here is an example of overloading () for the `loc` class. It assigns the value of its two arguments to the longitude and latitude of the object to which it is applied.

```
#include <iostream.h>
using namespace std;
class loc {
int longitude, latitude;
public:
loc() {}
loc(int lg, int lt) {
longitude = lg;
```

```

latitude = lt;
}

void show() {
cout << longitude << " ";
cout << latitude << "\n";
}

loc operator+(loc op2);
loc operator()(int i, int j);
};

// Overload ( ) for loc.
loc loc::operator()(int i, int j)
{
longitude = i;
latitude = j;
return *this;
}

// Overload + for loc.
loc loc::operator+(loc op2)
{
loc temp;
temp.longitude = op2.longitude + longitude;
temp.latitude = op2.latitude + latitude;
return temp;
}

int main()
{
loc ob1(10, 20), ob2(1, 1);
ob1.show();

```

```

ob1(7, 8); // can be executed by itself
ob1.show();
ob1 = ob2 + ob1(10, 10); // can be used in expressions
ob1.show();
return 0;
}

```

The output produced by the program is shown here.

```
10 20
```

```
7 8
```

```
11 11
```

Remember, when overloading (), you can use any type of parameters and return any type of value. These types will be dictated by the demands of your programs. You can also specify default arguments.

6.5.3 Overloading the Comma Operator

You can overload C++'s comma operator. The comma is a binary operator, and like all overloaded operators, you can make an overloaded comma perform any operation you want. However, if you want the overloaded comma to perform in a fashion similar to its normal operation, then your version must discard the values of all operands except the rightmost. The rightmost value becomes the result of the comma operation. This is the way the comma works by default in C++. Here is a program that illustrates the effect of overloading the comma operator.

```

#include <iostream.h>
using namespace std;
class loc {
int longitude, latitude;
public:
loc() {}
loc(int lg, int lt) {
longitude = lg;

```

```

latitude = lt;
}

void show() {
cout << longitude << " ";
cout << latitude << "\n";
}

loc operator+(loc op2);
loc operator,(loc op2);
};

// overload comma for loc
loc loc::operator,(loc op2)
{
loc temp;
temp.longitude = op2.longitude;
temp.latitude = op2.latitude;
cout << op2.longitude << " " << op2.latitude << "\n";
return temp;
}

// Overload + for loc
loc loc::operator+(loc op2)
{
loc temp;
temp.longitude = op2.longitude + longitude;
temp.latitude = op2.latitude + latitude;
return temp;
}

int main()
{

```

```

loc ob1(10, 20), ob2( 5, 30), ob3(1, 1);
ob1.show();
ob2.show();
ob3.show();
cout << "\n";
ob1 = (ob1, ob2+ob2, ob3);
ob1.show(); // displays 1 1, the value of ob3
return 0;
}

```

This program displays the following output:

```

10 20
5 30
1 1
10 60
1 1
1 1

```

Notice that although the values of the left-hand operands are discarded, each expression is still evaluated by the compiler so that any desired side effects will be performed. Remember, the left-hand operand is passed via this, and its value is discarded by the operator,() function. The value of the right-hand operation is returned by the function. This causes the overloaded comma to behave similarly to its default operation. If you want the overloaded comma to do something else, you will have to change these two features.

6.6 Summary

One of the nice features of C++ is that you can give special meanings to operators, when they are used with user-defined classes. This is called operator overloading. You can implement C++ operator overloads by providing special member-functions on your classes that follow a particular naming convention. For example, to overload the + operator for your class, you would provide a member-function named operator+ on your class.

The following set of operators is commonly overloaded for user-defined classes:

- `=` (assignment operator)
- `+` `-` `*` (binary arithmetic operators)
- `+=` `-=` `*=` (compound assignment operators)
- `==` `!=` (comparison operators)

6.8 Review Questions

1. Define operator overloading.
2. Discuss about operator overloading with example.
3. Explain operator overloading using a friend function.
4. Explain operator overloading in some special operators with example.
5. Explain Overloading the Comma Operator with example.

LESSON 7

INHERITANCE

Structure

- 7.1 Introduction
- 7.2 Learning Objectives
- 7.3 Inheritance
- 7.4 Inheritance and Protected member
- 7.5 Inheriting Multiple Base class
- 7.6 Virtual Base Class
- 7.7 Summary
- 7.8 Review Questions

7.1 Introduction

Inheritance is a mechanism of reusing and extending existing classes without modifying them, thus producing hierarchical relationships between them. Inheritance is almost like embedding an object into a class. Suppose that you declare an object *x* of class *A* in the class definition of *B*. As a result, class *B* will have access to all the public data members and member functions of class *A*. However, in class *B*, you have to access the data members and member functions of class *A* through object *x*.

One of the most important concepts in object-oriented programming is that of inheritance. Inheritance allows us to define a class in terms of another class, which makes it easier to create and maintain an application. This also provides an opportunity to reuse the code functionality and fast implementation time.

When creating a class, instead of writing completely new data members and member functions, the programmer can designate that the new class should inherit the members of an existing class. This existing class is called the base class, and the new class is referred to as the derived class.

The idea of inheritance implements the is a relationship. For example, mammal IS-A animal, dog IS-A mammal hence dog IS-A animal as well and so on.

7.2 Learning Objectives

After learning this lesson you should be able to know

- Inheritance
- Types of inheritance
- Inheritance and Protected member
- Protected base class inheritance
- Inheriting Multiple Base class
- Virtual Base Class

7.3 Inheritance

Inheritance is one of the cornerstones of OOP because it allows the creation of hierarchical classifications. Using inheritance, you can create a general class that defines traits common to a set of related items. This class may then be inherited by other, more specific classes, each adding only those things that are unique to the inheriting class.

In keeping with standard C++ terminology, a class that is inherited is referred to as a base class. The class that does the inheriting is called the derived class. Further, a derived class can be used as a base class for another derived class. In this way, multiple inheritance is achieved. C++'s support of inheritance is both rich and flexible.

7.3.1 Base-Class Access Control

When a class inherits another, the members of the base class become members of the derived class. Class inheritance uses this general form:

```
class derived-class-name : access base-class-name {  
    // body of class  
};
```

The access status of the base-class members inside the derived class is determined by access. The base-class access specifier must be public, private, or protected. If no access specifier is present, the access specifier is private by default if the derived class is a class. If the derived class is a struct, then public is the default in the absence of an explicit access specifier. Let's examine the ramifications of using public or private access.

When the access specifier for a base class is public, all public members of the base become public members of the derived class, and all protected members of the base become protected members of the derived class. In all cases, the base's private elements remain private to the base and are not accessible by members of the derived class. For example, as illustrated in this program, objects of type derived can directly access the public members of base:

```
#include <iostream.h>

using namespace std;

class base {
    int i, j;
    public:
    void set(int a, int b) { i=a; j=b; }
    void show() { cout << i << " " << j << "\n"; }
};

class derived : public base {
    int k;
    public:
    derived(int x) { k=x; }
    void showk() { cout << k << "\n"; }
};

int main()
{
    derived ob(3);
    ob.set(1, 2); // access member of base
    ob.show(); // access member of base
}
```

```

    ob.showk(); // uses member of derived class
    return 0;
}

```

When the base class is inherited by using the private access specifier, all public and protected members of the base class become private members of the derived class. For example, the following program will not even compile because both set() and show() are now private elements of derived:

```

#include <iostream.h>

using namespace std;

class base {
    int i, j;
public:
    void set(int a, int b) { i=a; j=b; }
    void show() { cout << i << " " << j << "\n";}
};

// Public elements of base are private in derived.

class derived : private base {
    int k;
public:
    derived(int x) { k=x; }
    void showk() { cout << k << "\n"; }
};

int main()
{
    derived ob(3);
    ob.set(1, 2); // error, can't access set()
}

```

```

ob.show(); // error, can't access show()

return 0;

}

```

When a base class' access specifier is private, public and protected members of the base become private members of the derived class. This means that they are still accessible by members of the derived class but cannot be accessed by parts of your program that are not members of either the base or derived class.

7.4 Inheritance and protected Members

The protected keyword is included in C++ to provide greater flexibility in the inheritance mechanism. When a member of a class is declared as protected, that member is not accessible by other, nonmember elements of the program. With one important exception, access to a protected member is the same as access to a private member—it can be accessed only by other members of its class. The sole exception to this is when a protected member is inherited. In this case, a protected member differs substantially from a private one.

As explained in the preceding section, a private member of a base class is not accessible by other parts of your program, including any derived class. However, protected members behave differently. If the base class is inherited as public, then the base class' protected members become protected members of the derived class and are, therefore, accessible by the derived class. By using protected, you can create class members that are private to their class but that can still be inherited and accessed by a derived class. Here is an example:

```

#include <iostream.h>

using namespace std;

class base {
protected:
    int i, j; // private to base, but accessible by derived
public:
    void set(int a, int b) { i=a; j=b; }
    void show() { cout << i << " " << j << "\n"; }
};

```

```

class derived : public base {
    int k;
public:
    // derived may access base's i and j
    void setk() { k=i*j; }
    void showk() { cout << k << "\n"; }
};

int main()
{
    derived ob;
    ob.set(2, 3); // OK, known to derived
    ob.show(); // OK, known to derived
    ob.setk();
    ob.showk();
    return 0;
}

```

In this example, because base is inherited by derived as public and because i and j are declared as protected, derived's function setk() may access them. If i and j had been declared as private by base, then derived would not have access to them, and the program would not compile.

When a derived class is used as a base class for another derived class, any protected member of the initial base class that is inherited (as public) by the first derived class may also be inherited as protected again by a second derived class. For example, this program is correct, and derived2 does indeed have access to i and j.

```

#include <iostream.h>

using namespace std;

class base {
protected:

```

```

int i, j;

public:

void set(int a, int b) { i=a; j=b; }

void show() { cout << i << " " << j << "\n"; }

};

// i and j inherited as protected.

class derived1 : public base {

int k;

public:

void setk() { k = i*j; } // legal

void showk() { cout << k << "\n"; }

};

// i and j inherited indirectly through derived1.

class derived2 : public derived1 {

int m;

public:

void setm() { m = i-j; } // legal

void showm() { cout << m << "\n"; }

};

int main()

{

derived1 ob1;

derived2 ob2;

ob1.set(2, 3);

ob1.show();

ob1.setk();

ob1.showk();

ob2.set(3, 4);

```

```

ob2.show();
ob2.setk();
ob2.setm();
ob2.showk();
ob2.showm();
return 0;
}

```

If, however, base were inherited as private, then all members of base would become private members of derived1, which means that they would not be accessible by derived2. (However, i and j would still be accessible by derived1.) This situation is illustrated by the following program, which is in error (and won't compile). The comments describe each error:

```

#include <iostream.h>
using namespace std;
class base {
protected:
int i, j;
public:
void set(int a, int b) { i=a; j=b; }
void show() { cout << i << " " << j << "\n"; }
};
// Now, all elements of base are private in derived1.
class derived1 : private base {
int k;
public:
// this is legal because i and j are private to derived1
void setk() { k = i*j; } // OK
void showk() { cout << k << "\n"; }
};

```



```

// Access to i, j, set(), and show() not inherited.
class derived2 : public derived1 {
    int m;
    public:
    // illegal because i and j are private to derived1
    void setm() { m = i-j; } // Error
    void showm() { cout << m << "\n"; }
};

int main()
{
    derived1 ob1;
    derived2 ob2;
    ob1.set(1, 2); // error, can't use set()
    ob1.show(); // error, can't use show()
    ob2.set(3, 4); // error, can't use set()
    ob2.show(); // error, can't use show()
    return 0;
}

```

Even though base is inherited as private by derived1, derived1 still has access to base's public and protected elements. However, it cannot pass along this privilege.

7.4.1 Protected Base-Class Inheritance

It is possible to inherit a base class as protected. When this is done, all public and protected members of the base class become protected members of the derived class.

For example,

```

#include <iostream.h>
using namespace std;
class base {

```

```

protected:
    int i, j; // private to base, but accessible by derived
public:
    void setij(int a, int b) { i=a; j=b; }
    void showij() { cout << i << " " << j << "\n"; }
};

// Inherit base as protected.
class derived : protected base{
    int k;
public:
    // derived may access base's i and j and setij().
    void setk() { setij(10, 12); k = i*j; }
    // may access showij() here
    void showall() { cout << k << " "; showij(); }
};

int main()
{
    derived ob;
    // ob.setij(2, 3); // illegal, setij() is
    // protected member of derived
    ob.setk(); // OK, public member of derived
    ob.showall(); // OK, public member of derived
    // ob.showij(); // illegal, showij() is protected
    // member of derived
    return 0;
}

```

As you can see by reading the comments, even though `setij()` and `showij()` are public members of base, they become protected members of derived when it is inherited using the protected access specifier. This means that they will not be accessible inside `main()` .

7.5 Inheriting Multiple Base Classes

It is possible for a derived class to inherit two or more base classes. For example, in this short example, derived inherits both `base1` and `base2`.

```
// An example of multiple base classes.

#include <iostream.h>

using namespace std;

class base1 {
protected:
    int x;
public:
    void showx() { cout << x << "\n"; }
};

class base2 {
protected:
    int y;
public:
    void showy() {cout << y << "\n";}
};

// Inherit multiple base classes.

class derived: public base1, public base2 {
public:
    void set(int i, int j) { x=i; y=j; }
};

int main()
{
```

```

    derived ob;
    ob.set(10, 20); // provided by derived
    ob.showx(); // from base1
    ob.showy(); // from base2
    return 0;
}

```

As the example illustrates, to inherit more than one base class, use a comma-separated list. Further, be sure to use an access-specifier for each base inherited.

7.6 Virtual Base Classes

An element of ambiguity can be introduced into a C++ program when multiple base classes are inherited. For example, consider this incorrect program:

```

// This program contains an error and will not compile.

#include <iostream.h>
using namespace std;

class base {
public:
    int i;
};

// derived1 inherits base.
class derived1 : public base {
public:
    int j;
};

// derived2 inherits base.
class derived2 : public base {
public:
    int k;
};

```

```

};
/* derived3 inherits both derived1 and derived2.
This means that there are two copies of base
in derived3! */
class derived3 : public derived1, public derived2 {
public:
    int sum;
};
int main()
{
    derived3 ob;
    ob.i = 10; // this is ambiguous, which i???
    ob.j = 20;
    ob.k = 30;
    // i ambiguous here, too
    ob.sum = ob.i + ob.j + ob.k;
    // also ambiguous, which i?
    cout << ob.i << " ";
    cout << ob.j << " " << ob.k << " ";
    cout << ob.sum;
    return 0;
}

```

As the comments in the program indicate, both derived1 and derived2 inherit base. However, derived3 inherits both derived1 and derived2. This means that there are two copies of base present in an object of type derived3. Therefore, in an expression like

ob.i = 10;

which `i` is being referred to, the one in `derived1` or the one in `derived2`? Because there are two copies of `base` present in object `ob`, there are two `ob.i`s! As you can see, the statement is inherently ambiguous.

There are two ways to remedy the preceding program. The first is to apply the scope resolution operator to `i` and manually select one `i`. For example, this version of the program does compile and run as expected:

```
// This program uses explicit scope resolution to select i.
```

```
#include <iostream.h>
```

```
using namespace std;
```

```
class base {
```

```
public:
```

```
int i;
```

```
};
```

```
// derived1
```

```
class derived1 : public base {
```

```
public:
```

```
int j;
```

```
};
```

```
// derived2 inherits base.
```

```
class derived2 : public base {
```

```
public:
```

```
int k;
```

```
};
```

```
/* derived3 inherits both derived1 and derived2.
```

```
This means that there are two copies of base
```

```
in derived3! */
```

```
class derived3 : public derived1, public derived2 {
```

```
public:
```

```

int sum;

};

int main()
{
    derived3 ob;
    ob.derived1::i = 10; // scope resolved, use derived1's i
    ob.j = 20;
    ob.k = 30;
    // scope resolved
    ob.sum = ob.derived1::i + ob.j + ob.k;
    // also resolved here
    cout << ob.derived1::i << " ";
    cout << ob.j << " " << ob.k << " ";
    cout << ob.sum;
    return 0;
}

```

As you can see, because the `::` was applied, the program has manually selected `derived1`'s version of `base`. However, this solution raises a deeper issue: What if only one copy of `base` is actually required? Is there some way to prevent two copies from being included in `derived3`? The answer, as you probably have guessed, is yes. This solution is achieved using virtual base classes.

When two or more objects are derived from a common base class, you can prevent multiple copies of the base class from being present in an object derived from those objects by declaring the base class as virtual when it is inherited. You accomplish this by preceding the base class' name with the keyword `virtual` when it is inherited. For example, here is another version of the example program in which `derived3` contains only one copy of `base`:

```

// This program uses virtual base classes.

#include <iostream.h>

```

```
using namespace std;

class base {
public:
    int i;
};

// derived1 inherits base as virtual.
class derived1 : virtual public base {
public:
    int j;
};

// derived2 inherits base as virtual.
class derived2 : virtual public base {
public:
    int k;
};

/* derived3 inherits both derived1 and derived2.
This time, there is only one copy of base class. */
class derived3 : public derived1, public derived2 {
public:
    int sum;
};

int main()
{
    derived3 ob;
    ob.i = 10; // now unambiguous
    ob.j = 20;
    ob.k = 30;
    // unambiguous
}
```



```

ob.sum = ob.i + ob.j + ob.k;
// unambiguous
cout << ob.i << " ";
cout << ob.j << " " << ob.k << " ";
cout << ob.sum;
return 0;
}

```

As you can see, the keyword `virtual` precedes the rest of the inherited class' specification. Now that both `derived1` and `derived2` have inherited `base` as `virtual`, any multiple inheritance involving them will cause only one copy of `base` to be present.

Therefore, in `derived3`, there is only one copy of `base` and `ob.i = 10` is perfectly valid and unambiguous. One further point to keep in mind: Even though both `derived1` and `derived2` specify `base` as `virtual`, `base` is still present in objects of either type. For example, the following sequence is perfectly valid:

```

// define a class of type derived1
derived1 myclass;
myclass.i = 88;

```

The only difference between a normal base class and a virtual one is what occurs when an object inherits the base more than once. If virtual base classes are used, then only one base class is present in the object. Otherwise, multiple copies will be found.

7.7 Summary

Inheritance is a mechanism of reusing and extending existing classes without modifying them, thus producing hierarchical relationships between them. Inheritance is almost like embedding an object into a class. Suppose that you declare an object `x` of class `A` in the class definition of `B`. As a result, class `B` will have access to all the public data members and member functions of class `A`. However, in class `B`, you have to access the data members and member functions of class `A` through object `x`.

Inheritance is a concept of linking two or more classes with each other in a hierarchical manner so that their properties and functions can be shared. (One class will extend to another class.) This leads to the biggest advantage of re-usability of the members and avoids redundancy.

Inheritance leads to various issues such as:

- What is Inherited
- types of inheritance
- accessibility modes (public, private and protected) in inheritance
- Friend function and inheritance etc

7.8 Review Questions

1. Define inheritance.
2. List out the different types of inheritance.
3. Explain about inheritance and protected members.
4. Discuss about inheritance multiple base class.
5. Explain about virtual base class with example program.

LESSON 8

POLYMORPHISM AND VIRTUAL FUNCTIONS

Structure

- 8.1 Introduction**
- 8.2 Learning Objectives**
- 8.3 Virtual Functions**
- 8.4 Pure Virtual Functions**
- 8.5 Abstract Class**
- 8.6 Templates**
- 8.7 Summary**
- 8.8 Review Questions**

8.1 Introduction

Polymorphism is supported by C++ both at compile time and at run time. As discussed in earlier lessons, compile-time polymorphism is achieved by overloading functions and operators. Run-time polymorphism is accomplished by using inheritance and virtual functions, and these are the topics of this lesson.

A class that contains at least one pure virtual function is said to be abstract. Because an abstract class contains one or more functions for which there is no definition (that is, a pure virtual function), no objects of an abstract class may be created. Instead, an abstract class constitutes an incomplete type that is used as a foundation for derived classes.

8.2 Learning Objectives

After learning this lesson you should be able to know

- Polymorphism
- Virtual functions
- Abstract Class
- Templates

- Generic functions
- Generic classes

8.3 Virtual Functions

A virtual function is a member function that is declared within a base class and redefined by a derived class. To create a virtual function, precede the function's declaration in the base class with the keyword `virtual`. When a class containing a virtual function is inherited, the derived class redefines the virtual function to fit its own needs. In essence, virtual functions implement the “one interface, multiple methods” philosophy that underlies polymorphism. The virtual function within the base class defines the form of the interface to that function. Each redefinition of the virtual function by a derived class implements its operation as it relates specifically to the derived class. That is, the redefinition creates a specific method.

When accessed “normally,” virtual functions behave just like any other type of class member function. However, what makes virtual functions important and capable of supporting run-time polymorphism is how they behave when accessed via a pointer.

A base-class pointer can be used to point to an object of any class derived from that base. When a base pointer points to a derived object that contains a virtual function, C++ determines which version of that function to call based upon the type of object pointed to by the pointer. And this determination is made at run time. Thus, when different objects are pointed to, different versions of the virtual function are executed. The same effect applies to base-class references.

To begin, examine this short example:

```
#include <iostream.h>

using namespace std;

class base {
public:
    virtual void vfunc() {
        cout << "This is base's vfunc().\n";
    }
};
```

```
class derived1 : public base {
public:
void vfunc() {
cout << "This is derived1's vfunc().\n";
}
};

class derived2 : public base {
public:
void vfunc() {
cout << "This is derived2's vfunc().\n";
}
};

int main()
{
base *p, b;
derived1 d1;
derived2 d2;
// point to base
p = &b;
p->vfunc(); // access base's vfunc()
// point to derived1
p = &d1;
p->vfunc(); // access derived1's vfunc()
// point to derived2
p = &d2;
p->vfunc(); // access derived2's vfunc()
return 0;
}
```

This program displays the following:

This is base's vfunc().

This is derived1's vfunc().

This is derived2's vfunc().

As the program illustrates, inside base, the virtual function vfunc() is declared. Notice that the keyword virtual precedes the rest of the function declaration. When vfunc() is redefined by derived1 and derived2, the keyword virtual is not needed. In this program, base is inherited by both derived1 and derived2. Inside each class definition, vfunc() is redefined relative to that class. Inside main(), four variables are declared:

Name Type

p base class pointer

b object of base

d1 object of derived1

d2 object of derived2

Next, p is assigned the address of b, and vfunc() is called via p. Since p is pointing to an object of type base, that version of vfunc() is executed. Next, p is set to the address of d1, and again vfunc() is called by using p. This time p points to an object of type derived1. This causes derived1::vfunc() to be executed. Finally, p is assigned the address of d2, and p->vfunc() causes the version of vfunc() redefined inside derived2 to be executed. The key point here is that the kind of object to which p points determines which version of vfunc() is executed. Further, this determination is made at run time, and this process forms the basis for run-time polymorphism.

Although you can call a virtual function in the “normal” manner by using an object's name and the dot operator, it is only when access is through a base-class pointer (or reference) that run-time polymorphism is achieved. For example, assuming the preceding example, this is syntactically valid:

```
d2.vfunc(); // calls derived2's vfunc()
```

Although calling a virtual function in this manner is not wrong, it simply does not take advantage of the virtual nature of vfunc() .

At first glance, the redefinition of a virtual function by a derived class appears similar to function overloading. However, this is not the case, and the term overloading is not applied to virtual function redefinition because several differences exist. Perhaps the most important is that the prototype for a redefined virtual function must match exactly the prototype specified in the base class. This differs from overloading a normal function, in which return types and the number and type of parameters may differ. (In fact, when you overload a function, either the number or the type of the parameters must differ! It is through these differences that C++ can select the correct version of an overloaded function.) However, when a virtual function is redefined, all aspects of its prototype must be the same. If you change the prototype when you attempt to redefine a virtual function, the function will simply be considered overloaded by the C++ compiler and its virtual nature will be lost. Another important restriction is that virtual functions must be nonstatic members of the classes of which they are part. They cannot be friends. Finally, constructor functions cannot be virtual, but destructor functions can. Because of the restrictions and differences between function overloading and virtual function redefinition, the term overriding is used to describe virtual function redefinition by a derived class.

8.3.1 Calling a Virtual Function through a Base Class Reference

In the preceding example, a virtual function was called through a base-class pointer, but the polymorphic nature of a virtual function is also available when called through a base-class reference. A reference is an implicit pointer. Thus, a base-class reference can be used to refer to an object of the base class or any object derived from that base. When a virtual function is called through a base-class reference, the version of the function executed is determined by the object being referred to at the time of the call.

The most common situation in which a virtual function is invoked through a base class reference is when the reference is a function parameter. For example, consider the following variation on the preceding program.

```
/* Here, a base class reference is used to access
a virtual function. */
#include <iostream.h>
using namespace std;
class base {
```

```
public:
virtual void vfunc() {
    cout << "This is base's vfunc().\n";
}
};

class derived1 : public base {
public:
    void vfunc() {
        cout << "This is derived1's vfunc().\n";
    }
};

class derived2 : public base {
public:
    void vfunc() {
        cout << "This is derived2's vfunc().\n";
    }
};

// Use a base class reference parameter.
void f(base &r) {
    r.vfunc();
}

int main()
{
    base b;
    derived1 d1;
    derived2 d2;
    f(b); // pass a base object to f()
```



```

f(d1); // pass a derived1 object to f()

f(d2); // pass a derived2 object to f()

return 0;

}

```

This program produces the same output as its preceding version. In this example, the function `f()` defines a reference parameter of type `base`. Inside `main()`, the function is called using objects of type `base`, `derived1`, and `derived2`. Inside `f()`, the specific version of `vfunc()` that is called is determined by the type of object being referenced when the function is called.

8.4 Pure Virtual Functions

As the examples in the preceding section illustrate, when a virtual function is not redefined by a derived class, the version defined in the base class will be used. However, in many situations there can be no meaningful definition of a virtual function within a base class. For example, a base class may not be able to define an object sufficiently to allow a base-class virtual function to be created. Further, in some situations you will want to ensure that all derived classes override a virtual function.

To handle these two cases, C++ supports the pure virtual function. A pure virtual function is a virtual function that has no definition within the base class. To declare a pure virtual function, use this general form:

```
virtual type func-name(parameter-list) = 0;
```

When a virtual function is made pure, any derived class must provide its own definition. If the derived class fails to override the pure virtual function, a compile-time error will result.

The following program contains a simple example of a pure virtual function. The base class, `number`, contains an integer called `val`, the function `setval()`, and the pure virtual function `show()`. The derived classes `hextype`, `dectype`, and `octtype` inherit `number` and redefine `show()` so that it outputs the value of `val` in each respective number base (that is, hexadecimal, decimal, or octal).

```

#include <iostream.h>

using namespace std;

class number {

```

```

protected:
    int val;
public:
    void setval(int i) { val = i; }
    // show() is a pure virtual function
    virtual void show() = 0;
};

class hextype : public number {
public:
    void show() {
        cout << hex << val << "\n";
    }
};

class dectype : public number {
public:
    void show() {
        cout << val << "\n";
    }
};

class octtype : public number {
public:
    void show() {
        cout << oct << val << "\n";
    }
};

int main()
{
    dectype d;

```

```

hextype h;
octtype o;
d.setval(20);
d.show(); // displays 20 - decimal
h.setval(20);
h.show(); // displays 14 – hexadecimal
o.setval(20);
o.show(); // displays 24 - octal
return 0;
}

```

Although this example is quite simple, it illustrates how a base class may not be able to meaningfully define a virtual function. In this case, `number` simply provides the common interface for the derived types to use. There is no reason to define `show()` inside `number` since the base of the `number` is undefined. Of course, you can always create a placeholder definition of a virtual function. However, making `show()` pure also ensures that all derived classes will indeed redefine it to meet their own needs. Keep in mind that when a virtual function is declared as pure, all derived classes must override it. If a derived class fails to do this, a compile-time error will result.

8.5 Abstract Classes

A class that contains at least one pure virtual function is said to be abstract. Because an abstract class contains one or more functions for which there is no definition (that is, a pure virtual function), no objects of an abstract class may be created. Instead, an abstract class constitutes an incomplete type that is used as a foundation for derived classes.

Although you cannot create objects of an abstract class, you can create pointers and references to an abstract class. This allows abstract classes to support run-time polymorphism, which relies upon base-class pointers and references to select the proper virtual function.

8.6 Templates

The template is one of C++'s most sophisticated and high-powered features. Although not part of the original specification for C++, it was added several years ago and is supported

by all modern C++ compilers. Using templates, it is possible to create generic functions and classes. In a generic function or class, the type of data upon which the function or class operates is specified as a parameter. Thus, you can use one function or class with several different types of data without having to explicitly recode specific versions for each data type.

8.6.1 Generic Functions

A generic function defines a general set of operations that will be applied to various types of data. The type of data that the function will operate upon is passed to it as a parameter. Through a generic function, a single general procedure can be applied to a wide range of data. As you probably know, many algorithms are logically the same no matter what type of data is being operated upon. For example, the Quick sort sorting algorithm is the same whether it is applied to an array of integers or an array of floats. It is just that the type of the data being sorted is different. By creating a generic function, you can define the nature of the algorithm, independent of any data. Once you have done this, the compiler will automatically generate the correct code for the type of data that is actually used when you execute the function. In essence, when you create a generic function you are creating a function that can automatically overload itself.

A generic function is created using the keyword `template`. The normal meaning of the word “template” accurately reflects its use in C++. It is used to create a template (or framework) that describes what a function will do, leaving it to the compiler to fill in the details as needed. The general form of a template function definition is shown here:

```
template <class Ttype> ret-type func-name(parameter list)
{
    // body of function
}
```

Here, `Ttype` is a placeholder name for a data type used by the function. This name may be used within the function definition. However, it is only a placeholder that the compiler will automatically replace with an actual data type when it creates a specific version of the function. Although the use of the keyword `class` to specify a generic type in a template declaration is traditional, you may also use the keyword `typename`.

The following example creates a generic function that swaps the values of the two variables with which it is called. Because the general process of exchanging two values is independent of the type of the variables, it is a good candidate for being made into a generic function.

```

// Function template example.
#include <iostream.h>

using namespace std;

// This is a function template.
template <class X> void swapargs(X &a, X &b)
{
    X temp;
    temp = a;
    a = b;
    b = temp;
}

int main()
{
    int i=10, j=20;
    double x=10.1, y=23.3;
    char a='x', b='z';
    cout << "Original i, j: " << i << ' ' << j << '\n';
    cout << "Original x, y: " << x << ' ' << y << '\n';
    cout << "Original a, b: " << a << ' ' << b << '\n';
    swapargs(i, j); // swap integers
    swapargs(x, y); // swap floats
    swapargs(a, b); // swap chars
    cout << "Swapped i, j: " << i << ' ' << j << '\n';
    cout << "Swapped x, y: " << x << ' ' << y << '\n';
    cout << "Swapped a, b: " << a << ' ' << b << '\n';
    return 0;
}

```

Let's look closely at this program. The line:

```
template <class X> void swapargs(X &a, X &b)
```

tells the compiler two things: that a template is being created and that a generic definition is beginning. Here, X is a generic type that is used as a placeholder. After the template portion, the function `swapargs()` is declared, using X as the data type of the values that will be swapped. In `main()`, the `swapargs()` function is called using three different types of data: ints, doubles, and chars. Because `swapargs()` is a generic function, the compiler automatically creates three versions of `swapargs()`: one that will exchange integer values, one that will exchange floating-point values, and one that will swap characters.

Here are some important terms related to templates. First, a generic function (that is, a function definition preceded by a template statement) is also called a template function. Both terms will be used interchangeably in this book. When the compiler creates a specific version of this function, it is said to have created a specialization. This is also called a generated function. The act of generating a function is referred to as instantiating it. Put differently, a generated function is a specific instance of a template function. Since C++ does not recognize end-of-line as a statement terminator, the template clause of a generic function definition does not have to be on the same line as the function's name. The following example shows another common way to format the

`swapargs()` function.

```
template <class X>
void swapargs(X &a, X &b)
{
    X temp;
    temp = a;
    a = b;
    b = temp;
}
```

If you use this form, it is important to understand that no other statements can occur between the template statement and the start of the generic function definition. For example, the fragment shown next will not compile.

```
// This will not compile.
```

```

template <class X>
int i; // this is an error
void swapargs(X &a, X &b)
{
    X temp;
    temp = a;
    a = b;
    b = temp;
}

```

As the comments imply, the template specification must directly precede the function definition.

8.6.2 Overloading a Function Template

In addition to creating explicit, overloaded versions of a generic function, you can also overload the template specification itself. To do so, simply create another version of the template that differs from any others in its parameter list. For example:

```

// Overload a function template declaration.
#include <iostream.h>
using namespace std;
// First version of f() template.
template <class X> void f(X a)
{
    cout << "Inside f(X a)\n";
}
// Second version of f() template.
template <class X, class Y> void f(X a, Y b)
{
    cout << "Inside f(X a, Y b)\n";
}

```

```

int main()
{
    f(10); // calls f(X)
    f(10, 20); // calls f(X, Y)
    return 0;
}

```

Here, the template for `f()` is overloaded to accept either one or two parameters.

8.6.3 Generic Classes

In addition to generic functions, you can also define a generic class. When you do this, you create a class that defines all the algorithms used by that class; however, the actual type of the data being manipulated will be specified as a parameter when objects of that class are created.

Generic classes are useful when a class uses logic that can be generalized. For example, the same algorithms that maintain a queue of integers will also work for a queue of characters, and the same mechanism that maintains a linked list of mailing addresses will also maintain a linked list of auto part information. When you create a generic class, it can perform the operation you define, such as maintaining a queue or a linked list, for any type of data. The compiler will automatically generate the correct type of object, based upon the type you specify when the object is created. The general form of a generic class declaration is shown here:

```

template <class Ttype> class class-name {
    .
    ..
}

```

Here, `Ttype` is the placeholder type name, which will be specified when a class is instantiated. If necessary, you can define more than one generic data type using a comma-separated list.

Once you have created a generic class, you create a specific instance of that class using the following general form:


```
class-name <type> ob;
```

Here, type is the type name of the data that the class will be operating upon. Member functions of a generic class are themselves automatically generic. You need not use template to explicitly specify them as such.

In the following program, the stack class is reworked into a generic class. Thus, it can be used to store objects of any type. In this example, a character stack and a floating-point stack are created, but any data type can be used.

```
// This function demonstrates a generic stack.
#include <iostream.h>
using namespace std;
const int SIZE = 10;
// Create a generic stack class
template <class StackType> class stack {
    StackType stck[SIZE]; // holds the stack
    int tos; // index of top-of-stack
public:
    stack() { tos = 0; } // initialize stack
    void push(StackType ob); // push object on stack
    StackType pop(); // pop object from stack
};
// Push an object.
template <class StackType> void stack<StackType>::push(StackType ob)
{
    if(tos==SIZE) {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = ob;
```

```

    tos++;
}
// Pop an object.
template <class StackType> StackType stack<StackType>::pop()
{
    if(tos==0) {
        cout << "Stack is empty.\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{
    // Demonstrate character stacks.
    stack<char> s1, s2; // create two character stacks
    int i;
    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');
    for(i=0; i<3; i++) cout << "Pop s1: " << s1.pop() << "\n";
    for(i=0; i<3; i++) cout << "Pop s2: " << s2.pop() << "\n";
    // demonstrate double stacks
    stack<double> ds1, ds2; // create two double stacks

```

```

ds1.push(1.1);
ds2.push(2.2);
ds1.push(3.3);
ds2.push(4.4);
ds1.push(5.5);
ds2.push(6.6);

for(i=0; i<3; i++) cout << "Pop ds1: " << ds1.pop() << "\n";
for(i=0; i<3; i++) cout << "Pop ds2: " << ds2.pop() << "\n";

return 0;
}

```

As you can see, the declaration of a generic class is similar to that of a generic function. The actual type of data stored by the stack is generic in the class declaration. It is not until an object of the stack is declared that the actual data type is determined. When a specific instance of stack is declared, the compiler automatically generates all the functions and variables necessary for handling the actual data. In this example, two different types of stacks are declared. Two are integer stacks. Two are stacks of doubles. Pay special attention to these declarations:

```

stack<char> s1, s2; // create two character stacks

stack<double> ds1, ds2; // create two double stacks

```

Notice how the desired data type is passed inside the angle brackets. By changing the type of data specified when stack objects are created, you can change the type of data stored in that stack. For example, by using the following declaration, you can create another stack that stores character pointers.

```

stack<char *> chrptrQ;

```

You can also create stacks to store data types that you create. For example, if you want to use the following structure to store address information,

```

struct addr {
    char name[40];

```

```
char street[40];  
char city[30];  
char state[3];  
char zip[12];  
};
```

then to use `stack` to generate a stack that will store objects of type `addr`, use a declaration like this:

```
stack<addr> obj;
```

As the `stack` class illustrates, generic functions and classes are powerful tools that you can use to maximize your programming efforts, because they allow you to define the general form of an object that can then be used with any type of data. You are saved from the tedium of creating separate implementations for each data type with which you want the algorithm to work.

8.6.4 The Power of Templates

Templates help you achieve one of the most elusive goals in programming: the creation of reusable code. Through the use of template classes you can create frameworks that can be applied over and over again to a variety of programming situations. For example, consider the `stack` class. It could only be used to store integer values. Even though the underlying algorithms could be used to store any type of data, the hard-coding of the data type into the `stack` class severely limited its application. However, by making `stack` into a generic class, it can create a stack for any type of data.

Generic functions and classes provide a powerful tool that you can use to amplify your programming efforts. Once you have written and debugged a template class, you have a solid software component that you can use with confidence in a variety of different situations. You are saved from the tedium of creating separate implementations for each data type with which you want the class to work.

While it is true that the template syntax can seem a bit intimidating at first, the rewards are well worth the time it takes to become comfortable with it. Template functions and classes are already becoming commonplace in programming, and this trend is expected to continue. For example, the STL (Standard Template Library) defined by C++ is, as its name implies, built upon templates. One last point: although templates add a layer of abstraction, they still ultimately compile down to the same, high-performance object code that you have come to expect from C++.

8.7 Summary

A virtual function is a member function that is declared within a base class and redefined by a derived class. To create a virtual function, precede the function's declaration in the base class with the keyword `virtual`. When a class containing a virtual function is inherited, the derived class redefines the virtual function to fit its own needs. In essence, virtual functions implement the "one interface, multiple methods" philosophy that underlies polymorphism. The virtual function within the base class defines the form of the interface to that function. Each redefinition of the virtual function by a derived class implements its operation as it relates specifically to the derived class. That is, the redefinition creates a specific method.

A generic function defines a general set of operations that will be applied to various types of data. The type of data that the function will operate upon is passed to it as a parameter. Through a generic function, a single general procedure can be applied to a wide range of data. As you probably know, many algorithms are logically the same no matter what type of data is being operated upon. For example, the Quick sort sorting algorithm is the same whether it is applied to an array of integers or an array of floats. It is just that the type of the data being sorted is different. By creating a generic function, you can define the nature of the algorithm, independent of any data. Once you have done this, the compiler will automatically generate the correct code for the type of data that is actually used when you execute the function. In essence, when you create a generic function you are creating a function that can automatically overload itself.

8.8 Review questions

1. Define Polymorphism.
2. Explain about virtual functions with an example.
3. Discuss about calling a virtual function through a base class reference.
4. Explain about pure virtual functions.
5. Define abstract class. Explain in detail.
6. Define template. Explain in detail.
7. What is meant by generic functions? Explain.
8. What is meant by generic classes? Explain.
9. Discuss about the power of template.

LESSON 9

EXCEPTION HANDLING

Structure

- 9.1 Introduction
- 9.2 Learning Objectives
- 9.3 Exception Handling Fundamentals
- 9.4 Understanding about `terminate()` & `unexpected`
- 9.5 Uncaught `exception()` function
- 9.6 Applying exception handling
- 9.7 Summary
- 9.8 Review Questions

9.1 Introduction

This lesson discusses the exception handling subsystem. Exception handling allows you to manage run-time errors in an orderly fashion. Using exception handling, your program can automatically invoke an error-handling routine when an error occurs. The principal advantage of exception handling is that it automates much of the error-handling code that previously had to be coded “by hand” in any large program.

Exception handling is a mechanism that separates code that detects and handles exceptional circumstances from the rest of your program. Note that an exceptional circumstance is not necessarily an error.

When a function detects an exceptional situation, you represent this with an object. This object is called an exception object. In order to deal with the exceptional situation you throw the exception. This passes control, as well as the exception, to a designated block of code in a direct or indirect caller of the function that threw the exception. This block of code is called a handler. In a handler, you specify the types of exceptions that it may process. The C++ run time, together with the generated code, will pass control to the first appropriate handler that is able to process the exception thrown. When this happens, an exception is caught. A handler may rethrow an exception so it can be caught by another handler.

The exception handling mechanism is made up of the following elements:

- try blocks
- catch blocks
- throw expressions
- Exception specifications

9.2 Learning Objectives

After learning this lesson you should be able to know as

- Exception Handling Fundamentals
- Understanding about terminate() & unexpected
- Uncaught exception() function
- Applying exception handling

9.3 Exception Handling Fundamentals

C++ exception handling is built upon three keywords: try, catch, and throw. In the most general terms, program statements that you want to monitor for exceptions are contained in a try block. If an exception (i.e., an error) occurs within the try block, it is thrown (using throw). The exception is caught, using catch, and processed. The following discussion elaborates upon this general description.

Code that you want to monitor for exceptions must have been executed from within a try block. (Functions called from within a try block may also throw an exception.) Exceptions that can be thrown by the monitored code are caught by a catch statement, which immediately follows the try statement in which the exception was thrown. The general form of try and catch are shown here.

```
try {  
    // try block  
}  
  
catch (type1 arg) {  
    // catch block
```



```

}
catch (type2 arg) {
    // catch block
}
catch (type3 arg) {
    // catch block
}
..
.
catch (typeN arg) {
    // catch block
}

```

The try can be as short as a few statements within one function or as all encompassing as enclosing the main() function code within a try block (which effectively causes the entire program to be monitored). When an exception is thrown, it is caught by its corresponding catch statement, which processes the exception. There can be more than one catch statement associated with a try. Which catch statement is used is determined by the type of the exception. That is, if the data type specified by a catch matches that of the exception, then that catch statement is executed (and all others are bypassed). When an exception is caught, arg will receive its value. Any type of data may be caught, including classes that you create. If no exception is thrown (that is, no error occurs within the try block), then no catch statement is executed.

The general form of the throw statement is shown here:

```
throw exception;
```

throw generates the exception specified by exception. If this exception is to be caught, then throw must be executed either from within a try block itself, or from any function called from within the try block (directly or indirectly).

If you throw an exception for which there is no applicable catch statement, an abnormal program termination may occur. Throwing an unhandled exception causes the standard library function terminate() to be invoked. By default, terminate() calls abort() to stop your program, but

you can specify your own termination handler. Here is a simple example that shows the way C++ exception handling operates.

```
// A simple exception handling example.
#include <iostream.h>
using namespace std;
int main()
{
    cout << "Start\n";
    try { // start a try block
        cout << "Inside try block\n";
        throw 100; // throw an error
        cout << "This will not execute";
    }
    catch (int i) { // catch an error
        cout << "Caught an exception — value is: ";
        cout << i << "\n";
    }
    cout << "End";
    return 0;
}
```

This program displays the following output:

Start

Inside try block

Caught an exception — value is: 100

End

Look carefully at this program. As you can see, there is a try block containing three statements and a catch(int i) statement that processes an integer exception. Within the try block, only two of the three statements will execute: the first cout statement and the throw.

Once an exception has been thrown, control passes to the catch expression and the try block is terminated. That is, catch is not called. Rather, program execution is transferred to it. (The program's stack is automatically reset as needed to accomplish this.) Thus, the cout statement following the throw will never execute.

Usually, the code within a catch statement attempts to remedy an error by taking appropriate action. If the error can be fixed, execution will continue with the statements following the catch. However, often an error cannot be fixed and a catch block will terminate the program with a call to `exit()` or `abort()` .

As mentioned, the type of the exception must match the type specified in a catch statement. For example, in the preceding example, if you change the type in the catch statement to `double`, the exception will not be caught and abnormal termination will occur. This change is shown here.

```
#include <iostream.h>
using namespace std;
int main()
{
    cout << "Start\n";
    try { // start a try block
        cout << "Inside try block\n";
        throw 100; // throw an error
        cout << "This will not execute";
    }
    catch (double i) { // won't work for an int exception
        cout << "Caught an exception — value is: ";
        cout << i << "\n";
    }
    cout << "End";
    return 0;
}
```

This program produces the following output because the integer exception will not be caught by the catch(double i) statement.

Start

Inside try block

Abnormal program termination

An exception can be thrown from outside the try block as long as it is thrown by a function that is called from within try block. For example, this is a valid program.

```
/* Throwing an exception from a function outside the
try block.
```

```
*/
```

```
#include <iostream.h>
```

```
using namespace std;
```

```
void Xtest(int test)
```

```
{
```

```
cout << "Inside Xtest, test is: " << test << "\n";
```

```
if(test) throw test;
```

```
}
```

```
int main()
```

```
{
```

```
cout << "Start\n";
```

```
try { // start a try block
```

```
cout << "Inside try block\n";
```

```
Xtest(0);
```

```
Xtest(1);
```

```
Xtest(2);
```

```
}
```

```
catch (int i) { // catch an error
```

```

cout << "Caught an exception — value is: ";
cout << i << "\n";
}
cout << "End";
return 0;
}

```

This program produces the following output:

Start

Inside try block

Inside Xtest, test is: 0

Inside Xtest, test is: 1

Caught an exception — value is: 1

End

A try block can be localized to a function. When this is the case, each time the function is entered, the exception handling relative to that function is reset. For example, examine this program.

```

#include <iostream.h>
using namespace std;
// Localize a try/catch to a function.
void Xhandler(int test)
{
    try{
        if(test) throw test;
    }
    catch(int i) {
        cout << "Caught Exception #: " << i << '\n';
    }
}

```

```

int main()
{
    cout << "Start\n";
    Xhandler(1);
    Xhandler(2);
    Xhandler(0);
    Xhandler(3);
    cout << "End";
    return 0;
}

```

This program displays this output:

```

Start
Caught Exception #: 1
Caught Exception #: 2
Caught Exception #: 3
End

```

As you can see, three exceptions are thrown. After each exception, the function returns. When the function is called again, the exception handling is reset. It is important to understand that the code associated with a catch statement will be executed only if it catches an exception. Otherwise, execution simply bypasses the catch altogether. (That is, execution never flows into a catch statement.) For example, in the following program, no exception is thrown, so the catch statement does not execute.

```

#include <iostream.h>
using namespace std;
int main()
{
    cout << "Start\n";
    try { // start a try block

```

```

cout << "Inside try block\n";
cout << "Still inside try block\n";
}
catch (int i) { // catch an error
cout << "Caught an exception — value is: ";
cout << i << "\n";
}
cout << "End";
return 0;
}

```

The preceding program produces the following output.

Start

Inside try block

Still inside try block

End

As you see, the catch statement is bypassed by the flow of execution.

9.3.1 Using Multiple catch Statements

The exceptions as we have seen so far dealt with a single exception in a program. Most of the time, a typical program will throw different types of errors. The C++ language allows you to include different catch blocks. Each catch block can face a specific error. The syntax used is:

```

try {
    Code to Try
}
catch(Arg1)
{
    One Exception
}

```

```

    catch(Arg2)
    {
        Another Exception
    }

```

The compiler would proceed in a top-down as follows:

1. Following the normal flow control of the program, the compiler enters the try block.
2. If no exception occurs in the try block, the rest of the try block is executed. If an exception occurs in the try block, the try displays a throw that specifies the type of error that happened.
 - a. The compiler gets out of the try block and examines the first catch
 - b. If the first catch doesn't match the thrown error, the compiler proceeds with the next catch. This continues until the compiler finds a catch that matches the thrown error.
 - c. If one of the catches matches the thrown error, its body executes. If no catch matches the thrown error, you have (or the compiler has) two alternatives. If there is no catch that matches the error (which means that you didn't provide a matching catch), the compiler hands the program flow to the operating system (which calls the `terminate()` function). Another alternative is to include a catch whose argument is three periods: `catch(...)`. The `catch(...)` is used if no other catch, provided there was another, matches the thrown error. The `catch(...)`, if included as part of a catch clause, must always be the last catch, unless it is the only catch of the clause.

Multiple catches are written if or when a try block is expected to throw different types of errors. Imagine a program that requests some numbers from the user and performs some operation on the numbers. Such a program can be written as follows.

```

#include <iostream.h.h>

int main()
{
    double Operand1, Operand2, Result;
    char Operator;

```



```
cout << "This program allows you to perform an operation on two
numbers\n";

cout << "To proceed, enter a number, an operator, and a number:\n";
cin >> Operand1 >> Operator >> Operand2;

switch(Operator)
{
case '+':
    Result = Operand1 + Operand2;
    break;

case '-':
    Result = Operand1 - Operand2;
    break;

case '*':
    Result = Operand1 * Operand2;
    break;

case '/':
    Result = Operand1 / Operand2;
    break;

default:
    cout << "Bad Operation";
}
```

```

        cout << "\n" << Operand1 << " " << Operator << " "
            << Operand2 << " = " << Result;

        cout << "\n\n";
        return 0;
    }

```

This program works fine as long as the user types a valid sequence of values made of a number followed by a valid arithmetic operator, followed by a number. Anything else, such as an invalid number, an unexpected operator, or a wrong sequence (such as Number Number Operator), would produce an unpredictable outcome. Obviously various bad things could happen when this program is running.

To handle the exceptions that this program could produce, you can start with the most likely problem that would occur. Trusting that a user is able to provide the two numbers that are requested, it is possible that she would type an invalid operator. For example, for this program we will perform only the addition (+), the subtraction(-), the multiplication(*), and the division(/). Therefore, we will first validate the operator. This can be done as follows:

```

#include <iostream.h>
#include <string>
using namespace std;
int main()
{
    double Operand1, Operand2, Result;
    char Operator;

    cout << "This program allows you to perform an operation on two
        numbers\n";

    try {
        cout << "To proceed, enter a number, an operator, and a number:\n";
        cin >> Operand1 >> Operator >> Operand2;
        if(Operator != '+' && Operator != '-' &&

```

```

        Operator != '*' && Operator != '/')
            throw Operator;
    switch(Operator)
    {
    case '+':
        Result = Operand1 + Operand2;
        break;
    case '-':
        Result = Operand1 - Operand2;
        break;
    case '*':
        Result = Operand1 * Operand2;
        break;
    case '/':
        Result = Operand1 / Operand2;
        break;
    }
    cout << "\n" << Operand1 << " " << Operator << " "
        << Operand2 << " = " << Result;
}
catch(const char n)
{
    cout << "\nOperation Error: " << n << " is not a valid operator";
}

cout << "\n\n";
return 0;
}

```

When this program runs, if the user provides two valid numbers but a wrong operator, the program asks a throw to send a character (in fact the character that was typed as the operator) that represents the error. Then, when the compiler gets out of the try block, it looks for and finds a catch clause that receives a character value. Therefore, this catch is executed.

Imagine that the user wants to perform a division. You need to tell the compiler what to do if the user enters the denominator as 0 (or 0.00). If this happens, the best option, and probably the only one you should consider, is to display a message and get out. To implement this behavior, we will add another catch block that displays a message:

```
#include <iostream.h>

int main()
{
    double Operand1, Operand2, Result;
    char Operator;

    // Request two numbers from the user
    cout << "This program allows you to perform a division of two numbers\n";
    cout << "To proceed, enter two numbers\n";

    try {
        cout << "First Number: ";
        cin >> Operand1;
        cout << "Operator:   ";
        cin >> Operator;
        cout << "Second Number: ";
        cin >> Operand2;

        // Make sure the user typed a valid operator
        if(Operator != '+' && Operator != '-' &&
           Operator != '*' && Operator != '/')
            throw Operator;

        // Find out if the denominator is 0
```

```

        if(Operator == '/')
            if(Operand2 == 0)
                throw 0;

        // Perform an operation based on the user's choice
        switch(Operator)
        {
        case '+':
            Result = Operand1 + Operand2;
            break;
        case '-':
            Result = Operand1 - Operand2;
            break;
        case '*':
            Result = Operand1 * Operand2;
            break;
        case '/':
            Result = Operand1 / Operand2;
            break;
        }

        // Display the result of the operation
        cout << "\n" << Operand1 << " " << Operator << " "
            << Operand2 << " = " << Result << "\n\n";
    }

    catch(const char n)
    {
        cout << "\nOperation Error: " << n << " is not a valid operator\n\n";
    }

    catch(const int p)

```

```

    {
        cout << "\nBad Operation: Division by " << p << " not
allowed\n\n";
    }
    return 0;
}

```

When running this program, if the user types a wrong operator, the compiler considers the integer error, gets out of the try block, and looks for a catch that can use a character. The first catch can validate it and gets executed. If the user enters the right values (Number Operator Number), then the compiler finds out if the operator entered was a forward slash "/" used to perform a division. If the user wants to perform a division, the compiler finds out if the second operand, the denominator, is 0. If it is, the program presents a throw that sends an integer. Based on this exception, the compiler gets out of the try block and starts looking for a catch block that can use an integer. The first catch can't, it uses a character. Therefore, the compiler looks at the next catch, if any. Our program provides a second catch that takes an integer as an argument. Therefore, this catch gets executed. Not all our problems are solved. Imagine the user types an invalid number. The first characteristic of an invalid number is one that contains anything else than a digit (a character between 0 and 9). Since our program performs its operations on decimal numbers, we need to allow the number to have a decimal portion. Instead of expecting numeric values from the user, we will request arrays of characters. We will use the `isdigit()` function to examine each character entered in order to find out whether any one of them is not a digit. Also, we will allow the user to type a period that separates the decimal part of a number. If any of the characters that the user entered is not a digit, we will throw a string error so a catch can deal with it. This means that we will add a catch that is different from the other already existing ones. So far, we were requesting two double-precision numbers from the user. In order to check each number and validate it, instead of decimals, we will ask the user to type two strings (arrays of characters):

```

#include <iostream.h.h>
#include <string.h>
using namespace std;
int main()

```

```

{
    char Number1[40], Number2[40];
    double Operand1, Operand2, Result;
    char Operator;

    // Request two numbers from the user
    cout << "This program allows you to perform a division of two numbers\n";
    cout << "To proceed, enter two numbers\n";
    try {
        cout << "First Number: ";
        cin >> Number1;
        cout << "Operator: ";
        cin >> Operator;
        cout << "Second Number: ";
        cin >> Number2;

        // Examine each character of the first operand
        // to find out if the user included a non-digit in the number
        for(int i = 0; i < strlen(Number1); i++)
            if( (!isdigit(Number1[i])) && (Number1[i] != '.') ) // Allow the
                period
                throw Number1; // Send the error as a string
        Operand1 = atof(Number1);

        // Do the same for the second number entered
        for(int j = 0; j < strlen(Number2); j++)
            if( (!isdigit(Number2[j])) && (Number2[j] != '.') ) // Allow the
                period
                throw Number2; // Send the error as a string
        Operand2 = atof(Number2);

        // Make sure the user typed a valid operator
    }
}

```

```

if(Operator != '+' && Operator != '-' &&
   Operator != '*' && Operator != '/')
    throw Operator;

// Find out if the denominator is 0
if(Operator == '/')
    if(Operand2 == 0)
        throw 0;

// Perform an operation based on the user's choice
switch(Operator)
{
case '+':
    Result = Operand1 + Operand2;
    break;
case '-':
    Result = Operand1 - Operand2;
    break;
case '*':
    Result = Operand1 * Operand2;
    break;
case '/':
    Result = Operand1 / Operand2;
    break;
}

// Display the result of the operation
cout << "\n" << Operand1 << " " << Operator << " "
     << Operand2 << " = " << Result << "\n\n";
}
catch(const int n)

```



```

{
    cout << "\nBad Operation: Division by " << n << " not
allowed\n\n";
}
catch(const char n)
{
    cout << "\nOperation Error: " << n << " is not a valid operator\n\n";
}
catch(const char *BadOperand)
{
    cout << "\nError: " << BadOperand << " is not a valid number\n\n";
}
return 0;

```

9.4 Understanding terminate() and unexpected()

As mentioned earlier, terminate() and unexpected() are called when something goes wrong during the exception handling process. These functions are supplied by the Standard C++ library. Their prototypes are shown here:

```
void terminate( );
```

```
void unexpected( );
```

These functions require the header <exception>.

The terminate () function is called whenever the exception handling subsystem fails to find a matching catch statement for an exception. It is also called if your program attempts to rethrow an exception when no exception was originally thrown. The terminate () function is also called under various other, more obscure circumstances. For example, such a circumstance could occur when, in the process of unwinding the stack because of an exception, a destructor for an object being destroyed throws an exception. In general, terminate () is the handler of last resort when no other handlers for an exception are available. By default, terminate () calls abort ().

The `unexpected ()` function is called when a function attempts to throw an exception that is not allowed by its throw list. By default, `unexpected ()` calls `terminate ()`.

9.4.1 Setting the Terminate and Unexpected Handlers

The `terminate ()` and `unexpected ()` functions simply call other functions to actually handle an error. As just explained, by default `terminate ()` calls `abort ()`, and `unexpected()` calls `terminate()`. Thus, by default, both functions halt program execution when an exception handling error occurs. However, you can change the functions that are called by `terminate ()` and `unexpected ()`. Doing so allows your program to take full control of the exception handling subsystem.

To change the terminate handler, use `set_terminate()`, shown here:

```
terminate_handler set_terminate(terminate_handler newhandler) throw( );
```

Here, `newhandler` is a pointer to the new terminate handler. The function returns a pointer to the old terminate handler. The new terminate handler must be of type `terminate_handler`, which is defined like this:

```
typedef void (*terminate_handler) ( );
```

The only thing that your terminate handler must do is stop program execution. It must not return to the program or resume it in any way. To change the unexpected handler, use `set_unexpected()`, shown here:

```
unexpected_handler set_unexpected(unexpected_handler newhandler) throw( );
```

Here, `newhandler` is a pointer to the new unexpected handler. The function returns a pointer to the old unexpected handler. The new unexpected handler must be of type `unexpected_handler`, which is defined like this:

```
typedef void (*unexpected_handler) ( );
```

This handler may itself throw an exception, stop the program, or call `terminate ()`. However, it must not return to the program. Both `set_terminate()` and `set_unexpected()` require the header `<exception>`. Here is an example that defines its own `terminate()` handler.

```
// Set a new terminate handler.
```

```
#include <iostream.h>
```

```

#include <cstdlib>
#include <exception>
using namespace std;
void my_Thandler() {
    cout << "Inside new terminate handler\n";
    abort();
}
int main()
{
    // set a new terminate handler
    set_terminate(my_Thandler);
    try {
        cout << "Inside try block\n";
        throw 100; // throw an error
    }
    catch (double i) { // won't catch an int exception
        // ...
    }
    return 0;
}

```

The output from this program is shown here.

Inside try block

Inside new terminate handler

abnormal program termination

9.5 The `uncaught_exception()` Function

The C++ exception handling subsystem supplies one other function that you may find useful: `uncaught_exception()`. Its prototype is shown here:

```
bool uncaught_exception( );
```

This function returns true if an exception has been thrown but not yet caught. Once caught, the function returns false.

9.5.1 The `exception` and `bad_exception` Classes

When a function supplied by the C++ standard library throws an exception, it will be an object derived from the base class `exception`. An object of the class `bad_exception` can be thrown by the unexpected handler. These classes require the header `<exception>`.

9.6 Applying Exception Handling

Exception handling is designed to provide a structured means by which your program can handle abnormal events. This implies that the error handler must do something rational when an error occurs. For example, consider the following simple program. It inputs two numbers and divides the first by the second. It uses exception handling to manage a divide-by-zero error.

```
#include <iostream.h>

using namespace std;

void divide(double a, double b);

int main()
{
    double i, j;
    do {
        cout << "Enter numerator (0 to stop): ";
        cin >> i;
        cout << "Enter denominator: ";
        cin >> j;
        divide(i, j);
    } while(i != 0);
    return 0;
}

void divide(double a, double b)
```

```

{
try {
    if(!b) throw b; // check for divide-by-zero
    cout << "Result: " << a/b << endl;
}
catch (double b) {
    cout << "Can't divide by zero.\n";
}
}

```

While the preceding program is a very simple example, it does illustrate the essential nature of exception handling. Since division by zero is illegal, the program cannot continue if a zero is entered for the second number. In this case, the exception is handled by not performing the division (which would have caused abnormal program termination) and notifying the user of the error. The program then reprompts the user for two more numbers. Thus, the error has been handled in an orderly fashion and the user may continue on with the program. The same basic concepts will apply to more complex applications of exception handling.

Exception handling is especially useful for exiting from a deeply nested set of routines when a catastrophic error occurs. In this regard, C++'s exception handling is designed to replace the rather clumsy C-based `setjmp()` and `longjmp()` functions. Remember, the key point about using exception handling is to provide an orderly way of handling errors. This means rectifying the situation, if possible.

9.7 Summary

Exception handling is the process of responding to the occurrence, during computation, of exceptions – anomalous or exceptional events requiring special processing – often changing the normal flow of program execution. It is provided by specialized programming language constructs or computer hardware mechanisms.

In general, an exception is handled (resolved) by saving the current state of execution in a predefined place and switching the execution to a specific subroutine known as an exception handler. If exceptions are continuable, the handler may later resume the execution at the original

location using the saved information. For example, a floating point divide by zero exception will typically, by default, allow the program to be resumed, while an out of memory condition might not be resolvable transparently.

9.8 Review Questions

1. What is exception handling?
2. Explain the fundamentals of exception handling.
3. Explain about `terminate()` and `unexpected` with example.
4. Explain about `uncaught_exception()` function.
5. How apply exception handling? Explain.

LESSON 10

STREAMS AND FILES

Structure

- 10.1 Introduction**
- 10.2 Learning Objectives**
- 10.3 Input Output with Files**
- 10.4 Methods of Input and Output Classes**
- 10.5 Text mode files**
- 10.6 State flags**
- 10.7 get and put stream pointers**
- 10.8 Binary files**
- 10.9 I/O Manipulators**
- 10.10 Overloading << and >>**
- 10.11 Summary**
- 10.12 Review Questions**

10.1 Introduction

A stream is an important concept in modern programming. Earlier programming languages such as Fortran, Cobol and Basic had input and output built in to those languages. Modern languages like C, C++ and C# tend to avoid having this and instead implement input and output via library functions. Output involves sending a stream of byte to a device and whether that device is the screen, an object in memory, a file on disk, another computer on a network or a printer is unimportant. The same routines can be used with different destinations. In earlier languages you needed separate routines to handle each device. Streams make this flexibility possible.

10.2 Learning Objectives

After learning this lesson you should be able to know

- Input Output with Files
- Methods of Input and Output Classes
- Text mode files
- State flags
- get and put stream pointers
- Binary files
- I/O Manipulators

10.3 INPUT OUTPUT WITH FILES

The techniques for file input and output, I/O, in C++ are virtually identical to those introduced in earlier lessons for writing and reading to the standard output devices, the screen and keyboard. To perform file input and output the include file `fstream` must be used.

```
#include <fstream>
```

`fstream` contains class definitions for classes used in file I/O. Within a program needing file i/o, for each output file required, an object of class `ofstream` is instantiated. For each input file required, an object of class `ifstream` is instantiated. The `ofstream` object is used exactly as the `cout` object for standard output is used. The `ifstream` object is used exactly as the `cin` object for standard input is used. This is best understood by studying an example.

C++ has support both for input and output with files through the following classes:

ofstream: File class for writing operations (derived from `ostream`)

ifstream : File class for reading operations (derived from `istream`)

fstream : File class for both reading and writing operations (derived from `iostream`)

10.3.1 Open a file

The first operation generally done on an object of one of these classes is to associate it to a real file, that is to say, to open a file. The open file is represented within the program by a

stream object (an instantiation of one of these classes) and any input or output performed on this stream object will be applied to the physical file.

In order to open a file with a stream object we use its member function `open()`:

```
void open (const char * filename, openmode mode);
```

where `filename` is a string of characters representing the name of the file to be opened and `mode` is a combination of the following flags:

`ios::in` Open file for reading

`ios::out` Open file for writing

`ios::ate` Initial position: end of file

`ios::app` Every output is appended at the end of file

`ios::trunc` If the file already existed it is erased

`ios::binary` Binary mode

These flags can be combined using bitwise operator OR: `||` For example, if we want to open the file “example.bin” in binary mode to add data we could do it by the following call to function member `open`:

```
ofstream file;
```

```
file.open (“example.bin”, ios::out | ios::app | ios::binary);
```

All of the member functions `open` of classes `ofstream`, `ifstream` and `fstream` include a default mode when opening files that varies from one to the other:

<u>Class</u>	<u>default mode to parameter</u>
<code>ofstream</code>	<code>ios::out ios::trunc</code>
<code>ifstream</code>	<code>ios::in</code>
<code>fstream</code>	<code>ios::in ios::out</code>

The default value is only applied if the function is called without specifying a mode parameter. If the function is called with any value in that parameter the default mode is stepped on, not combined.

Since the first task that is performed on an object of classes `ofstream`, `ifstream` and `fstream` is frequently to open a file, these three classes include a constructor that directly calls the `open` member function and has the same parameters as this. This way, we could also have declared the previous object and conducted the same opening operation just by writing:

```
ofstream file ("example.bin", ios::out | ios::app | ios::binary);
```

Both forms to open a file are valid. You can check if a file has been correctly opened by calling the member function `is_open()`:

```
bool is_open();
```

that returns a `bool` type value indicating true in case that indeed the object has been correctly associated with an open file or false otherwise.

10.3.2 Closing a file

When reading, writing or consulting operations on a file are complete we must close it so that it becomes available again. In order to do that we shall call the member function `close ()`, that is in charge of flushing the buffers and closing the file. Its form is quite simple:

```
void close ();
```

Once this member function is called, the stream object can be used to open another file, and the file is available again to be opened by other processes. In case that an object is destructed while still associated with an open file, the destructor automatically calls the member function `close`.

10.4 METHODS OF INPUT AND OUTPUT CLASSES

The `ifstream` class has several useful methods for input. These methods are also in the class `cin`, which is used to read from standard input. These methods are used to read from any input stream. An input stream is a source of input such as the keyboard, a file or a buffer.

10.4.1 Extraction Operator >>

This overloaded operator handles all built in C++ data types. By default, any intervening white space is disregarded. That is, blanks, tabs, new lines, form feeds and carriage returns are skipped over.

10.4.2 get()

This form of `get` extracts a single character from the input stream, that is, from the standard input, a file or a buffer. It does not skip white space. It returns type `int`.

10.4.3 get(char &ch)

This form of `get` also extracts a single character from the input stream, but it stores the value in the character variable passed in as an argument.

10.4.4 get(char *buff, int buffsize, char delimiter='\n')

This form of `get` reads characters into the C-style buffer passed as an argument `buffsize` characters are read, the delimiter is encountered or an end of file is encountered. The `'\n'` is the new line character. The delimiter is not read into the buffer but is instead left in the input stream. It must be removed separately but using either another `get` or an `ignore`. Because of this added step, this form of `get` is a frequent source of errors and should be avoided. Fortunately, another method shown below, `getline`, reads in the delimiter as well and should be used in place of this form of `get`.

10.4.5 Getline

There are several useful forms of `getline`.

`ignore(int count=1, int delim=traits_type::eof)`

This method reads and discards “count” characters from the input stream.

10.4.6 peek()

This method returns the next character from the input stream, but does not remove it. It is useful to look ahead at what the next character read will be.

10.4.7 putback(char &ch)

This method puts `ch` onto the input stream. The character in `ch` will then be the next character read from the input stream.

10.4.8 unget()

This method puts the last read character back into the input stream.

10.4.9 read(char *buff, int count)

This method is used to perform an unformatted read of count bytes from the input stream into a character buffer. The ofstream class has several useful methods for writing to an output stream. An output stream is standard output (usually the screen), a file or a buffer. These methods are also in the object cout, which is used for standard output. The simplest way to understand how to use these methods is by looking at a few examples. Since we have seen the extraction, >>, and insertion, << in several lessons, let's look at the other methods. Getline, which is very useful to read entire lines of text into a string.

Suppose we need to read a file and determine the number of alphanumeric characters, the number of blanks and the number of sentences. To determine the number of sentences we will count the number of periods (dots). We will disregard newlines and tabs. Here is a program that solves the problem.

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int blank_count = 0;
    int char_count = 0;
    int sentence_count = 0;
    char ch;
    ifstream iFile("c:/lesson12.txt");
    if (!iFile)
    {
        cout << "Error opening input file" << endl;
        return -1;
    }
    while (iFile.get(ch))
    {
```

```

switch (ch)
{
case ' ':
blank_count++;
break;
case '\n':
case '\t':
break;
case '.':
sentence_count++;
break;
default:
char_count++;
break;
}
}

cout << "There are " << blank_count << " blanks" <<
endl;
cout << "There are " << char_count << " characters"
<< endl;
cout << "There are " << sentence_count << "
sentences" << endl;
return 0;
}

```

As a second example, let's implement a program that will copy the contents of one file to another. The program will prompt the user for the input and output file names, and then copy.

```

#include <iostream>

#include <fstream>

```

```

#include <string>
using namespace std;
int main()
{
    char ch;
    string iFileName;
    string oFileName;
    cout << "Enter the source file name: ";
    cin >> iFileName;
    cout << "Enter the destination file name: ";
    cin >> oFileName;
    ofstream oFile(oFileName.c_str());
    ifstream iFile(iFileName.c_str());
    //Error checking on file opens omitted for brevity.
    while (iFile.get(ch))
    {
        oFile.put(ch);
    } return 0;
}

```

10.5 TEXT MODE FILES

Classes ofstream, ifstream and fstream are derived from ostream, istream and iostream respectively. That's why fstream objects can use the members of these parent classes to access data. Generally, when using text files we shall use the same members of these classes that we used in communication with the console (cin and cout).

As in the following example, where we use the overloaded insertion operator <<:

```

// writing on a text file
#include <fstream.h>
int main ()

```

```

{
ofstream examplefile ("example.txt");
if (examplefile.is_open())
{
examplefile << "This is a line.\n";
examplefile << "This is another line.\n";
examplefile.close();
}
return 0;
}

```

The output is: This is a line.

This is another line.

Data input from file can also be performed in the same way that we did with cin:

```

// reading a text file
#include <iostream.h>
#include <fstream.h>
#include <stdlib.h>
int main ()
{
char buffer[256];
ifstream examplefile ("example.txt");
if (! examplefile.is_open())
{ cout << "Error opening file"; exit (1); }
while (! examplefile.eof() )
{
examplefile.getline (buffer,100);
cout << buffer << endl;
} return 0;
}

```

```
}
```

The output is : This is a line.

This is another line.

This last example reads a text file and prints out its content on the screen. Notice how we have used a new member function, called `eof` that `ifstream` inherits from class `ios` and that returns true in case that the end of the file has been reached.

10.6 STATE FLAGS

In addition to `eof()`, other member functions exist to verify the state of the stream (all of them return a bool value):

10.6.1 `bad()`

Returns true if a failure occurs in a reading or writing operation. For example, in case we try to write to a file that is not open for writing or if the device where we try to write has no space left.

10.6.2 `fail()`

Returns true in the same cases as `bad()` plus in case that a format error happens, as trying to read an integer number and an alphabetical character is received.

10.6.3 `eof()`

Returns true if a file opened for reading has reached the end.

10.6.4 `good()`

It is the most generic: returns false in the same cases in which calling any of the previous functions would return true. In order to reset the state flags checked by the previous member functions you can use member function `clear()`, with no parameters.

10.7 GET AND PUT STREAM POINTERS

All I/O streams objects have, at least, one stream pointer:

- 1) **`ifstream`**, like **`istream`**, has a pointer known as get pointer that points to the next element to be read.

- 2) **ofstream**, like **ostream**, has a pointer put pointer that points to the location where the next element has to be written.
- 3) **fstream**, like **iostream**, inherits both: get and put These stream pointers that point to the reading or writing locations within a stream can be read and/or manipulated using the following member functions:

tellg() and tellp()

These two member functions admit no parameters and return a value of type `pos_type` (according ANSI-C++ standard) that is an integer data type representing the current position of get stream pointer (in case of `tellg`) or put stream pointer (in case of `tellp`).

seekg() and seekp()

This pair of functions serves respectively to change the position of stream pointers get and put. Both functions are overloaded with two different prototypes:

seekg (pos_type position);

seekp (pos_type position);

Using this prototype the stream pointer is changed to an absolute position from the beginning of the file. The type required is the same as that returned by functions `tellg` and `tellp`.

seekg (off_type offset, seekdir direction);

seekp (off_type offset, seekdir direction);

Using this prototype, an offset from a concrete point determined by parameter direction can be specified. It can be:

`ios::beg` Offset specified from the beginning of the stream

`ios::cur` Offset specified from the current position of the stream pointer

`ios::end` Offset specified from the end of the stream

The values of both stream pointers get and put are counted in different ways for text files than for binary files, since in text mode files some modifications to the appearance of some special characters can occur. For that reason it is advisable to use only the first prototype of `seekg` and `seekp` with files opened in text mode and always use non modified values returned by `tellg` or `tellp`. With binary files, you can freely use all the implementations for these functions.

They should not have any unexpected behavior. The following example uses the member functions just seen to obtain the size of a binary file:

```
// obtaining file size

#include <iostream.h>
#include <fstream.h>

const char * filename = "example.txt";

int main ()
{
    long l,m;
    ifstream file (filename, ios::in|ios::binary);
    l = file.tellg();
    file.seekg (0, ios::end);
    m = file.tellg();
    file.close();
    cout << "size of " << filename;
    cout << " is " << (m-l) << " bytes.\n";
    return 0;
}
```

The output is : size of example.txt is 40 bytes.

10.7 BINARY FILES

In binary files inputting and outputting data with operators like << and >> and functions like getline, does not make too much sense, although they are perfectly valid. File streams include two member functions specially designed for input and output of data sequentially: write and read. The first one (write) is a member function of ostream, also inherited by ofstream. And read is member function of istream and it is inherited by ifstream. Objects of class fstream have both. Their prototypes are:

```
write ( char * buffer, streamsize size );
read ( char * buffer, streamsize size );
```

Where buffer is the address of a memory block where the read data are stored or from where the data to be written are taken. The size parameter is an integer value that specifies the number of characters to be read/written from/to the buffer.

```
// reading binary file

#include <iostream.h>
#include <fstream.h>

const char * filename = "example.txt";

int main ()
{
    char * buffer;
    long size;
    ifstream file (filename,
        ios::in|ios::binary|ios::ate);
    size = file.tellg();
    file.seekg (0, ios::beg);
    buffer = new char [size];
    file.read (buffer, size);
    file.close();
    cout << "the complete file is in a buffer";
    delete[] buffer;
    return 0;
}
```

The output is: the complete file is in a buffer

10.8 BUFFERS AND SYNCHRONIZATION

When we operate with file streams, these are associated to a buffer of type streambuf. This buffer is a memory block that acts as an intermediary between the stream and the physical file. For example, with an out stream, each time the member function put (write a single character)

is called, the character is not written directly to the physical file with which the stream is associated. Instead of that, the character is inserted in the buffer for that stream.

When the buffer is flushed, all data that it contains is written to the physical media (if it is an out stream) or simply erased (if it is an in stream). This process is called synchronization and it takes place under any of the following circumstances:

When the file is closed: before closing a file all buffers that have not yet been completely written or read are synchronized.

When the buffer is full: Buffers have a certain size. When the buffer is full it is automatically synchronized.

Explicitly with manipulators: When certain manipulators are used on streams synchronization takes place. These manipulators are:

flush and endl.

Explicitly with function sync (): Calling member function sync () (no parameters) causes an immediate synchronization. This function returns an int value equal to -1 if the stream has no associated buffer or in case of failure.

10.9 I/O MANIPULATORS

Up till now, we have accepted the default output formatting. C++ defines a set of manipulators which are used to modify the state of iostream objects. These control how data is formatted. They are defined in the include file, <ios>. It is not usually necessary to explicitly include this file because it is included indirectly via the use of other includes such as <iostream> or <fstream>. Let's see how some of these manipulators work in a simple program.

<u>Manipulator</u>	<u>Use</u>
Boolalpha	Causes bool variables to be output as true or false.
Noboolalpha(default)	Causes bool variables to be displayed as 0 or 1
Dec (default)	Specifies that integers are displayed in base 10
Hex	Specifies that integers are displayed in hexadecimal.

Oct	Specified that integers are displayed in octal
Left	Causes text to be left justified in the output field
Right	Causes text to be right justified in the output field.
Internal	Causes the sign of a number to be left justified and the value to be right justified.
Noshowbase (default)	Turns off displaying a prefix indicating the base of a number.
Showbase	Turns on displaying a prefix indicating the base of a number
Noshowpoint (default)	Displays decimal point only if a fractional part exists
Showpoint	Displays decimal point always
Noshowpos (default)	No “+” prefixing a positive number
Showpos	Displays a “+” prefixing a positive number
Skipws(default)	Causes white space (blanks, tabs, newlines) to be skipped by the input operator, >>.
Noskipws	White space not skipped by the extraction operator, >>.
Fixed (default)	Causes floating point numbers to be displayed in fixed notation.
Scientific	Causes floating point numbers to be displayed in fixed notation.
Nouppercase (default)	0x displayed for hexadecimal numbers, e for scientific notation
Uppercase	0X displayed for hexadecimal numbers, E for scientific notation

Table 10.1. manipulator and its use

The manipulators in the above table modify the state of the iostream object. This means that once used on an iostream object they will affect all subsequent input or output done with the object. There are several other manipulators that are used to format a particular output but do not modify the state of the object.

Setting Output Width

setw(w) - sets output or input width to w; requires <iomanip> to be included.

width(w) - a member function of the istream classes.

Filling White Space

setfill(ch) - fills white space in output fields with ch; requires <iomanip> to be included.

fill(ch) = a member function of the istream classes.

Setting Precision

setprecision (n) - sets the display of floating point numbers at precision n. This does not affect the way floating point numbers are handled during calculations in your program.

Here is a simple program illustrating the use of the I/O manipulators.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;
int main()
{
    int intValue = 15;
    cout << "Integer Number" << endl;
    cout << "Default: " << intValue << endl;
    cout << "Octal: " << oct << intValue << endl;
    cout << "Hex: " << hex << intValue << endl;
    cout << "Turning showbase on" << showbase <<
    endl;
    cout << "Dec: " << dec << intValue << endl;
    cout << "Octal: " << oct << intValue << endl;
    cout << "Hex: " << hex << intValue << endl;
    cout << "Turning showbase off" << noshowbase <<
```

```
endl;
cout << endl;
double doubleVal = 12.345678;
cout << "Floating Point Number" << endl;
cout << "Default: " << doubleVal << endl;
cout << setprecision(10);
cout << "Precision of 10: " << doubleVal << endl;
cout << scientific << "Scientific Notation: " <<
doubleVal << endl;
cout << uppercase;
cout << "Uppercase: " << doubleVal << endl;
cout << endl;
bool theBool = true;
cout << "Boolean" << endl;
cout << "Default: " << theBool << endl;
cout << boolalpha << "BoolAlpha set: " << theBool <<
endl;
cout << endl;
string myName = "John";
cout << "Strings" << endl;
cout << "Default: " << myName << endl;
cout << setw(35) << right << "With setw(35) and right:
"<< myName << endl;
cout.width(20);
cout << "With width(20): " << myName << endl;
cout << endl;
return 0;
}
```

10.10 Overloading << and >>

As you know, the << and the >> operators are overloaded in C++ to perform I/O operations on C++'s built-in types. You can also overload these operators so that they perform I/O operations on types that you create.

In the language of C++, the << output operator is referred to as the insertion operator because it inserts characters into a stream. Likewise, the >> input operator is called the extraction operator because it extracts characters from a stream. The functions that overload the insertion and extraction operators are generally called inserters and extractors, respectively.

10.10.1 Creating Your Own Inserters

It is quite simple to create an inserter for a class that you create. All inserter functions have this general form:

```
ostream &operator<<(ostream &stream, class_type obj)
{
    // body of inserter
    return stream;
}
```

Notice that the function returns a reference to a stream of type ostream. (Remember, ostream is a class derived from ios that supports output.) Further, the first parameter to the function is a reference to the output stream. The second parameter is the object being inserted. (The second parameter may also be a reference to the object being inserted.) The last thing the inserter must do before exiting is return stream. This allows the inserter to be used in a larger I/O expression.

Within an inserter function, you may put any type of procedures or operations that you want. That is, precisely what an inserter does is completely up to you. However, for the inserter to be in keeping with good programming practices, you should limit its operations to outputting information to a stream. For example, having an inserter compute pi to 30 decimal places as a side effect to an insertion operation is probably not a very good idea! To demonstrate a custom inserter, one will be created for objects of type phonebook, shown here.


```

class phonebook {
public:
    char name[80];
    int areacode;
    int prefix;
    int num;
    phonebook(char *n, int a, int p, int nm)
    {
        strcpy(name, n);
        areacode = a;
        prefix = p;
        num = nm;
    }
};

```

This class holds a person's name and telephone number. Here is one way to create an inserter function for objects of type phonebook.

```

// Display name and phone number
ostream &operator<<(ostream &stream, phonebook o)
{
    stream << o.name << " ";
    stream << "(" << o.areacode << ") ";
    stream << o.prefix << "-" << o.num << "\n";
    return stream; // must return stream
}

```

Here is a short program that illustrates the phonebook inserter function:

```

#include <iostream>
#include <cstring>

```

```

using namespace std;

class phonebook {
public:
    char name[80];
    int areacode;
    int prefix;
    int num;

    phonebook(char *n, int a, int p, int nm)
    {
        strcpy(name, n);
        areacode = a;
        prefix = p;
        num = nm;
    }
};

// Display name and phone number.
ostream &operator<<(ostream &stream, phonebook o)
{
    stream << o.name << " ";
    stream << "(" << o.areacode << ") ";
    stream << o.prefix << "-" << o.num << "\n";
    return stream; // must return stream
}

int main()
{
    phonebook a("Ted", 111, 555, 1234);
    phonebook b("Alice", 312, 555, 5768);

```

```

phonebook c("Tom", 212, 555, 9991);
cout << a << b << c;
return 0;
}

```

The program produces this output:

```

Ted (111) 555-1234
Alice (312) 555-5768
Tom (212) 555-9991

```

In the preceding program, notice that the phonebook inserter is not a member of phonebook. Although this may seem weird at first, the reason is easy to understand. When an operator function of any type is a member of a class, the left operand (passed implicitly through this) is the object that generates the call to the operator function.

Further, this object is an object of the class for which the operator function is a member.

There is no way to change this. If an overloaded operator function is a member of a class, the left operand must be an object of that class. However, when you overload inserters, the left operand is a stream and the right operand is an object of the class. Therefore, overloaded inserters cannot be members of the class for which they are overloaded. The variables name, areacode, prefix, and num are public in the preceding program so that they can be accessed by the inserter.

10.10.2 Creating Your Own Extractors

Extractors are the complement of inserters. The general form of an extractor function is

```

istream &operator>>(istream &stream, class_type &obj)
{
    // body of extractor
    return stream;
}

```

Extractors return a reference to a stream of type istream, which is an input stream. The first parameter must also be a reference to a stream of type istream. Notice that the second

parameter must be a reference to an object of the class for which the extractor is overloaded. This is so the object can be modified by the input (extraction) operation. Continuing with the phonebook class, here is one way to write an extraction function:

```
istream &operator>>(istream &stream, phonebook &o)
{
    cout << "Enter name: ";
    stream >> o.name;
    cout << "Enter area code: ";
    stream >> o.areacode;
    cout << "Enter prefix: ";
    stream >> o.prefix;
    cout << "Enter number: ";
    stream >> o.num;
    cout << "\n";
    return stream;
}
```

Notice that although this is an input function, it performs output by prompting the user. The point is that although the main purpose of an extractor is input, it can perform any operations necessary to achieve that end. However, as with inserters, it is best to keep the actions performed by an extractor directly related to input. If you don't, you run the risk of losing much in terms of structure and clarity. Here is a program that illustrates the phonebook extractor:

```
#include <iostream>
#include <cstring>
using namespace std;
class phonebook {
    char name[80];
    int areacode;
    int prefix;
```

```

int num;

public:
phonebook() { };
phonebook(char *n, int a, int p, int nm)
{
strcpy(name, n);
areacode = a;
prefix = p;
num = nm;
}

friend ostream &operator<<(ostream &stream, phonebook o);
friend istream &operator>>(istream &stream, phonebook &o);
};

// Display name and phone number.
ostream &operator<<(ostream &stream, phonebook o)
{
stream << o.name << " ";
stream << "(" << o.areacode << ") ";
stream << o.prefix << "-" << o.num << "\n";
return stream; // must return stream
}

// Input name and telephone number.
istream &operator>>(istream &stream, phonebook &o)
{
cout << "Enter name: ";
stream >> o.name;
cout << "Enter area code: ";
stream >> o.areacode;

```

```

    cout << "Enter prefix: ";
    stream >> o.prefix;
    cout << "Enter number: ";
    stream >> o.num;
    cout << "\n";
    return stream;
}

int main()
{
    phonebook a;
    cin >> a;
    cout << a;
    return 0;
}

```

Actually, the extractor for phonebook is less than perfect because the cout statements are needed only if the input stream is connected to an interactive device such as the console (that is, when the input stream is cin). If the extractor is used on a stream connected to a disk file, for example, then the cout statements would not be applicable. For fun, you might want to try suppressing the cout statements except when the input stream refers to cin. For example, you might use if statements such as the one shown here.

```
if(stream == cin) cout << "Enter name: ";
```

10.11 Summary

The techniques for file input and output, I/O, in C++ are virtually identical to those introduced in earlier lessons for writing and reading to the standard output devices, the screen and keyboard. When we operate with file streams, these are associated to a buffer of type streambuf. This buffer is a memory block that acts as an intermediary between the stream and the physical file. For example, with an out stream, each time the member function put (write a single character) is called, the character is not written directly to the physical file with which the stream is associated. Instead of that, the character is inserted in the buffer for that stream.

10.12 Review Questions

1. Define files.
2. Explain I/O classes in detail.
3. Explain text mode files.
4. Discuss about state files.
5. Explain binary files in detail.
6. Explain I/O manipulator in detail.

Lesson 11

NAMESPACES AND CONVERSION FUNCTIONS

Structure

- 11.1 Introduction
- 11.2 Learning Objectives
- 11.3 Namespaces
- 11.4 using
- 11.5 unnamed namespace
- 11.6 The std namespace
- 11.7 creating connection function
- 11.8 Summary
- 11.9 Review Questions

11.1 Introduction

They are a relatively recent addition to C++. Their purpose is to localize the names of identifiers to avoid name collisions. The C++ programming environment has seen an explosion of variable, function, and class names. Prior to the invention of namespaces, all of these names competed for slots in the global namespace and many conflicts arose. For example, if your program defined a function called `abs()`, it could (depending upon its parameter list) override the standard library function `abs()` because both names would be stored in the global namespace. Name collisions were compounded when two or more third-party libraries were used by the same program. In this case, it was possible—even likely—that a name defined by one library would conflict with the same name defined by the other library. The situation can be particularly troublesome for class names. For example, if your program defines a class call `ThreeDCircle` and a library used by your program defines a class by the same name, a conflict will arise. The creation of the namespace keyword was a response to these problems. Because it localizes the visibility of names declared within it, a namespace allows the same name to be used in different contexts without conflicts arising. Perhaps the most noticeable beneficiary of namespace is the C++ standard library. Prior to namespace, the entire C++ library was defined within the

global namespace (which was, of course, the only namespace). Since the addition of namespace, the C++ library is now defined within its own namespace, called `std`, which reduces the chance of name collisions. You can also create your own namespaces within your program to localize the visibility of any names that you think may cause conflicts. This is especially important if you are creating class or function libraries.

11.2 Learning Objectives

After learning this lesson you should be able to know

- Namespaces
- `using`
- unnamed namespace
- The `std` namespace
- creating connection function

11.3 Namespace Fundamentals

The `namespace` keyword allows you to partition the global namespace by creating a declarative region. In essence, a namespace defines a scope. The general form of namespace is shown here:

```
namespace name {  
    // declarations  
}
```

Anything defined within a namespace statement is within the scope of that namespace. Here is an example of a namespace. It localizes the names used to implement a simple countdown counter class. In the namespace are defined the counter class, which implements the counter, and the variables `upperbound` and `lowerbound`, which contain the upper and lower bounds that apply to all counters.

```
namespace CounterNameSpace {  
    int upperbound;  
    int lowerbound;
```

```

class counter {
    int count;
public:
    counter(int n) {
        if(n <= upperbound) count = n;
        else count = upperbound;
    }
    void reset(int n) {
        if(n <= upperbound) count = n;
    }
    int run() {
        if(count > lowerbound) return count—;
        else return lowerbound;
    }
};

```

Here, upperbound, lowerbound, and the class counter are part of the scope defined by the CounterNameSpace namespace. Inside a namespace, identifiers declared within that namespace can be referred to directly, without any namespace qualification. For example, within CounterNameSpace, the run() function can refer directly to lowerbound in the statement

```
if(count > lowerbound) return count—;
```

However, since namespace defines a scope, you need to use the scope resolution operator to refer to objects declared within a namespace from outside that namespace.

For example, to assign the value 10 to upperbound from code outside CounterNameSpace, you must use this statement:

```
CounterNameSpace::upperbound = 10;
```

Or to declare an object of type counter from outside CounterNameSpace, you will use a statement like this:

```
CounterNameSpace::counter ob;
```

In general, to access a member of a namespace from outside its namespace, precede the member's name with the name of the namespace followed by the scope resolution operator. Here is a program that demonstrates the use of CounterNameSpace.

```
// Demonstrate a namespace.
#include <iostream>
using namespace std;
namespace CounterNameSpace {
    int upperbound;
    int lowerbound;
    class counter {
    int count;
    public:
    counter(int n) {
        if(n <= upperbound) count = n;
        else count = upperbound;
    }
    void reset(int n) {
        if(n <= upperbound) count = n;
    }
    int run() {
        if(count > lowerbound) return count—;
        else return lowerbound;
    }
};
}
int main()
{
```

```

CounterNameSpace::upperbound = 100;
CounterNameSpace::lowerbound = 0;
CounterNameSpace::counter ob1(10);
int i;
do {
    i = ob1.run();
    cout << i << " ";
} while(i > CounterNameSpace::lowerbound);
cout << endl;
CounterNameSpace::counter ob2(20);
do {
    i = ob2.run();
    cout << i << " ";
} while(i > CounterNameSpace::lowerbound);
cout << endl;
ob2.reset(100);
CounterNameSpace::lowerbound = 90;
do {
    i = ob2.run();
    cout << i << " ";
} while(i > CounterNameSpace::lowerbound);
return 0;
}

```

Notice that the declaration of a counter object and the references to upperbound and lowerbound are qualified by CounterNameSpace. However, once an object of type counter has been declared, it is not necessary to further qualify it or any of its members. Thus, ob1.run() can be called directly; the namespace has already been resolved.

11.4. using

As you can imagine, if your program includes frequent references to the members of a namespace, having to specify the namespace and the scope resolution operator each time you need to refer to one quickly becomes a tedious chore. The using statement was invented to alleviate this problem. The using statement has these two general forms:

```
using namespace name;
```

```
using name::member;
```

In the first form, name specifies the name of the namespace you want to access. All of the members defined within the specified namespace are brought into view (i.e., they become part of the current namespace) and may be used without qualification. In the second form, only a specific member of the namespace is made visible. For example, assuming CounterNameSpace as shown above, the following using statements and assignments are valid.

```
using CounterNameSpace::lowerbound; // only lowerbound is visible
```

```
lowerbound = 10; // OK because lowerbound is visible
```

```
using namespace CounterNameSpace; // all members are visible
```

```
upperbound = 100; // OK because all members are now visible
```

The following program illustrates using by reworking the counter example from the previous section.

```
// Demonstrate using.
```

```
#include <iostream>
```

```
using namespace std;
```

```
namespace CounterNameSpace {
```

```
int upperbound;
```

```
int lowerbound;
```

```
class counter {
```

```
int count;
```

```
public:
```

```

counter(int n) {
    if(n <= upperbound) count = n;
    else count = upperbound;
}

void reset(int n) {
    if(n <= upperbound) count = n;
}

int run() {
    if(count > lowerbound) return count—;
    else return lowerbound;
}

};

}

int main()
{
    // use only upperbound from CounterNameSpace
    using CounterNameSpace::upperbound;
    // now, no qualification needed to set upperbound
    upperbound = 100;
    // qualification still needed for lowerbound, etc.
    CounterNameSpace::lowerbound = 0;
    CounterNameSpace::counter ob1(10);
    int i;
    do {
        i = ob1.run();
        cout << i << “ “;
    } while(i > CounterNameSpace::lowerbound);
    cout << endl;
}

```

```

// now, use entire CounterNameSpace
using namespace CounterNameSpace;
counter ob2(20);
do {
    i = ob2.run();
    cout << i << " ";
} while(i > lowerbound);
cout << endl;
ob2.reset(100);
lowerbound = 90;
do {
    i = ob2.run();
    cout << i << " ";
} while(i > lowerbound);
return 0;
}

```

The program illustrates one other important point: using one namespace does not override another. When you bring a namespace into view, it simply adds its names to whatever other namespaces are currently in effect. Thus, by the end of the program, both `std` and `CounterNameSpace` have been added to the global namespace.

11.5 Unnamed Namespaces

There is a special type of namespace, called an unnamed namespace, that allows you to create identifiers that are unique within a file. Unnamed namespaces are also called anonymous namespaces. They have this general form:

```

namespace {
    // declarations
}

```

Unnamed namespaces allow you to establish unique identifiers that are known only within the scope of a single file. That is, within the file that contains the unnamed namespace, the members of that namespace may be used directly, without qualification. But outside the file, the identifiers are unknown. Unnamed namespaces eliminate the need for certain uses of the static storage class

modifier. One way to restrict the scope of a global name to the file in which it is declared is to use static. For example, consider the following two files that are part of the same program.

File One

```
static int k;  
void f1() {  
    k = 99; // OK  
}
```

File Two

```
extern int k;  
void f2() {  
    k = 10; // error  
}
```

Because `k` is defined in File One, it may be used in File One. In File Two, `k` is specified as `extern`, which means that its name and type are known but that `k` itself is not actually defined. When these two files are linked, the attempt to use `k` within File Two results in an error because there is no definition for `k`. By preceding `k` with `static` in File One, its scope is restricted to that file and it is not available to File Two. While the use of static global declarations is still allowed in C++, a better way to accomplish the same effect is to use an unnamed namespace. For example:

File One

```
namespace {  
    int k;  
}  
void f1() {
```



```
k = 99; // OK
}
```

File Two

```
extern int k;
void f2() {
    k = 10; // error
}
```

Here, `k` is also restricted to File One. The use of the unnamed namespace rather than `static` is recommended for new code.

11.5.1 Some Namespace Options

There may be more than one namespace declaration of the same name. This allows a namespace to be split over several files or even separated within the same file.

For example:

```
#include <iostream>
using namespace std;
namespace NS {
    int i;
}
// ...
namespace NS {
    int j;
}
int main()
{
    NS::i = NS::j = 10;
    // refer to NS specifically
    cout << NS::i * NS::j << "\n";
}
```

```
// use NS namespace
using namespace NS;
cout << i * j;
return 0;
}
```

This program produces the following output:

```
100
100
```

Here, NS is split into two pieces. However, the contents of each piece are still within the same namespace, that is, NS.

A namespace must be declared outside of all other scopes. This means that you cannot declare namespaces that are localized to a function, for example. There is, however, one exception: a namespace can be nested within another. Consider this program:

```
#include <iostream>
using namespace std;
namespace NS1 {
    int i;
    namespace NS2 { // a nested namespace
        int j;
    }
}
int main()
{
    NS1::i = 19;
    // NS2::j = 10; Error, NS2 is not in view
    NS1::NS2::j = 10; // this is right
    cout << NS1::i << " " << NS1::NS2::j << "\n";
    // use NS1
```

```

using namespace NS1;

/* Now that NS1 is in view, NS2 can be used to
refer to j. */
cout << i * NS2::j;
return 0;
}

```

This program produces the following output:

```

19 10
190

```

Here, the namespace NS2 is nested within NS1. Thus, when the program begins, to refer to `j`, you must qualify it with both the NS1 and NS2 namespaces. NS2 by itself is insufficient. After the statement `using namespace NS1;` executes, you can refer directly to NS2 since the `using` statement brings NS1 into view.

Typically, you will not need to create namespaces for most small to medium-sized programs. However, if you will be creating libraries of reusable code or if you want to ensure the widest portability, then consider wrapping your code within a namespace.

11.6 The `std` Namespace

Standard C++ defines its entire library in its own namespace called `std`. This is the reason that most of the programs in this book include the following statement:

```
using namespace std;
```

This causes the `std` namespace to be brought into the current namespace, which gives you direct access to the names of the functions and classes defined within the library without having to qualify each one with `std::`. Of course, you can explicitly qualify each name with `std::` if you like. For example, the following program does not bring the library into the global namespace.

```

// Use explicit std:: qualification.

#include <iostream>

int main()

```

```

{
    int val;

    std::cout << "Enter a number: ";
    std::cin >> val;

    std::cout << "This is your number: ";
    std::cout << std::hex << val;

    return 0;
}

```

Here, `cout`, `cin`, and the manipulator `hex` are explicitly qualified by their namespace. That is, to write to standard output, you must specify `std::cout`; to read from standard input, you must use `std::cin`; and the `hex` manipulator must be referred to as `std::hex`. You may not want to bring the standard C++ library into the global namespace if your program will be making only limited use of it. However, if your program contains hundreds of references to library names, then including `std` in the current namespace is far easier than qualifying each name individually.

If you are using only a few names from the standard library, it may make more sense to specify a `using` statement for each individually. The advantage to this approach is that you can still use those names without an `std::` qualification, but you will not be bringing the entire standard library into the global namespace. For example:

```

// Bring only a few names into the global namespace.
#include <iostream>

// gain access to cout, cin, and hex
using std::cout;
using std::cin;
using std::hex;

int main()
{
    int val;

    cout << "Enter a number: ";
    cin >> val;
}

```

```

cout << "This is your number: ";
cout << hex << val;
return 0;
}

```

Here, `cin`, `cout`, and `hex` may be used directly, but the rest of the `std` namespace has not been brought into view.

As explained, the original C++ library was defined in the global namespace. If you will be converting older C++ programs, then you will need to either include a `using namespace std` statement or qualify each reference to a library member with `std::`. This is especially important if you are replacing old `.H` header files with the new-style headers. Remember, the old `.H` headers put their contents into the global namespace; the new-style headers put their contents into the `std` namespace.

11.7 Creating Conversion Functions

In some situations, you will want to use an object of a class in an expression involving other types of data. Sometimes, overloaded operator functions can provide the means of doing this. However, in other cases, what you want is a simple type conversion from the class type to the target type. To handle these cases, C++ allows you to create custom conversion functions. A conversion function converts your class into a type compatible with that of the rest of the expression. The general format of a type conversion function is

```
operator type( ) { return value; }
```

Here, `type` is the target type that you are converting your class to, and `value` is the value of the class after conversion. Conversion functions return data of type `type`, and no other return type specifier is allowed. Also, no parameters may be included. A conversion function must be a member of the class for which it is defined. Conversion functions are inherited and they may be virtual.

The following illustration of a conversion function uses the `stack` class. Suppose that you want to be able to use objects of type `stack` within an integer expression. Further, suppose that the value of a `stack` object used in an integer expression is the number of values currently on the stack. (You might want to do something like this if, for example, you are using `stack` objects in a simulation and are monitoring how quickly the stacks fill up.) One way to approach this is to

convert an object of type `stack` into an integer that represents the number of items on the stack. To accomplish this, you use a conversion function that looks like this:

```
operator int() { return tos; }
```

Here is a program that illustrates how the conversion function works:

```
#include <iostream>

using namespace std;

const int SIZE=100;

// this creates the class stack

class stack {

int stck[SIZE];

int tos;

public:

stack() { tos=0; }

void push(int i);

int pop(void);

operator int() { return tos; } // conversion of stack to int

};

void stack::push(int i)

{

if(tos==SIZE) {

cout << "Stack is full.\n";

return;

}

stck[tos] = i;

tos++;

}

int stack::pop()
```

```

{
if(tos==0) {
cout << "Stack underflow.\n";
return 0;
}
tos--;
return stck[tos];
}

int main()
{
stack stck;
int i, j;
for(i=0; i<20; i++) stck.push(i);
j = stck; // convert to integer
cout << j << " items on stack.\n";
cout << SIZE - stck << " spaces open.\n";
return 0;
}

```

This program displays this output:

20 items on stack.

80 spaces open.

As the program illustrates, when a stack object is used in an integer expression, such as `j = stck`, the conversion function is applied to the object. In this specific case, the conversion function returns the value 20. Also, when `stck` is subtracted from `SIZE`, the conversion function is also called.

Here is another example of a conversion function. This program creates a class called `pwr()` that stores and computes the outcome of some number raised to some power. It stores the result as a double. By supplying a conversion function to type double and returning the result, you can use objects of type `pwr` in expressions involving other double values.

```

#include <iostream>

using namespace std;

class pwr {
double b;

int e;

double val;

public:

pwr(double base, int exp);

pwr operator+(pwr o) {
double base;
int exp;
base = b + o.b;
exp = e + o.e;
pwr temp(base, exp);
return temp;
}

operator double() { return val; } // convert to double
};

pwr::pwr(double base, int exp)
{
b = base;
e = exp;
val = 1;
if(exp==0) return;
for( ; exp>0; exp—) val = val * b;
}

int main()
{

```



```

pwr x(4.0, 2);
double a;
a = x; // convert to double
cout << x + 100.2; // convert x to double and add 100.2
cout << "\n";
pwr y(3.3, 3), z(0, 0);
z = x + y; // no conversion
a = z; // convert to double
cout << a;
return 0;
}

```

The output from the program is shown here.

```

116.2
20730.7

```

As you can see, when `x` is used in the expression `x + 100.2`, the conversion function is used to produce the double value. Notice also that in the expression `x + y`, no conversion is applied because the expression involves only objects of type `pwr`.

As you can infer from the foregoing examples, there are many situations in which it is beneficial to create a conversion function for a class. Often, conversion functions provide a more natural syntax to be used when class objects are mixed with the built-in types. Specifically, in the case of the `pwr` class, the availability of the conversion to double makes objects of that class used in “normal” mathematical expressions both easier to program and easier to understand.

You can create different conversion functions to meet different needs. You could define one that converts to double or long, for example. Each will be applied automatically as determined by the type of each expression.

const Member Functions and mutable

Class member functions may be declared as `const`, which causes this to be treated as a `const` pointer. Thus, that function cannot modify the object that invokes it. Also, a `const` object

may not invoke a non-const member function. However, a const member function can be called by either const or non-const objects. To specify a member function as const, use the form shown in the following example.

```
class X {
    int some_var;
public:
    int f1() const; // const member function
};
```

As you can see, the const follows the function's parameter declaration. The purpose of declaring a member function as const is to prevent it from modifying the object that invokes it. For example, consider the following program.

```
/*
Demonstrate const member functions.
This program won't compile.
*/
#include <iostream>
using namespace std;
class Demo {
    int i;
public:
    int geti() const {
        return i; // ok
    }
    void seti(int x) const {
        i = x; // error!
    }
};
int main()
```

```

{
    Demo ob;
    ob.seti(1900);
    cout << ob.geti();
    return 0;
}

```

This program will not compile because `seti()` is declared as `const`. This means that it is not allowed to modify the invoking object. Since it attempts to change `i`, the program is in error. In contrast, since `geti()` does not attempt to modify `i`, it is perfectly acceptable. Sometimes there will be one or more members of a class that you want a `const` function to be able to modify even though you don't want the function to be able to modify any of its other members. You can accomplish this through the use of `mutable`. It overrides `constness`. That is, a `mutable` member can be modified by a `const` member function. For example:

```

// Demonstrate mutable.
#include <iostream>
using namespace std;
class Demo {
    mutable int i;
    int j;
public:
    int geti() const {
        return i; // ok
    }
    void seti(int x) const {
        i = x; // now, OK.
    }
    /* The following function won't compile.
    void setj(int x) const {
        j = x; // Still Wrong!
    }

```

```
}  
*/  
};  
  
int main()  
{  
    Demo ob;  
    ob.seti(1900);  
    cout << ob.geti();  
    return 0;  
}
```

Here, `i` is specified as mutable, so it may be changed by the `seti()` function. However, `j` is not mutable and `setj()` is unable to modify its value.

11.8 Summary

The `namespace` keyword allows you to partition the global namespace by creating a declarative region. In essence, a namespace defines a scope. There is a special type of namespace, called an unnamed namespace, which allows you to create identifiers that are unique within a file. Unnamed namespaces are also called anonymous namespaces. In some situations, you will want to use an object of a class in an expression involving other types of data. Sometimes, overloaded operator functions can provide the means of doing this. However, in other cases, what you want is a simple type conversion from the class type to the target type. To handle these cases, C++ allows you to create custom conversion functions. A conversion function converts your class into a type compatible with that of the rest of the expression.

11.9 Review Questions

1. Define namespaces.
2. Explain about using with example.
3. Discuss about unnamed namespace.
4. Explain about The `std` namespace.
5. Explain about conversion function.

LESSON 12

INTRODUCTION TO DATA STRUCTURES

Structure

- 12.1 Introduction**
- 12.2 Learning Objectives**
- 12.3 Data Structures**
- 12.4 Abstract Data Type(ADT)**
- 12.5 Asymptotic notations**
- 12.6 Arrays**
- 12.7 Polynomial Equation**
- 12.8 Summary**
- 12.9 Review Questions**

12.1 Introduction

The field of computer science is so new that one feels obliged to furnish a definition before proceeding with this lesson. One often quoted definition views computer science as the study of algorithms. This study encompasses four distinct areas:

- (i) machines for executing algorithms—this area includes everything from the smallest pocket calculator to the largest general purpose digital computer. The goal is to study various forms of machine fabrication and organization so that algorithms can be effectively carried out.
- (ii) Languages for describing algorithms—these languages can be placed on a continuum. At one end are the languages which are closest to the physical machine and at the other end are languages designed for sophisticated problem solving. One often distinguishes between two phases of this area: language design and translation. The first calls for methods for specifying the syntax and semantics of a language. The second requires a means for translation into a more basic set of commands.

- (iii) Foundations of algorithms—here people ask and try to answer such questions as: is a particular task accomplishable by a computing device; or what is the minimum number of operations necessary for any algorithm which performs a certain function? Abstract models of computers are devised so that these properties can be studied.
- (iv) Analysis of algorithms—whenever an algorithm can be specified it makes sense to wonder about its behavior. This was realized as far back as 1830 by Charles Babbage, the father of computers. An algorithm's behavior pattern or performance profile is measured in terms of the computing time and space that are consumed while the algorithm is processing. Questions such as the worst and average time and how often they occur are typical.

We see that in this definition of computer science, “algorithm” is a fundamental notion. Thus it deserves a precise definition. The dictionary's definition “any mechanical or recursive computational procedure” is not entirely satisfying since these terms are not basic enough.

12.2 Learning Objects

At the end of this lesson you can able to

- Define data structure
- Abstract data type
- asymptotic notations
- Arrays and its types

12.3 Data Structures

An algorithm is a finite set of instructions which, if followed, accomplish a particular task. In addition every algorithm must satisfy the following criteria:

- (i) input: there are zero or more quantities which are externally supplied;
- (ii) output: at least one quantity is produced;
- (iii) definiteness: each instruction must be clear and unambiguous;
- (iv) finiteness: if we trace out the instructions of an algorithm, then for all cases the algorithm will terminate after a finite number of steps;

- (v) effectiveness: every instruction must be sufficiently basic that it can in principle be carried out by a person using only pencil and paper. It is not enough that each operation be definite as in (iii), but it must also be feasible.

In formal computer science, one distinguishes between an algorithm, and a program. A program does not necessarily satisfy condition (iv). One important example of such a program for a computer is its operating system which never terminates (except for system crashes) but continues in a wait loop until more jobs are entered. Hence, we will use these terms interchangeably.

An algorithm can be described in many ways. A natural language such as English can be used but we must be very careful that the resulting instructions are definite (condition iii). An improvement over English is to couple its use with a graphical form of notation such as flowcharts. This form places each processing step in a “box” and uses arrows to indicate the next step. Different shaped boxes stand for different kinds of operations. The point is that algorithms can be devised for many common activities.

Have you studied the flowchart? Then you probably have realized that it isn’t an algorithm at all! Which properties does it lack?

Returning to our earlier definition of computer science, we find it extremely unsatisfying as it gives us no insight as to why the computer is revolutionizing our society nor why it has made us re-examine certain basic assumptions about our own role in the universe. While this may be an unrealistic demand on a definition even from a technical point of view it is unsatisfying. The definition places great emphasis on the concept of algorithm, but never mentions the word “data”. If a computer is merely a means to an end, then the means may be an algorithm but the end is the transformation of data. That is why we often hear a computer referred to as a data processing machine. Raw data is input and algorithms are used to transform it into refined data. So, instead of saying that computer science is the study of algorithms, alternatively, we might say that computer science is the study of data:

- (i) machines that hold data;
- (ii) languages for describing data manipulation;
- (iii) foundations which describe what kinds of refined data can be produced from raw data;
- (iv) structures for representing data.

There are several terms we need to define carefully before we proceed. These include data structure, data object, data type and data representation. These four terms have no standard meaning in computer science circles, and they are often used interchangeably.

A data type is a term which refers to the kinds of data that variables may “hold” in a programming language. In FORTRAN the data types are INTEGER, REAL, LOGICAL, COMPLEX, and DOUBLE PRECISION. In PL/I there is the data type CHARACTER. The fundamental data type of SNOBOL is the character string and in LISP it is the list (or S-expression). With every programming language there is a set of built-in data types. This means that the language allows variables to name data of that type and provides a set of operations which meaningfully manipulates these variables. Some data types are easy to provide because they are already built into the computer’s machine language instruction set. Integer and real arithmetic are examples of this. Other data types require considerably more effort to implement.

In some languages, there are features which allow one to construct combinations of the built-in types. In COBOL and PL/I this feature is called a STRUCTURE while in PASCAL it is called a RECORD. However, it is not necessary to have such a mechanism. All of the data structures we will see here can be reasonably built within a conventional programming language.

Data object is a term referring to a set of elements, say D . For example the data object integers refers to $D = \{0, 1, 2, \dots\}$. The data object alphabetic character strings of length less than thirty one implies $D = \{ "", 'A', 'B', \dots, 'Z', 'AA', \dots \}$. Thus, D may be finite or infinite and if D is very large we may need to devise special ways of representing its elements in our computer. The notion of a data structure as distinguished from a data object is that we want to describe not only the set of objects, but the way they are related. Saying another way, we want to describe the set of operations which may legally be applied to elements of the data object. This implies that we must specify the set of operations and show how they work. For integers we would have the arithmetic operations $+$, $-$, $*$, $/$ and perhaps many others such as mod, ceil, floor, greater than, less than, etc. The data object integers plus a description of how $+$, $-$, $*$, $/$, etc. behave constitutes a data structure definition.

A data structure is a specialized format for organizing and storing data. General data structure types include the array, the file, the record, the table, the tree, and so on. Any data structure is designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate ways. In computer programming, a data structure may be selected or designed to store data for the purpose of working on it with various algorithms.

12.4 Abstract Data Types (ADT)

Abstract data types or ADTs are a mathematical specification of a set of data and the set of operations that can be performed on the data. They are abstract in the sense that the focus is on the definitions of the constructor that returns an abstract handle that represents the data, and the various operations with their arguments. The actual implementation is not defined, and does not affect the use of the ADT.

For example, rational numbers (numbers that can be written in the form A/B where A and B are integers) cannot be represented natively in a computer. A Rational ADT could be defined as shown below.

Construction:

Create an instance of a rational number ADT using two integers, A and B , where A represents the numerator and B represents the denominator.

12.4.1 Operations

Addition, subtraction, multiplication, division, exponentiation, comparison, simplify, conversion to a real (floating point) number.

To be a complete specification, each operation should be defined in terms of the data. For example, when multiplying two rational numbers a/b and c/d , the result is defined as ac/bd . Typically, inputs, outputs, preconditions, postconditions, and assumptions to the ADT are specified as well.

When realized in a computer program, the ADT is represented by an interface, which shields a corresponding implementation. Users of an ADT are concerned with the interface, but not the implementation, as the implementation can change in the future. ADTs typically seen in textbooks and implemented in programming languages (or their libraries) include:

- String ADT
- List ADT
- Stack (last-in, first-out) ADT
- Queue (first-in, first-out) ADT
- Binary Search Tree ADT

- Priority Queue ADT
- Complex Number ADT (imaginary numbers)

There is a distinction, although sometimes subtle, between the abstract data type and the data structure used in its implementation. For example, a List ADT can be represented using an array-based implementation or a linked-list implementation. A List is an abstract data type with well-defined operations (add element, remove element, etc.) while a linked-list is a pointer-based data structure that can be used to create a representation of a List. The linked-list implementation is so commonly used to represent a List ADT that the terms are interchanged and understood in common use.

Similarly, a Binary Search Tree ADT can be represented in several ways:

Binary tree, AVL tree, red-black tree, array, etc. Regardless of the implementation, the Binary Search Tree always has the same operations (insert, remove, find, etc.)

12.5 Asymptotic Notation

When analyzing data structures in this book, we want to talk about the running times of various operations. The exact running times will, of course, vary from computer to computer and even from run to run on an individual computer. When we talk about the running time of an operation we are referring to the number of computer instructions performed during the operation. Even for simple code, this quantity can be difficult to compute exactly. Therefore, instead of analyzing running times exactly, we will use the so-called big-Oh notation: For a function $f(n)$, $O(f(n))$ denotes a set of functions,

$$O(f(n)) = \{ g(n) : \text{there exists } c > 0, \text{ and } n_0 \text{ such that } g(n) \leq c \cdot f(n) \text{ for all } n \geq n_0 \}$$

Thinking graphically, this set consists of the functions $g(n)$ where $c \cdot f(n)$ starts to dominate $g(n)$ when n is sufficiently large.

We generally use asymptotic notation to simplify functions. For example, in place of $5n \log n + 8n - 200$ we can write $O(n \log n)$. This is proven as follows:

$$\begin{aligned} 5n \log n + 8n - 200 &\leq 5n \log n + 8n \\ &\leq 5n \log n + 8n \log n \text{ for } n \geq 2 \text{ (so that } \log n \geq 1) \\ &\leq 13n \log n. \end{aligned}$$

This demonstrates that the function $f(n) = 5n \log n + 8n - 200$ is in the set $O(n \log n)$ using the constants $c = 13$ and $n_0 = 2$.

A number of useful shortcuts can be applied when using asymptotic notation. First:

$$O(n^{c_1}) \subset O(n^{c_2}) ;$$

for any $c_1 < c_2$. Second: For any constants $a, c > 0$,

$$O(a) \subset O(\log n) \subset O(n^b) \subset O(c^n) :$$

These inclusion relations can be multiplied by any positive value, and they still hold. For example, multiplying by n yields:

$$O(n) \subset O(n \log n) \subset O(n^{1+b}) \subset O(nc^n) .$$

Continuing in a long and distinguished tradition, we will abuse this notation by writing things like $f_1(n) \in O(f(n))$ when what we really mean is $f_1(n) \in O(f(n))$. We will also make statements like “the running time of this operation is $O(f(n))$ ” when this statement should be “the running time of this operation is a member of $O(f(n))$.” These shortcuts are mainly to avoid awkward language and to make it easier to use asymptotic notation within strings of equations. A particularly strange example of this occurs when we write statements like

$$T(n) = 2 \log n + O(1) :$$

Again, this would be more correctly written as

$$T(n) \leq 2 \log n + [\text{some member of } O(1)] :$$

The expression $O(1)$ also brings up another issue. Since there is no variable in this expression, it may not be clear which variable is getting arbitrarily large. Without context, there is no way to tell. In the example above, since the only variable in the rest of the equation is n , we can assume that this should be read as $T(n) = 2 \log n + O(f(n))$, where $f(n) = 1$.

Big-Oh notation is not new or unique to computer science. It was used by the number theorist Paul Bachmann as early as 1894, and is immensely useful for describing the running times of computer algorithms. Consider the following piece of code:

Simple

```
void snippet() {
  for (int i = 0; i < n; i++)
    a[i] = i;
}
```

One execution of this method involves

- 1 assignment ($\text{inti} = 0$),
- $n + 1$ comparisons ($i < n$),
- n increments ($i + +$),
- n array offset calculations ($a[i]$), and
- n indirect assignments ($a[i] = i$).

So we could write this running time as

$$T(n) = a + b(n + 1) + cn + dn + en ;$$

where a , b , c , d , and e are constants that depend on the machine running the code and represent the time to perform assignments, comparisons, increment operations, array offset calculations, and indirect assignments, respectively. However, if this expression represents the running time of two lines of code, then clearly this kind of analysis will not be tractable to complicated code or algorithms. Using big-Oh notation, the running time can be simplified to

$$T(n) = O(n) :$$

Not only is this more compact, but it also gives nearly as much information.

The fact that the running time depends on the constants a , b , c , d , and e in the above example means that, in general, it will not be possible to compare two running times to know which is faster without knowing the values of these constants. Even if we make the effort to determine these constants (say, through timing tests), then our conclusion will only be valid for the machine we run our tests on.

Big-Oh notation allows us to reason at a much higher level, making it possible to analyze more complicated functions. If two algorithms have the same big-Oh running time, then we

won't know which is faster, and there may not be a clear winner. One may be faster on one machine, and the other may be faster on a different machine. However, if the two algorithms have demonstrably different big-Oh running times, then we can be certain that the one with the smaller running time will be faster for large enough values of n .

An example of how big-Oh notation allows us to compare two different functions is which compares the rate of growth of $f_1(n) = 15n$ versus $f_2(n) = 2n \log n$. It might be that $f_1(n)$ is the running time of a complicated linear time algorithm while $f_2(n)$ is the running time of a considerably simpler algorithm based on the divide-and-conquer paradigm. This illustrates that, although $f_1(n)$ is greater than $f_2(n)$ for small values of n , the opposite is true for large values of n . Eventually $f_1(n)$ wins out, by an increasingly wide margin. Analysis using big-Oh notation told us that this would happen, since $O(n) \subset O(n \log n)$.

In a few cases, we will use asymptotic notation on functions with more than one variable. There seems to be no standard for this, but for our purposes, the following definition is sufficient:

$$O(f(n_1, \dots, n_k)) = \{g(n_1, \dots, n_k) : \text{there exists } c > 0, \text{ and } z \text{ such that } g(n_1, \dots, n_k) \leq c \cdot f(n_1, \dots, n_k) \text{ for all } n_1, \dots, n_k \text{ such that } g(n_1, \dots, n_k) \geq z\}$$

This definition captures the situation we really care about: when the arguments $(n_1, \dots, n_k)$ make g take on large values. This definition also agrees with the univariate definition of $O(f(n))$ when $f(n)$ is an increasing function of n .

12.6 Arrays

In data storage, an array is a method for storing information on multiple devices. In general, an array is a number of items arranged in some specified way - for example, in a list or in a three-dimensional table. In computer programming languages, an array is a group of objects with the same attributes that can be addressed individually, using such techniques as subscripting.

General Guidelines. Common to all data structures in C++ are some fundamental rules and guidelines about allocation and deallocation. Here are some guidelines, briefly stated: The method of deallocation should imitate the method of creation.

- If you allocate memory with `new` (and not `[]`), you should free memory with `delete`.
- If you allocate memory with `new` in a loop, you should free memory with `delete` in a loop.

- If you allocate memory with `new` and `[]`, you should free memory with `delete []`.

The following example shows how to define an array at run time:

```
// array.cpp
// compile with: /EHsc
#include <iostream.h>
int main() {
    using namespace std;
    int size = 3, i = 0;
    int* myarr = new int[size];
    for (i = 0 ; i < size ; i++)
        myarr[i] = 10;
    for (i = 0 ; i < size ; i++)
        printf("myarr[%d] = %d\n", i, myarr[i]);
    delete [] myarr;
}
```

Arrays are derived types and can therefore be constructed from any other derived or fundamental type except functions, references, and void.

Arrays constructed from other arrays are multidimensional arrays. These multidimensional arrays are specified by placing multiple bracketed constant expressions in sequence. For example, consider this declaration:

```
int i2[5][7];
```

It specifies an array of type `int`, conceptually arranged in a two-dimensional matrix of five rows and seven columns.

12.6.1 Basic Rules

The guidelines are derived from more fundamental rules about how C++ memory management works:

- If you allocate memory with a declaration, is allocated on the run-time stack and is freed automatically when control leaves the block that contains the declaration.
- Dynamic memory is allocated using `new` and freed using `delete`. It is allocated in a memory segment called “the heap”. If it is not freed, it continues to exist in the heap until program termination.
- If the base type of an array is a class type, you must use `delete[]` to free it. This automatically calls the base-type destructor function for each element of the array, to free the dynamic extensions attached to it.
- As a general habit, use `delete []` to free arrays, even when the `[]` are not needed. Some programming tools and IDE’s become confused when you omit the `[]`. Note, however, that if the base type of an array is a pointer type or a non-class type, it does not matter whether you use `delete` or `delete[]` because there are no destructors to run. Dynamic memory attached to the elements of an array of pointers must be freed by explicit calls on `delete`.
- In one common data structure, the flexible array, an array “grows” by reallocation, as needed. After growth, the array contents are copied from the old array to the new, longer array. In this case, you must free the old array but you must not free anything more than that, so `delete` must be used without the brackets.

12.6.2 Operation on Arrays

There are various operations performed by arrays: the primary operations are Creation, insertion, traverse, sorting and searching

Search

The search algorithm described next is called a sequential or linear search. Consider the list of seven elements shown in below table 1.1.

Position	0	1	2	3	4	5	6	7
List	35	12	27	18	45	16	38	

Table 12.1. list of elements

Suppose that you want to determine whether 27 is in the list. The sequential search works as follows: First, you compare 27 with list[0]—that is, compare 27 with 35. Because list[0] $\neq$ 27, you then compare 27 with list[1] (that is, with 12, the second item in the list). Because list[1] $\neq$ 27, you compare 27 with the next element in the list—that is, compare 27 with list[2]. Because list[2] = 27, the search stops. This is a successful search.

Let us now search for 10. As before, the search starts with the first element in the list—that is, at list[0]. This time the search item, which is 10, is compared with every item in the list. Eventually, no more data is left in the list to compare with the search item. This is an unsuccessful search.

It now follows that, as soon as you find an element in the list that is equal to the search item, you must stop the search and report “success.” (In this case, you usually also tell the location in the list where the search item was found.) Otherwise, after the search item is compared with every element in the list, you must stop the search and report “failure.” Suppose that the name of the array containing the list elements is list. The following function performs a sequential search on a list:

```
template <class elemType>

int arrayListType<elemType>::seqSearch(const elemType& item) const
{
    int loc;
    bool found = false;
    for (loc = 0; loc < length; loc++)
        if (list[loc] == item)
        {
            found = true;
            break;
        }

    if (found)
        return loc;
```



```

else
return -1;
} //end seqSearch

```

Now that we know how to implement the (sequential) search algorithm, we can give the definitions of the functions insert and remove. Recall that the function insert inserts a new item at the end of the list if this item does not exist in the list and the list is not full. The function remove removes an item from the list if the list is not empty.

Insert

The function insert inserts a new item in the list. Because duplicates are not allowed, this function first searches the list to determine whether the item to be inserted is already in the list. To determine whether the item to be inserted is already in the list, this function calls the member function seqSearch, described previously. If the item to be inserted is not in the list, the new item is inserted at the end of the list and the length of the list is increased by 1. Also, the item to be inserted is passed as a parameter to this function. The definition of this function is as follows:

```

template <class elemType>
void arrayListType<elemType>::insert(const elemType& insertItem)
{
int loc;
if (length == 0) //list is empty
list[length++] = insertItem; //insert the item and
//increment the length
else if (length == maxSize)
cerr << "Cannot insert in a full list." << endl;
else
{
loc = seqSearch(insertItem);
if (loc == -1) //the item to be inserted
//does not exist in the list
list[length++] = insertItem;

```

```

else
    cerr << "the item to be inserted is already in " << "the list. No duplicates are allowed."
    << endl;
}
} //end insert

```

The function insert uses the function seqSearch to determine whether the insertItem is already in the list. Because the function seqSearch is of $O(n)$, it follows that the function insert is of $O(n)$.

Remove

The function remove deletes an item from the list. The item to be deleted is passed as a parameter to this function. To delete the item, the function calls the member function seqSearch to determine whether the item to be deleted is in the list. If the item to be deleted is found in the list, the item is removed from the list and the length of the list is decremented by 1. If the item to be removed is found in the list, the function seqSearch returns the index of the item in the list to be deleted. We can now use the index returned by the function seqSearch, and use the function removeAt to remove the item from the list. Therefore, the definition of the function remove is as follows:

```

template<class elemType>
void arrayListType<elemType>::remove(const elemType& removeItem)
{
    int loc;
    if (length == 0)
        cerr << "Cannot delete from an empty list." << endl;
    else
    {
        loc = seqSearch(removeItem);
        if (loc != -1)
            removeAt(loc);
    }
}

```

```

cout << "The item to be deleted is not in the list."
<< endl;
}
} //end remove

```

The function remove uses the functions seqSearch and removeAt to remove an item from the list. Because each of these functions is of $O(n)$ and because they are called in sequence, it follows that the function remove is of $O(n)$.

The following program tests the various operations on array-based lists.

```

#include <iostream.h> //Line 1
#include <string> //Line 2
#include "arrayListType.h" //Line 3
using namespace std; //Line 4
int main() //Line 5
{ //Line 6
    arrayListType<int> intList(100); //Line 7
    arrayListType<string> stringList; //Line 8
    int number; //Line 9
    cout << "List 10: Enter 5 integers: "; //Line 10
    for (int counter = 0; counter < 5; counter++) //Line 11
    { //Line 12
        cin >> number; //Line 13
        intList.insertAt(counter, number); //Line 14
    } //Line 15
    cout << endl; //Line 16
    cout << "List 19: The list you entered is: "; //Line 17
    intList.print(); //Line 18
    cout << endl; //Line 19
    cout << "Line 20: Enter the item to be deleted: "; //Line 20

```

```

cin >> number; //Line 21
intList.remove(number); //Line 22
cout << "Line 23: After removing " << number << ", the list is:" << endl;
//Line 23
intList.print(); //Line 24
cout << endl; //Line 25
string str; //Line 26
cout << "Line 27: Enter 5 strings: "; //Line 27
for (int counter = 0; counter < 5; counter++) //Line 28
{ //Line 29
    cin >> str; //Line 30
    stringList.insertAt(counter, str); //Line 31
} //Line 32
cout << endl; //Line 33
cout << "Line 34: The list you entered is: " << endl; //Line 34
stringList.print(); //Line 35
cout << endl; //Line 36
cout << "Line 37: Enter the string to be deleted: "; //Line 37
cin >> str; //Line 38
stringList.remove(str); //Line 39
cout << "Line 40: After removing " << str << ", the list is:" << endl; //Line 40
stringList.print(); //Line 41
cout << endl; //Line 42
return 0; //Line 43
} //Line 44

```

12.7 Polynomial

You learned in college algebra or calculus course that a polynomial, $p(x)$, in one variable, x , is an expression of the form:

$$p(x) = a_0 + a_1x + \dots + a_nx^n$$

Where a_i is real (or complex) numbers and n is a nonnegative integer. If $p(x) = a_0$, $p(x)$ is called a constant polynomial. If $p(x)$ is a nonzero constant polynomial, the degree of $p(x)$ is defined to be 0. Even though, in mathematics, the degree of the zero polynomial is undefined, for the purpose of this program, we consider the degree of such polynomials to be zero. If $p(x)$ is not constant and $a_n \neq 0$, n is called the degree of $p(x)$; that is, the degree of a nonconstant polynomial is defined to be the exponent of the highest power of x . (Note that the symbol " $\neq$ " means not equal to.)

The basic operations performed on polynomials are add, subtract, multiply, divide, and evaluate a polynomial at any given point. For example, suppose that

$$P(x) = 1 + 2x + 3x^2$$

And

$$Q(x) = 4 + x.$$

$$P(x) + Q(x) = 5 + 3x + 3x^2$$

$$P(x) - Q(x) = -3 + x + 3x^2$$

$$P(x) * Q(x) = 4 + 9x + 14x^2 + 3x^3$$

The purpose of this programming example is to design and implement the class `polynomialType` to perform the various polynomial operations in a program. To be specific, in this program, we will implement the following operations on polynomials:

1. Evaluate a polynomial at a given value.
2. Add polynomials.
3. Subtract polynomials.
4. Multiply polynomials.

Furthermore, we assume that the coefficients of polynomials are real numbers. You will be asked in Programming Exercise 8 to generalize it so that the coefficients can also be complex numbers. To store a polynomial, we use a dynamic array as follows:

Suppose $p(x)$ is a polynomial of degree $n > 0$. Let `list` be an array of size $n + 1$. The coefficient a_i of x^i is stored in `list[i]`.

```

// This class specifies the members to implement the basic
// polynomial operations.
//*****

class polynomialType: public arrayListType<double>
{
friend ostream& operator<<(ostream&, const polynomialType&);
//Overloads the stream insertion operator
friend istream& operator>>(istream&, polynomialType&);
//Overloads the stream extraction operator
public:
polynomialType operator+(const polynomialType&);
//Overloads the operator +
polynomialType operator-(const polynomialType&);
//Overloads the operator -
polynomialType operator*(const polynomialType&);
//Overloads the operator *
double operator() (double x);
//Overloads the operator () to evaluate the polynomial at a given point
//Postcondition: The value of the polynomial at x is calculated and returned
polynomialType(int size = 100);
//constructor
int min(int x, int y) const;
//Function to return the smaller of x and y
int max(int x, int y) const;
//Function to return the larger of x and y
};

```

If $p(x)$ is a polynomial of degree 3, we can create an object, say p , of type `polynomialType` and set the size of the array list to 4. The following statement declares such an

object p:

```
polynomialType p(4);
```

The degree of the polynomial is stored in the data member length, which is inherited from the class arrayListType.

Next we discuss the definitions of the functions. The constructor sets the value of length to the size of the array and initializes the array list to 0.

```
polynomialType::polynomialType(int size)
: arrayListType<double>(size)
{
    length = size;
    for (int i = 0; i < size; i++)
        list[i] = 0;
}
```

The definition of the function to overload the operator () is quite straightforward and is given next.

```
double polynomialType::operator() (double x)
{
    double value = 0.0;
    for (int i = 0; i < length; i++)
    {
        if (list[i] != 0.0)
            value = value + list[i] * pow(x,i);
    }
    return value;
}
```

Suppose that $p(x)$ is a polynomial of degree n and $q(x)$ is a polynomial of degree m . If $n = m$, the operator $+$ adds the corresponding coefficients of $p(x)$ and $q(x)$. If $n > m$, the first m

coefficients of $p(x)$ are added with the corresponding coefficients of $q(x)$. The remaining coefficients of $p(x)$ are copied into the polynomial containing the sum of $p(x)$ and $q(x)$. Similarly, if $n < m$, the first n coefficients of $q(x)$ are added with the corresponding coefficients of $p(x)$. The remaining coefficients of $q(x)$ are copied into the polynomial containing the sum. The definition of the operator $-$ is similar to the definition of the operator $+$. The definitions of these two operator functions are as follows:

```

polynomialType polynomialType::operator+ (const polynomialType& right)
{
    int size = max(length, right.length);
    polynomialType temp(size); //polynomial to store the sum
    for (int i = 0; i < min(length, right.length); i++)
        temp.list[i] = list[i] + right.list[i];
    if (size == length)
        for (int i = min(length, right.length); i < length; i++)
            temp.list[i] = list[i];
    else
        for (int i = min(length, right.length); i < right.length; i++)
            temp.list[i] = right.list[i];
    return temp;
}

polynomialType polynomialType::operator- (const polynomialType& right)
{
    int size = max(length, right.length);
    polynomialType temp(size); //polynomial to store the difference
    for (int i = 0; i < min(length, right.length); i++)
        temp.list[i] = list[i] - right.list[i];
    if (size == length)
        for (int i = min(length, right.length); i < length; i++)

```



```

temp.list[i] = list[i];
else
for (int i = min(length, right.length); i < right.length; i++)
temp.list[i] = -right.list[i];
return temp;
}

```

The definition of the function to overload the operator * to multiply two polynomials is left as an exercise for you.

12.8 Summary

A data structure is a specialized format for organizing and storing data. General data structure types include the array, the file, the record, the table, the tree, and so on. Any data structure is designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate ways. In computer programming, a data structure may be selected or designed to store data for the purpose of working on it with various algorithms.

12.9 Review Questions

1. What is data structure?
2. Define algorithm.
3. Define ADT.
4. Explain the operations on ADT.
5. Write short note on Asymptotic notations.
6. Define arrays.
7. Explain the operation on arrays.
8. Define polynomial. Explain the polynomial addition procedure with example.

Lesson 13

LINKED LIST

Structure

13.1 Introduction

13.2 Learning objectives

13.3 Single linked list

13.4 Doubly linked list

13.5 Circular linked list

13.6 Summary

13.7 Review questions

13.1 Introduction

A linked list is a data structure consisting of a group of nodes which together represent a sequence. Under the simplest form, each node is composed of a data and a reference (in other words, a link) to the next node in the sequence; more complex variants add additional links. This structure allows for efficient insertion or removal of elements from any position in the sequence.

13.2 Learning objectives

At the end of this lesson you can able to

- Single linked list
- Doubly linked list
- Circular linked list

13.3 Single linked list

Each record of a linked list is often called an element or node. The field of each node that contains the address of the next node is usually called the next link or next pointer. The remaining fields are known as the data, information, value, cargo, or payload fields. The head of a list is

its first node. The tail of a list may refer either to the rest of the list after the head, or to the last node in the list. (see figure 13.1)

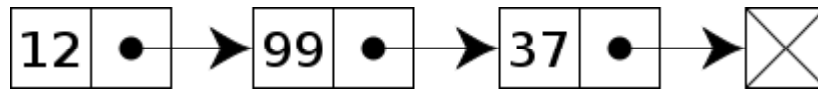


Figure 13.1. Single linked list

Linked lists are among the simplest and most common data structures. They can be used to implement several other common abstract data types, including lists (the abstract data type), stacks, queues, associative arrays, and expressions, though it is not uncommon to implement the other data structures directly without using a list as the basis of implementation.

The principal benefit of a linked list over a conventional array is that the list elements can easily be inserted or removed without reallocation or reorganization of the entire structure because the data items need not be stored contiguously in memory or on disk. Linked lists allow insertion and removal of nodes at any point in the list, and can do so with a constant number of operations if the link previous to the link being added or removed is maintained during list traversal.

13.3.1 Defining the data structure for linked list

The key part of a linked list is a structure, which holds the data for each node (the name, address, age or whatever for the items in the list), and, most importantly, a pointer to the next node. Here given the structure of a typical node:

```

struct Node
{
    char name[20];
    int age;
    float height;
    Node *next; // Pointer to next node
};

Node *start_ptr = NULL; // Start Pointer (root)
  
```

The important part of the structure is the line before the closing curly brackets. This gives a pointer to the next node in the list. This is the only case in C++ where you are allowed to refer to a data type (in this case Node) before you have even finished defining it.

We have also declared a pointer called start_ptr which will permanently point to the start of the list. To start with, there are no nodes in the list, which is why start_ptr is set to NULL.

13.3.2 Adding a node to the end of the list

The first problem that we face is how to add a node to the list. For simplicity's sake, we will assume that it has to be added to the end of the list, although it could be added anywhere in the list. Firstly, we declare the space for a pointer item and assign a temporary pointer to it. This is done using the new statement as follows:

```
Node *temp;
temp = new Node;
```

We can refer to the new node as *temp, i.e. "the node that temp points to". Having declared the node, we ask the user to fill in the details of the person, i.e. the name, age, address or whatever: `cout << "Please enter the name of the person: "; cin >> temp->name; cout << "Please enter the age of the person : "; cin >> temp->age; cout << "Please enter the height of the person : "; cin >> temp->height; temp->next = NULL;`

The last line sets the pointer from this node to the next to NULL, indicating that this node, when it is inserted in the list, will be the last node. Having set up the information, we have to decide what to do with the pointers. It is harder if there are already nodes in the list. In this case, the secret is to declare a second pointer, temp2, to step through the list until it finds the last node.

```
Node *temp2;
temp2 = start_ptr; // We know temp2 is not NULL - list not empty!
while (temp2->next != NULL)
temp2 = temp2->next; // Move to next link in chain
```

The loop will terminate when temp2 points to the last node in the chain, and it knows when this happened because the next pointer in that node will point to NULL. When it has found it, it sets the pointer from that last node to point to the node we have just declared:

```
temp2->next = temp;
```

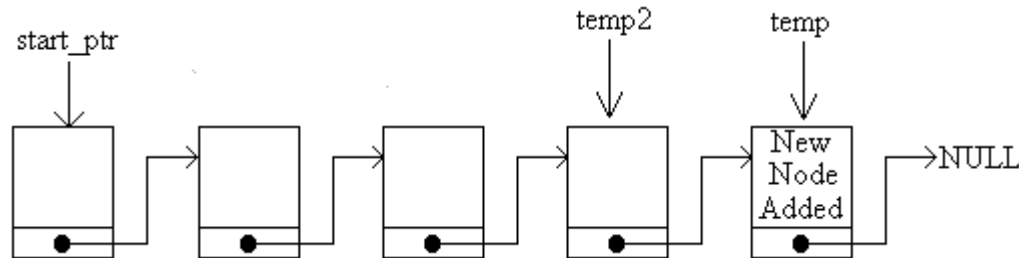


Figure 13.2 Linked two nodes

The link `temp2->next` in this diagram (figure 13.2) is the link joining the last two nodes.

13.3.3 Displaying the list of nodes

Having added one or more nodes, we need to display the list of nodes on the screen. This is comparatively easy to do. You just have to traverse the whole list as we learned it in the previous part. Here is again a summary of the method:

1. Set a temporary pointer to point to the same thing as the start pointer.
2. If the pointer points to NULL, display the message "End of list" and stop.
3. Otherwise, display the details of the node pointed to by the start pointer.
4. Make the temporary pointer point to the same thing as the next pointer of the node it is currently indicating.
5. Jump back to step 2.

The temporary pointer moves along the list, displaying the details of the nodes it comes across. At each stage, it can get hold of the next node in the list by using the next pointer of the node it is currently pointing to.

```
Node *temp;
```

```
temp = start_ptr;
// Display details for what temp points to
cout << "Name : " << temp->name << endl;
cout << "Age : " << temp->age << endl;
cout << "Height : " << temp->height << endl;
cout << endl;
```

13.3.4 Deleting a node from the list

When it comes to deleting nodes, we have three choices: Delete a node from the start of the list, delete one from the end of the list, or delete one from somewhere in the middle. For simplicity, we shall just deal with deleting one from the start or from the end. When a node is deleted, the space that it took up should be reclaimed. Otherwise the computer will eventually run out of memory space. This is done with the delete instruction:

```
delete temp; // Release the memory pointed to by temp
```

However, we can't just delete the nodes willy-nilly as it would break the chain. We need to reassign the pointers and then delete the node at the last moment. Here is how we go about deleting the first node in the linked list:

```
Node *temp;
temp = start_ptr; // Make the temporary pointer
// identical to the start pointer
```

Now that the first node has been safely tagged (so that we can refer to it even when the start pointer has been reassigned), we can move the start pointer to the next node in the chain:

```
start_ptr = start_ptr->next; // Second node in chain.
```

Now that the first node has been safely tagged (so that we can refer to it even when the start pointer has been reassigned), we can move the start pointer to the next node in the chain:

```
start_ptr = start_ptr->next; // Second node in chain.
```

Here is the function that deletes a node from the start:

```
void delete_start_node()
```

```

{
Node *temp;
temp = start_ptr;
start_ptr = start_ptr->next;
delete temp;
}

```

Deleting a node from the end of the list is not much harder, as the temporary pointer must find where the end of the list is by hopping along from the start. This is done using code that is almost identical to that used to insert a node at the end of the list. But it is necessary to maintain two temporary pointers, temp1 and temp2. The pointer temp1 will point to the last node in the list and temp2 will point to the previous node. We have to keep track of both as it is necessary to delete the last node and immediately afterwards, to set the next pointer of the previous node to NULL (it is now the new last node).

1. Look at the start pointer. If it is NULL, then the list is empty, so print out a “No nodes to delete” message.
2. Make temp1 point to whatever the start pointer is pointing to.
3. If the next pointer of what temp1 indicates is NULL, then we’ve found the last node of the list, so jump to step 7.
4. Make another pointer, temp2, point to the current node in the list (where temp1 is pointing to).
5. Make temp1 point to the next item in the list.
6. Go to step 3.
7. If you get this far, then the temporary pointer, temp1, should point to the last item in the list and the other temporary pointer, temp2, should point to the last-but-one item.
8. Delete the node pointed to by temp1.
9. Mark the next pointer of the node pointed to by temp2 as NULL - it is the new last node.

Suppose we want to delete the last node from this list:

Firstly, the start pointer doesn't point to NULL, so we don't have to display a "Empty list, wise guy!" message. Let's get straight on with step2 - set the pointer temp1 to the same as the start pointer. The next pointer from this node isn't NULL, so we haven't found the end node. Instead, we set the pointer temp2 to the same node as temp1 and then move temp1 to the next node in the list. Going back to step 3, we see that temp1 still doesn't point to the last node in the list, so we make temp2 point to what temp1 points to and temp1 is made to point to the next node along. Eventually, this goes on until temp1 really is pointing to the last node in the list, with temp2 pointing to the penultimate node. Now we have reached step 8. The next thing to do is to delete the node pointed to by temp1 and set the next pointer of what temp2 indicates to NULL:

```
{
Node *temp1, *temp2;
if (start_ptr == NULL)
    cout << "The list is empty!" << endl;
else
{
    temp1 = start_ptr;
    while (temp1->next != NULL)
    {
        temp2 = temp1;
        temp1 = temp1->next;
    }
    delete temp1;
    temp2->next = NULL;
}
}
```

13.4 Doubly Linked Lists

There is also an other possibility how you can traverse back- and forward through linked list. But for this we have to adapted our node struct to a double linked list. The idea is very easy.

We add another pointer to each node linking to the previous node. The pointer of the first node is set to NULL similarly as the next pointer of the last node.

```

Struct Node
{
    char name[20];
    int age;
    float height;
    Node *next;
    Node *previous;
};

```

In a doubly linked list, each node contains, besides the next-node link, a second link field pointing to the previous node in the sequence. The two links may be called forward(s) and backwards, or next and prev(ious). (See Figure 13.3)

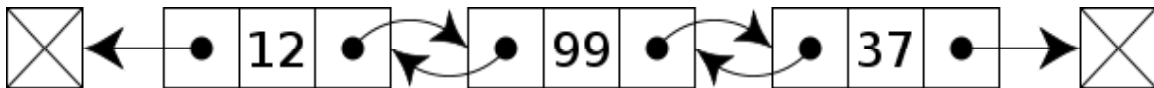


Figure 13.3 Doubly Linked List

A doubly linked list is a linked list in which every node has a next pointer and a back pointer. In other words, every node contains the address of the next node (except the last node), and every node contains the address of the previous node (except the first node).

A doubly linked list can be traversed in either direction. That is, we can traverse the list starting at the first node or, if a pointer to the last node is given, we can traverse the list starting at the last node.

As before, the typical operations on a doubly linked list are as follows: Initialize the list, destroy the list, determine whether the list is empty, search the list for a given item, insert an item, delete an item, and so on. The following class defines a doubly linked list as an ADT and specifies the basic operations on a doubly linked list:

```

//Definition of the node
template <class Type>

```

```

struct nodeType
{
    Type info;
    nodeType<Type> *next;
    nodeType<Type> *back;
};

template <class Type>
class doublyLinkedList
{
public:
    const doublyLinkedList<Type>& operator=
    (const doublyLinkedList<Type> &);
    //Overload the assignment operator.
    Void initializeList();
    //Function to initialize the list to an empty state.
    //Postcondition: first = NULL; last = NULL; count = 0;
    bool isEmptyList() const;
    //Function to determine whether the list is empty.
    //Postcondition: Returns true if the list is empty,
    // otherwise returns false.
    Void destroy();
    //Function to delete all the nodes from the list.
    //Postcondition: first = NULL; last = NULL; count = 0;
    void print() const;
    //Function to output the info contained in each node.
    Void reversePrint() const;
    //Function to output the info contained in each node
    //in reverse order.

```

```

Int length() const;
//Function to return the number of nodes in the list.
//Postcondition: The value of count is returned.
Type front() const;
//Function to return the first element of the list.
//Precondition: The list must exist and must not be empty.
//Postcondition: If the list is empty, the program terminates;
// otherwise, the first element of the list is returned.
Type back() const;
//Function to return the last element of the list.
//Precondition: The list must exist and must not be empty.
//Postcondition: If the list is empty, the program terminates;
// otherwise, the last element of the list is returned.
Bool search(const Type& searchItem) const;
//Function to determine whether searchItem is in the list.
//Postcondition: Returns true if searchItem is found in the
// list, otherwise returns false.
Void insert(const Type& insertItem);
//Function to insert insertItem in the list.
//Precondition: If the list is nonempty, it must be in order.
//Postcondition: insertItem is inserted at the proper place
// in the list, first points to the first node, last points
// to the last node of the new list, and count is incremented by 1.
Void deleteNode(const Type& deleteItem);
//Function to delete deleteItem from the list.
//Postcondition: If found, the node containing deleteItem is
// deleted from the list; first points to the first node of
// the new list, last points to the last node of the new

```

```

// list, and count is decremented by 1; otherwise an appropriate message is printed.
doublyLinkedList();
//default constructor
//Initializes the list to an empty state.
//Postcondition: first = NULL; last = NULL; count = 0;
doublyLinkedList(const doublyLinkedList<Type>& otherList);
//copy constructor
~doublyLinkedList();
//destructor
//Postcondition: The list object is destroyed.
Protected:
int count;
nodeType<Type> *first; //pointer to the first node
nodeType<Type> *last; //pointer to the last node
private:
void copyList(const doublyLinkedList<Type>& otherList);
//Function to make a copy of otherList.
//Postcondition: A copy of otherList is created and assigned
// to this list.
};

```

Here, every node has two pointers, back and next, some of the operations require the adjustment of two pointers in each node. For the insert and delete operations, because we can traverse the list in either direction, we use only one pointer to traverse the list. Let us call this pointer current. We can set the value of trailCurrent by using both the current pointer and the back pointer of the node pointed to by current. We give the definition of each function here, with four exceptions. Definitions of the functions copyList, the copy constructor, overloading the assignment operator, and the destructor are left as exercises for you. Furthermore, the function copyList is used only to implement the copy constructor and overload the assignment operator.

Default Constructor

The default constructor initializes the doubly linked list to an empty state. It sets first and last to NULL and count to 0.

```
template <class Type>
doublyLinkedList<Type>::doublyLinkedList()
{
    first= NULL;
    last = NULL;
    count = 0;
}
```

isEmptyList

This operation returns true if the list is empty; otherwise, it returns false. The list is empty if the pointer first is NULL.

```
template <class Type>
bool doublyLinkedList<Type>::isEmptyList() const
{
    return (first == NULL);
}
```

Destroy the List

This operation deletes all the nodes in the list, leaving the list in an empty state. We traverse the list starting at the first node and then delete each node. Furthermore, count is set to 0.

```
template <class Type>
void doublyLinkedList<Type>::destroy()
{
    nodeType<Type> *temp; //pointer to delete the node
    while (first != NULL)
```

```

{
temp = first;
first = first->next;
delete temp;
}
last = NULL;
count = 0;
}

```

Initialize the List

This operation reinitializes the doubly linked list to an empty state. This task can be done by using the operation destroy. The definition of the function initializeList is as follows:

```

template <class Type>
void doublyLinkedList<Type>::initializeList()
{
destroy();
}

```

Length of the List

The length of a linked list (that is, how many nodes are in the list) is stored in the variable count. Therefore, this function returns the value of this variable.

```

template <class Type>
int doublyLinkedList<Type>::length() const
{
return count;
}

```

Print the List

The function print outputs the info contained in each node. We traverse the list starting from the first node.

```
template <class Type>
void doublyLinkedList<Type>::print() const
{
    nodeType<Type> *current; //pointer to traverse the list
    current = first; //set current to point to the first node
    while (current != NULL)
    {
        cout << current->info << " "; //output info
        current = current->next;
    } //end while
} //end print
```

Search the List

The function search returns true if searchItem is found in the list; otherwise, it returns false. The search algorithm is exactly the same as the search algorithm for an ordered linked list.

```
template <class Type>
bool doublyLinkedList<Type>::search(const Type& searchItem) const
{
    bool found = false;
    nodeType<Type> *current; //pointer to traverse the list
    current = first;
    while (current != NULL && !found)
    {
        if (current->info == searchItem)
        {
            found = true;
        }
        else
        {
            current = current->next;
        }
    }
}
```

```

if (found)

found = (current->info == searchItem); //test for equality

return found;

} //end search

```

First and Last Elements

The function front returns the first element of the list and the function back returns the last element of the list. If the list is empty, both functions terminate the program. Their definitions are as follows:

```

template <class Type>

Type doublyLinkedList<Type>::front() const
{
    assert(first != NULL);
    return first->info;
}

template <class Type>

Type doublyLinkedList<Type>::back() const
{
    assert(last != NULL);
    return last->info;
}

```

13.4.1 INSERT A NODE

Because we are inserting an item in a doubly linked list, the insertion of a node in the list requires the adjustment of two pointers in certain nodes. As before, we find the place where the new item is supposed to be inserted, create the node, store the new item, and adjust the link fields of the new node and other particular nodes in the list.

There are four cases:

Case 1: Insertion in an empty list

Case 2: Insertion at the beginning of a nonempty list

Case 3: Insertion at the end of a nonempty list

Case 4: Insertion somewhere in a nonempty list

Both cases 1 and 2 require us to change the value of the pointer first. Cases 3 and 4 are similar. After inserting an item, count is incremented by 1. Next, we show case 4. The definition of the function insert is as follows:

```
template <class Type>
void doublyLinkedList<Type>::insert(const Type& insertItem)
{
    nodeType<Type> *current; //pointer to traverse the list
    nodeType<Type> *trailCurrent; //pointer just before current
    nodeType<Type> *newNode; //pointer to create a node
    bool found;
    newNode = new nodeType<Type>; //create the node
    newNode->info = insertItem; //store the new item in the node
    newNode->next = NULL;
    newNode->back = NULL;
    if (first == NULL) //if list is empty, newNode is the only node
    {
        first = newNode;
        last = newNode;
        count++;
    }
    Else
    {
        found = false;
        current = first;
        while (current != NULL && !found) //search the list
            if (current->info >= insertItem)
```

```
found = true;
else
{
    trailCurrent = current;
    current = current->next;
}if (current == first) //insert newNode before first
{
    first->back = newNode;
    newNode->next = first;
    first = newNode;
    count++;
}
else
{
    //insert newNode between trailCurrent and current
    if (current != NULL)
    {
        trailCurrent->next = newNode;
        newNode->back = trailCurrent;
        newNode->next = current;
        current->back = newNode;
    }
    else
    {
        trailCurrent->next = newNode;
        newNode->back = trailCurrent;
        last = newNode;
    }
}
```

```

count++;
} //end else
} //end else
} //end insert

```

13.4.2 DELETE A NODE

This operation deletes a given item (if found) from the doubly linked list. As before, we first search the list to see whether the item to be deleted is in the list. The search algorithm is the same as before. Similar to the insert operation, this operation (if the item to be deleted is in the list) requires the adjustment of two pointers in certain nodes. The delete operation has several cases:

Case 1: The list is empty.

Case 2: The item to be deleted is in the first node of the list, which would require us to change the value of the pointer first.

Case 3: The item to be deleted is somewhere in the list.

Case 4: The item to be deleted is not in the list.

After deleting a node, count is decremented by 1.

The definition of the function deleteNode is as follows:

```

template <class Type>
void doublyLinkedList<Type>::deleteNode(const Type& deleteltem)
{
    nodeType<Type> *current; //pointer to traverse the list
    nodeType<Type> *trailCurrent; //pointer just before current
    bool found;
    if (first == NULL)
        cout << "Cannot delete from an empty list." << endl;
    else if (first->info == deleteltem) //node to be deleted is
        //the first node

```

```

{
    current = first;
    first = first->next;
    if (first != NULL)
        first->back = NULL;
    else
        last = NULL;
    count--;
    delete current;
}
else
{
    found = false;
    current = first;
    while (current != NULL && !found) //search the list
        if (current->info == deleteItem)
        {
            found = true;
            else
            current = current->next;
            if (current == NULL)
            cout << "The item to be deleted is not in " << "the list." << endl;
            else if (current->info == deleteItem) //check for equality
            {
                trailCurrent = current->back;
                trailCurrent->next = current->next;
                if (current->next != NULL)
                    current->next->back = trailCurrent;
                if (current == last)

```

```

last = trailCurrent;
count--;
delete current;
}
else
cout << "The item to be deleted is not in list." endl;
} //end else
} //end deleteNode

```

13.5 Circular linked list

A linked list in which the last node points to the first node is called a circular linked list.

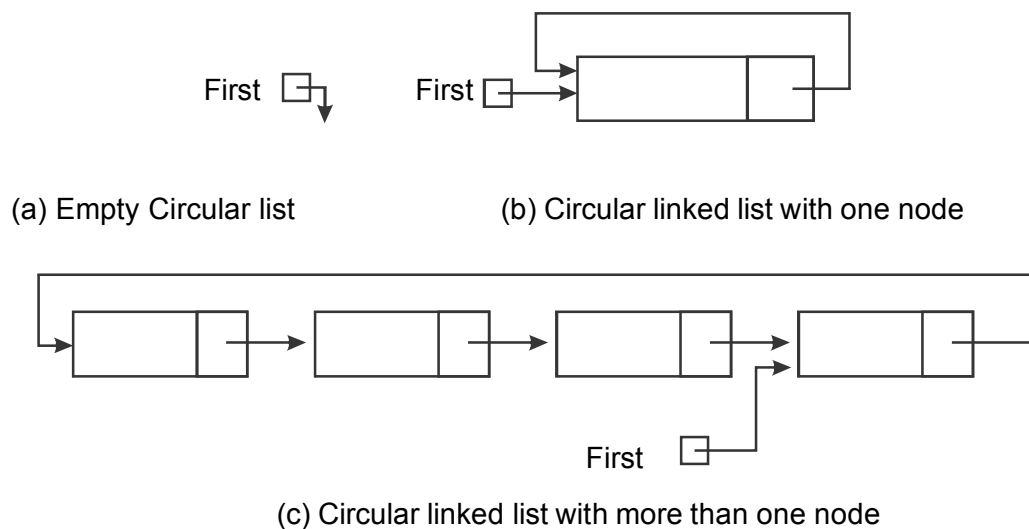


Figure 13.4 shows various circular linked lists.

In a circular linked list with more than one node, as in Figure it is convenient to make the pointer first point to the last node of the list. Then by using first you can access both the first and the last node of the list. For example, first points to the last node and first->link points to the first node.

As before, the usual operations on a circular list are as follows: Initialize the list (to an empty state), determine if the list is empty, destroy the list, print the list, find the length of the list, search the list for a given item, insert an item in the list, delete an item from the list, and copy the list.

The RetrieveItem, InsertItem, and DeleteItem operations all require a search of the list. Rather than rewriting each of these with minor variations, let's write a generalFindItem routine that takes item as a parameter and returns location, predLoc, and found. InsertItem and DeleteItem need the location of the predecessor node (predLoc); RetrieveItem can just ignore it.

In the linear list implementation, we searched the list using a pair of pointers, location and predLoc. (Remember the inchworm?) We modify this approach slightly for the circular list. In the linear list version, we initialized location to point to the first node in the list and set predLoc to NULL. For the circular list search, we initialize location to point to the first node and predLoc to point to its "predecessor"-the last node in the list.

The search loop executes until (1) a key greater than or equal to the item's key is encountered, or (2) we reach the "end" of the list. In a linear list, the end of the list is detected when location equals NULL. Because the external pointer to the list points to the last element, we know we have processed all of the items and not found a match when location points to the first element again: location = listData->next. Because it makes no sense to search an empty list, let's make it a precondition that the list is not empty. Because we now have overloaded the relational operators "<" and "==", we use them in our algorithm.

```

Set location to Next(listData)
Set predLoc to listData.
Set found to false
Set moreToSearch to true
while moreToSearch AND NOT found DO
    if item < Info(location)
        Set moreToSearch to false
    else if item == Info(location)
        Set found to true
    else
        Set predLoc to location
        Set location to Next(location)
        Set moreToSearch to (location != Next(listData))

```

13.6 Summary

A linked list is a list of items, called nodes, in which the order of the nodes is determined by the address, called a link, stored in each node. The pointer to a linked list—that is, the pointer to the first node in the list—is stored in a separate location, called the head or first. A linked list is a dynamic data structure. A linked list in which the last node points to the first node is called a circular linked list.

13.7 Review Questions

1. Define linked list.
2. Define single linked list.
3. Explain the operations of single linked list.
4. Define doubly linked list.
5. How do insert a node into doubly linked list? Explain.
6. Explain the deletion procedure for doubly linked list.
7. Discuss about circular linked list.

LESSON 14

STACKS & QUEUES

Structure

- 14.1 Introduction**
- 14.2 Learning Objectives**
- 14.3 Stacks**
- 14.4 Queues**
- 14.5 Evaluation of Expression**
- 14.6 Summary**
- 14.7 Review Questions**

14.1 INTRODUCTION

Linear data structures are a kind of data structure that has homogenous elements. Each element is referred to by an index. A good example of a linear data structure is a 2-D array. A stack is a list of homogenous elements in which the addition and deletion of elements occurs only at one end, called the top of the stack.

14.2 Learning Objectives

At the end of this lesson you can be able to know –

- Stacks
- Queues
- Application of stack
- Applications of Queue
- Evaluation of Expression

14.3 STACK

A stack is a list of homogenous elements in which the addition and deletion of elements occurs only at one end, called the top of the stack. For example, in a cafeteria, the second tray

in a stack of trays can be removed only if the first tray has been removed. For another example, to get to your favorite computer science book, which is underneath your math and history books, you must first remove the math and history books. After removing these books, the computer science book becomes the top book—that is, the top element of the stack. Figure 14.1 shows some examples of stacks.

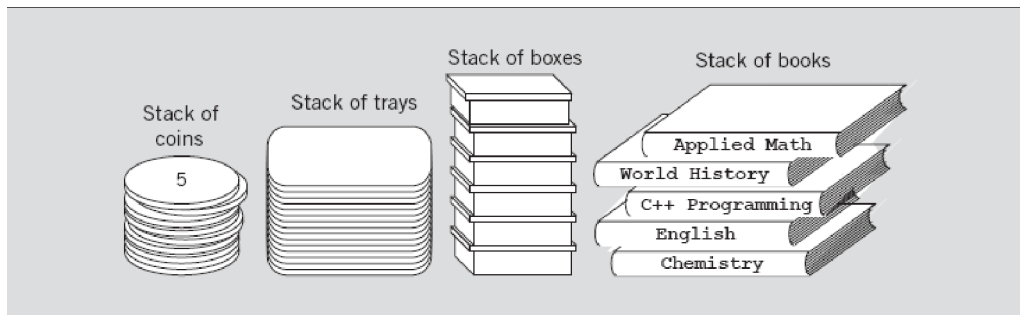


Figure 14.1 Various Examples of Stack

The elements at the bottom of the stack have been in the stack the longest. The top element of the stack is the last element added to the stack. Because the elements are added and removed from one end (that is, the top), it follows that the item that is added last will be removed first. For this reason, a stack is also called a Last In First Out (LIFO) data structure.

Stack: A data structure in which the elements are added and removed from one end only; a Last In First Out (LIFO) data structure.

Now that you know what a stack is, let us see what kinds of operations can be performed on a stack. Because new items can be added to the stack, we can perform the add operation, called push, to add an element onto the stack. Similarly, because the top item can be retrieved and/or removed from the stack, we can perform the operation top to retrieve the top element of the stack, and the operation pop to remove the top element from the stack.

The push, and pop operations work as follows: Suppose there are boxes lying on the floor that need to be stacked on a table. Initially, all of the boxes are on the floor and the stack is empty.

An element can be removed from the stack only if there is something in the stack, and an element can be added to the stack only if there is room. The two operations that immediately follow from push, top, and pop are isFullStack (checks whether the stack is full) and isEmptyStack (checks whether the stack is empty). Because a stack keeps changing as we add

and remove elements, the stack must be empty before we first start using it. Thus, we need another operation, called `initializeStack`, which initializes the stack to an empty state. Therefore, to successfully implement a stack, we need at least these six operations, which are described next. (We might also need other operations on a stack, depending on the specific implementation.)

- `initializeStack`—Initializes the stack to an empty state.
- `isEmptyStack`—Determines whether the stack is empty. If the stack is empty, it returns the value `true`; otherwise, it returns the value `false`.
- `isFullStack`—Determines whether the stack is full. If the stack is full, it returns the value `true`; otherwise, it returns the value `false`.
- `push`—Adds a new element to the top of the stack. The input to this operation consists of the stack and the new element. Prior to this operation, the stack must exist and must not be full.
- `top`—Returns the top element of the stack. Prior to this operation, the stack must exist and must not be empty.
- `pop`—Removes the top element of the stack. Prior to this operation, the stack must exist and must not be empty.

The following abstract class `stackADT` defines these operations as an ADT:

```
//*****

// This class specifies the basic operations on a stack.

//*****

template <class Type>
class stackADT
{
public:
    virtual void initializeStack() = 0;
    //Method to initialize the stack to an empty state.
    //Postcondition: Stack is empty.
```

```

virtual bool isEmptyStack() const = 0;
//Function to determine whether the stack is empty.
//Postcondition: Returns true if the stack is empty, otherwise returns false.
virtual bool isFullStack() const = 0;
//Function to determine whether the stack is full.
//Postcondition: Returns true if the stack is full, otherwise returns false.
virtual void push(const Type& newItem) = 0;
//Function to add newItem to the stack.
//Precondition: The stack exists and is not full.
//Postcondition: The stack is changed and newItem is added to the top of the
stack.
virtual Type top() const = 0;
//Function to return the top element of the stack.
//Precondition: The stack exists and is not empty.
//Postcondition: If the stack is empty, the program terminates; otherwise, the top
element of the //stack is returned.
virtual void pop() = 0;
//Function to remove the top element of the stack.
//Precondition: The stack exists and is not empty.
//Postcondition: The stack is changed and the top element is removed from the
stack.
};

```

We now consider the implementation of our abstract stack data structure. Functions such as push and pop that are required to implement a stack are not available to C++ programmers. We must write the functions to implement the stack operations. Because all the elements of a stack are of the same type, a stack can be implemented as either an array or a linked structure.

14.3.1 Implementation of Stacks as Arrays

Because all the elements of a stack are of the same type, you can use an array to implement a stack. The first element of the stack can be put in the first array slot, the second element of the stack in the second array slot, and so on. The top of the stack is the index of the last element added to the stack.

In this implementation of a stack, stack elements are stored in an array, and an array is a random access data structure; that is, you can directly access any element of the array. However, by definition, a stack is a data structure in which the elements are accessed (popped or pushed) at only one end—that is, a Last In First Out data structure. Thus, a stack element is accessed only through the top, not through the bottom or middle. This feature of a stack is extremely important and must be recognized in the beginning.

To keep track of the top position of the array, we can simply declare another variable, called `stackTop`.

The following class, `stackType`, implements the functions of the abstract class `stackADT`. By using a pointer, we can dynamically allocate arrays, so we will leave it for the user to specify the size of the array (that is, the stack size). We assume that the default stack size is 100. Because the class `stackType` has a pointer member variable (the pointer to the array to store the stack elements), we must overload the assignment operator and include the copy constructor and destructor. Moreover, we give a generic definition of the stack. Depending on the specific application, we can pass the stack element type when we declare a stack object.

```
template <class Type>
class stackType: public stackADT<Type>
{
public:
    const stackType<Type>& operator=(const stackType<Type>&);
    //Overload the assignment operator.
    void initializeStack();
    //Function to initialize the stack to an empty state.
    //Postcondition: stackTop = 0;
```

```

bool isEmptyStack() const;
//Function to determine whether the stack is empty.
//Postcondition: Returns true if the stack is empty,
// otherwise returns false.
bool isFullStack() const;
//Function to determine whether the stack is full.
//Postcondition: Returns true if the stack is full,
// otherwise returns false.
void push(const Type& newlItem);
//Function to add newlItem to the stack.
//Precondition: The stack exists and is not full.
//Postcondition: The stack is changed and newlItem is
// added to the top of the stack.
Type top() const;
//Function to return the top element of the stack.
//Precondition: The stack exists and is not empty.
//Postcondition: If the stack is empty, the program
// terminates; otherwise, the top element of the stack
// is returned.
void pop();
//Function to remove the top element of the stack.
//Precondition: The stack exists and is not empty.
//Postcondition: The stack is changed and the top element is
// removed from the stack.
stackType(int stackSize = 100);
//Constructor
//Create an array of the size stackSize to hold
//the stack elements. The default stack size is 100.

```

```

//Postcondition: The variable list contains the base address
// of the array, stackTop = 0, and maxStackSize = stackSize
stackType(const stackType<Type>& otherStack);
//Copy constructor
~stackType();
//Destructor
//Remove all the elements from the stack.
//Postcondition: The array (list) holding the stack
// elements is deleted.
private:
int maxStackSize; //variable to store the maximum stack size
int stackTop; //variable to point to the top of the stack
Type *list; //pointer to the array that holds the stack elements
void copyStack(const stackType<Type>& otherStack);
//Function to make a copy of otherStack.
//Postcondition: A copy of otherStack is created and assigned
// to this stack.
};

```

Initialize Stack

Let us consider the initializeStack operation. Because the value of stackTop indicates whether the stack is empty, we can simply set stackTop to 0 to initialize the stack.

The definition of the function initializeStack is as follows:

```

template <class Type>
void stackType<Type>::initializeStack()
{
    stackTop = 0;
} //end initializeStack

```

Empty Stack

We have seen that the value of `stackTop` indicates whether the stack is empty. If `stackTop` is 0, the stack is empty; otherwise, the stack is not empty. The definition of the function `isEmptyStack` is as follows:

```
template <class Type>
bool stackType<Type>::isEmptyStack() const
{
    return(stackTop == 0);
} //end isEmptyStack
```

Full Stack

Next, we consider the operation `isFullStack`. It follows that the stack is full if `stackTop` is equal to `maxStackSize`. The definition of the function `isFullStack` is as follows:

```
template <class Type>
bool stackType<Type>::isFullStack() const
{
    return(stackTop == maxStackSize);
} //end isFullStack
```

Push

Adding, or pushing, an element onto the stack is a two-step process. Recall that the value of `stackTop` indicates the number of elements in the stack, and `stackTop - 1` gives the position of the top element of the stack. Therefore, the push operation is as follows:

1. Store the `newItem` in the array component indicated by `stackTop`.
2. Increment `stackTop`.

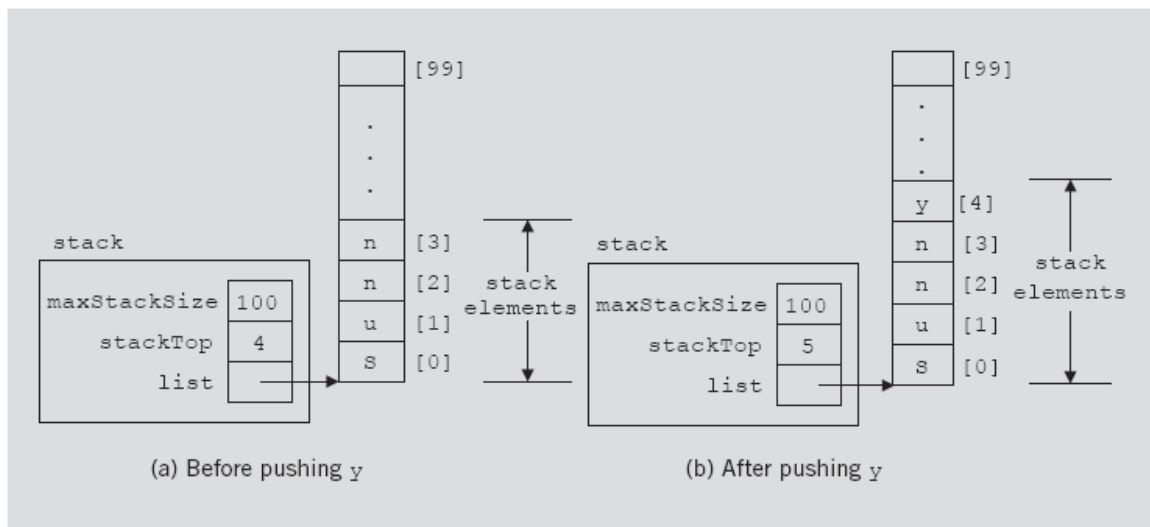


Figure 14.2(a) Shows the stack before pushing 'y' into stack

Figure 14.2(b) Shows the stack after pushing 'y' into the stack

The definition of the function push is as follows:

```
template <class Type>
void stackType<Type>::push(const Type& newItem)
{
    if (!isFullStack())
    {
        list[stackTop] = newItem; //add newItem at the top
        stackTop++; //increment stackTop
    }
    else
        cout << "Cannot add to a full stack." << endl;
} //end push
```

If we try to add a new item to a full stack, the resulting condition is called an overflow.

Error checking for an overflow can be handled in different ways. One way is as shown previously. Or, we can check for an overflow before calling the function push, as shown next (assuming stack is an object of type stackType).

```
if (!stack.isFullStack())
    stack.push(newItem);
```

Pop

To remove, or pop, an element from the stack, we simply decrement stack Top by 1.

Figure 14-3(a) shows the stack before popping 'D' from the stack. Figure 14.3(b) shows the stack after popping 'D' from the stack.

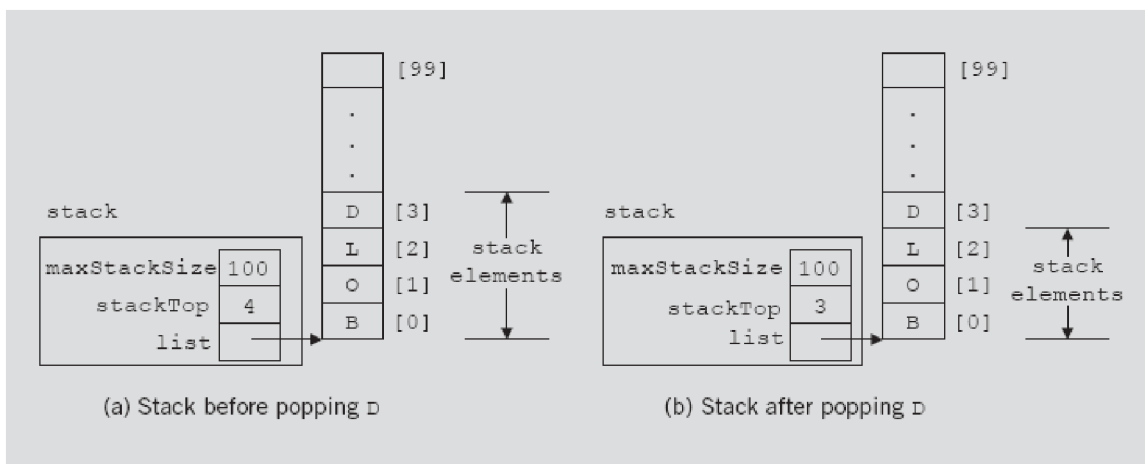


Figure 14.3 Stack before and after the pop operation

The definition of the function pop is as follows:

```
template <class Type>
void stackType<Type>::pop()
{
    if (!isEmptyStack())
        stackTop-; //decrement stackTop
    else
```

```
cout << "Cannot remove from an empty stack." << endl;
} //end pop
```

If we try to remove an item from an empty stack, the resulting condition is called an underflow. Error checking for an underflow can be handled in different ways. One way is as shown previously. Or, we can check for an underflow before calling the function pop, as shown next (assuming stack is an object of type stackType).

```
if (!stack.isEmptyStack())
    stack.pop();
```

14.4 Queue

This section discusses another important data structure, called a queue. The notion of a queue in computer science is the same as the notion of the queues to which you are accustomed in everyday life. There are queues of customers in a bank or in a grocery store and queues of cars waiting to pass through a tollbooth. Similarly, because a computer can send a print request faster than a printer can print, a queue of documents is often waiting to be printed at a printer. The general rule to process elements in a queue is that the customer at the front of the queue is served next and that when a new customer arrives; he or she stands at the end of the queue. That is, a queue is a First In First Out data structure.

Queues have numerous applications in computer science. Whenever a system is modeled on the First In First Out principle, queues are used. At the end of this section, we will discuss one of the most widely used applications of queues, computer simulation. First, however, we need to develop the tools necessary to implement a queue.

A queue is a set of elements of the same type in which the elements are added at one end, called the back or rear, and deleted from the other end, called the front. For example, consider a line of customers in a bank, wherein the customers are waiting to withdraw/deposit money or to conduct some other business. Each new customer gets in the line at the rear. Whenever a teller is ready for a new customer, the customer at the front of the line is served. The rear of the queue is accessed whenever a new element is added to the queue, and the front of the queue is accessed whenever an element is deleted from the queue. As in a stack, the middle elements of the queue are inaccessible, even if the queue elements are stored in an array.

Queue: A data structure in which the elements are added at one end, called the rear, and deleted from the other end, called the front; a First In First Out (FIFO) data structure.

14.4.1 Queue Operations

From the definition of queues, we see that the two key operations are add and delete. We call the add operation `addQueue` and the delete operation `deleteQueue`. Because elements can be neither deleted from an empty queue nor added to a full queue, we need two more operations to successfully implement the `addQueue` and `deleteQueue` operations: `isEmptyQueue` (checks whether the queue is empty) and `isFullQueue` (checks whether a queue is full). We also need an operation, `initializeQueue`, to initialize the queue to an empty state. Moreover, to retrieve the first and last elements of the queue, we include the operations `front` and `back` as described in the following list. Some of the queue operations are as follows:

- `initializeQueue`—Initializes the queue to an empty state.
- `isEmptyQueue`—Determines whether the queue is empty. If the queue is empty, it returns the value `true`; otherwise, it returns the value `false`.
- `isFullQueue`—Determines whether the queue is full. If the queue is full, it returns the value `true`; otherwise, it returns the value `false`.
- `front`—Returns the front, that is, the first element of the queue. Prior to this operation, the queue must exist and must not be empty.
- `back`—Returns the last element of the queue. Prior to this operation, the queue must exist and must not be empty.
- `addQueue`—Adds a new element to the rear of the queue. Prior to this operation, the queue must exist and must not be full.
- `deleteQueue`—Removes the front element from the queue. Prior to this operation, the queue must exist and must not be empty.

As in the case of a stack, a queue can be stored in an array or in a linked structure. We will consider both implementations. Because elements are added at one end and removed from the other end, we need two pointers to keep track of the front and rear of the queue, called `queueFront` and `queueRear`.

The following abstract class queueADT defines these operations as an ADT:

```
template <class Type>
class queueADT
{
public:
    virtual bool isEmptyQueue() const = 0;
    //Function to determine whether the queue is empty.
    //Postcondition: Returns true if the queue is empty,
    // otherwise returns false.
    virtual bool isFullQueue() const = 0;
    //Function to determine whether the queue is full.
    //Postcondition: Returns true if the queue is full,
    // otherwise returns false.
    virtual void initializeQueue() = 0;
    //Function to initialize the queue to an empty state.
    //Postcondition: The queue is empty.
    virtual Type front() const = 0;
    //Function to return the first element of the queue.
    //Precondition: The queue exists and is not empty.
    //Postcondition: If the queue is empty, the program
    // terminates; otherwise, the first element of the queue is returned.
    virtual Type back() const = 0;
    //Function to return the last element of the queue.
    //Precondition: The queue exists and is not empty.
    //Postcondition: If the queue is empty, the program
    // terminates; otherwise, the last element of the queue
    // is returned.
    virtual void addQueue(const Type& queueElement) = 0;
```

```

//Function to add queueElement to the queue.
//Precondition: The queue exists and is not full.
//Postcondition: The queue is changed and queueElement is
// added to the queue.
virtual void deleteQueue() = 0;
//Function to remove the first element of the queue.
//Precondition: The queue exists and is not empty.
//Postcondition: The queue is changed and the first element
// is removed from the queue.
};

```

14.4.2 Implementation of Queues as Arrays

Before giving the definition of the class to implement a queue as an ADT, we need to decide how many member variables are needed to implement the queue. Of course, we need an array to store the queue elements, the variables `queueFront` and `queueRear` to keep track of the first and last elements of the queue, and the variable `maxQueueSize` to specify the maximum size of the queue. Thus, we need at least four member variables.

Before writing the algorithms to implement the queue operations, we need to decide how to use `queueFront` and `queueRear` to access the queue elements. How do `queueFront` and `queueRear` indicate that the queue is empty or full? Suppose that `queueFront` gives the index of the first element of the queue, and `queueRear` gives the index of the last element of the queue. To add an element to the queue, first we advance `queueRear` to the next array position and then add the element to the position that `queueRear` is pointing to.

To delete an element from the queue, first we retrieve the element that `queueFront` is pointing to and then advance `queueFront` to the next element of the queue. Thus, `queueFront` changes after each `deleteQueue` operation and `queueRear` changes after each `addQueue` operation.

Let us see what happens when `queueFront` changes after a `deleteQueue` operation and `queueRear` changes after an `addQueue` operation. Assume that the array to hold the queue elements is of size 100.

Initially, the queue is empty. After the operation: the array is as shown in Figure 14.4.

`addQueue(Queue,'A');`

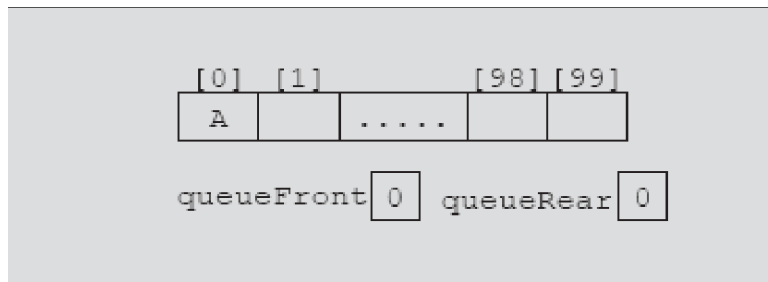


Figure 14.4 Queue after the first addqueue operation

After two more addQueue operations:

`addQueue(Queue,'B');`

`addQueue(Queue,'C');` the array is as shown in Figure 14.5.

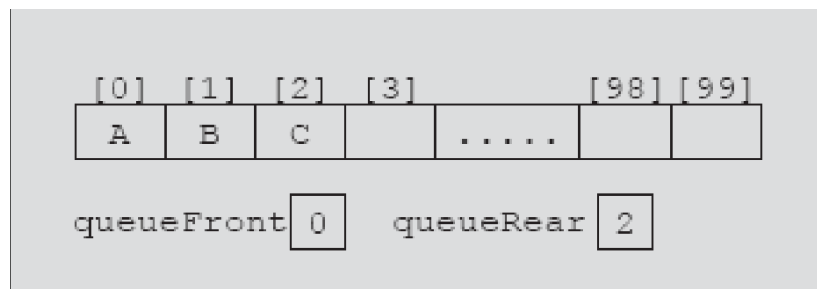


Figure 14.5 Queue after two more addQueue operations

Now consider the deleteQueue operation:

`deleteQueue();`

After this operation, the array containing the queue is as shown in Figure 14.6.

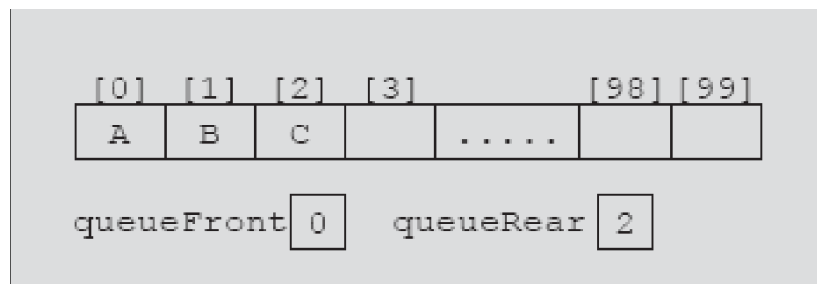


Figure 14.6 Queue after the delete Queue operation

Will this queue design work? Suppose A stands for adding (that is, `addQueue`) an element to the queue, and D stands for deleting (that is, `deleteQueue`) an element from the queue. Consider the following sequence of operations:

AAADADADADADADADA...

This sequence of operations would eventually set the index `queueRear` to point to the last array position, giving the impression that the queue is full. However, the queue has only two or three elements and the front of the array is empty.

One solution to this problem is that when the queue overflows to the rear (that is, `queueRear` points to the last array position), we can check the value of the index `queueFront`. If the value of `queueFront` indicates that there is room in the front of the array, then when `queueRear` gets to the last array position, we can slide all of the queue elements toward the first array position. This solution is good if the queue size is very small; otherwise, the program might execute more slowly.

Another solution to this problem is to assume that the array is circular—that is, the first array position immediately follows the last array position. (See Figure 14-7.)

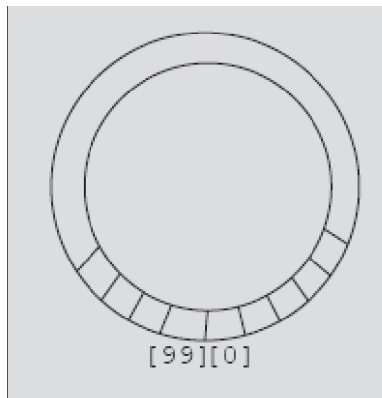


Figure 14.7 Circular Queue

We will consider the array containing the queue to be circular, although we will draw the figures of the array holding the queue elements as before. Suppose that we have the queue as shown in Figure 14-8(a).

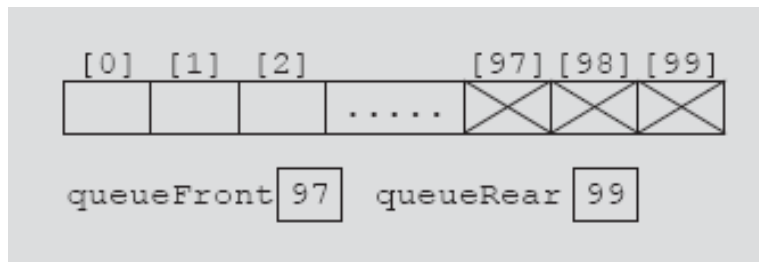


Figure 14.8 Queue before and after the add operation

After the operation `addQueue(Queue, 'Z');`, the queue is as shown in Figure 14-6(b).

Because the array containing the queue is circular, we can use the following statement to advance `queueRear` (`queueFront`) to the next array position:

```
queueRear = (queueRear + 1) % maxQueueSize;
```

If $\text{queueRear} < \text{maxQueueSize} - 1$, then $\text{queueRear} + 1 \leq \text{maxQueueSize} - 1$, so

$(\text{queueRear} + 1) \% \text{maxQueueSize} = \text{queueRear} + 1$. If $\text{queueRear} == \text{maxQueueSize} - 1$ (that is, `queueRear` points to the last array position), $\text{queueRear} + 1 == \text{maxQueueSize}$, so $(\text{queueRear} + 1) \% \text{maxQueueSize} = 0$. In this case, `queueRear` will be set to 0, which is the first array position. This queue design seems to work well. Before we write the algorithms to implement the queue operations, consider the following two cases.

Case 1: Suppose that after certain operations, the array containing the queue is as shown in Figure 14-9(a). After the operation `deleteQueue();`, the resulting array is as shown in Figure 14-9(b).

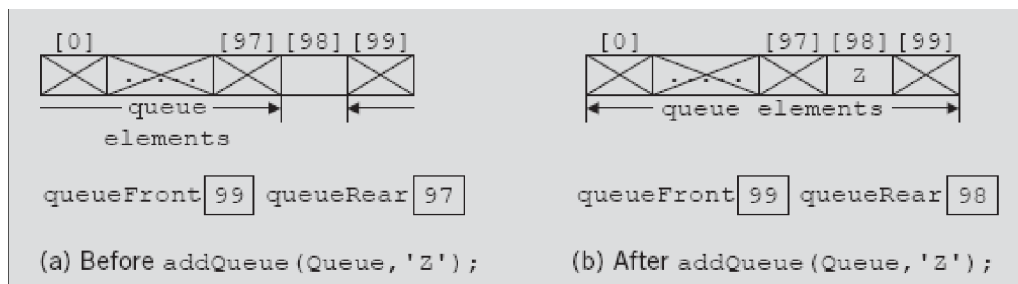


Figure 14.9 Queue before and after the delete

Case 2: Let us now consider the queue shown in Figure 14-10(a).

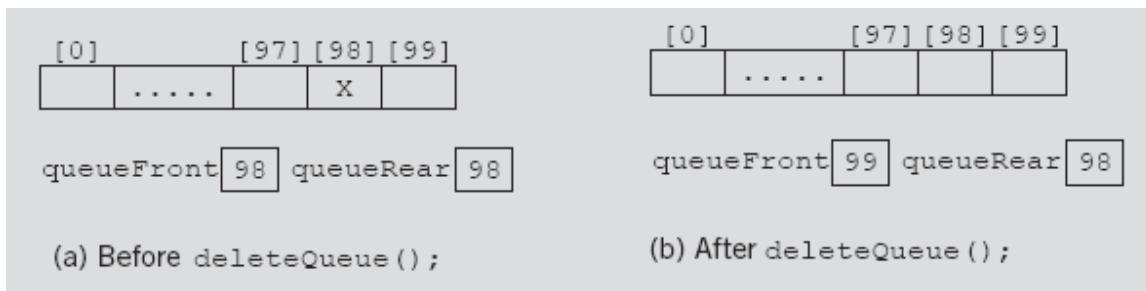


Figure 14.10 Queue before and after the add operation

After the operation `addQueue(Queue, 'Z');`, the resulting array is as shown in Figure 14.10(b). The arrays in Figures 14.10(a) and 14.10(b) have identical values for `queueFront` and `queueRear`. However, the resulting array in Figure 14.10(a) represents an empty queue, whereas the resulting array in Figure 14.10(b) represents a full queue. This latest queue design has brought up another problem of distinguishing between an empty and a full queue.

This problem has several solutions. One solution is to keep a count. In addition to the member variables `queueFront` and `queueRear`, we need another variable, `count`, to implement the queue. The value of `count` is incremented whenever a new element is added to the queue, and it is decremented whenever an element is removed from the queue. In this case, the function `initializeQueue` initializes `count` to 0. This solution is very useful if the user of the queue frequently needs to know the number of elements in the queue.

Another solution is to let `queueFront` indicate the index of the array position preceding the first element of the queue, rather than the index of the (actual) first element itself. In this case, assuming `queueRear` still indicates the index of the last element in the queue, the queue is empty if `queueFront == queueRear`. In this solution, the slot indicated by the index `queueFront` (that is, the slot preceding the first true element) is reserved. The queue will be full if the next available space is the special reserved slot indicated by `queueFront`.

Finally, because the array position indicated by `queueFront` is to be kept empty, if the array size is, say, 100, then 99 elements can be stored in the queue. (See Figure 14.11.)

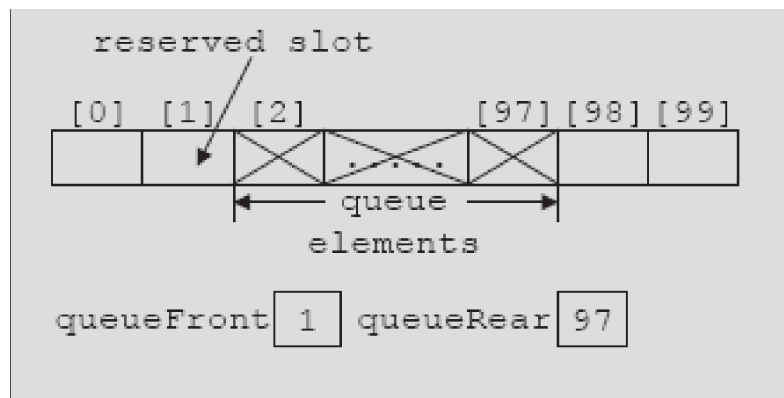


Figure 14.11 Array to store the queue elements with a reserved slot

Let us implement the queue using the first solution. That is, we use the variable count to indicate whether the queue is empty or full.

The following class implements the functions of the abstract class queueADT. Because arrays can be allocated dynamically, we will leave it for the user to specify the size of the array to implement the queue. The default size of the array is 100.

```
template <class Type>
class queueType: public queueADT<Type>
{
public:
    const queueType<Type>& operator=(const queueType<Type>&);
    //Overload the assignment operator.
    bool isEmptyQueue() const;
    //Function to determine whether the queue is empty.
    //Postcondition: Returns true if the queue is empty,
    // otherwise returns false.
    bool isFullQueue() const;
    //Function to determine whether the queue is full.
    //Postcondition: Returns true if the queue is full,
    // otherwise returns false.
```

```

void initializeQueue();
//Function to initialize the queue to an empty state.
//Postcondition: The queue is empty.
Type front() const;
//Function to return the first element of the queue.
//Precondition: The queue exists and is not empty.
//Postcondition: If the queue is empty, the program
// terminates; otherwise, the first element of the
// queue is returned.
Type back() const;
//Function to return the last element of the queue.
//Precondition: The queue exists and is not empty.
//Postcondition: If the queue is empty, the program
// terminates; otherwise, the last element of the queue
// is returned.
void addQueue(const Type& queueElement);
//Function to add queueElement to the queue.
//Precondition: The queue exists and is not full.
//Postcondition: The queue is changed and queueElement is
// added to the queue.
void deleteQueue();
//Function to remove the first element of the queue.
//Precondition: The queue exists and is not empty.
//Postcondition: The queue is changed and the first element
// is removed from the queue.
queueType(int queueSize = 100);
//Constructor
queueType(const queueType<Type>& otherQueue);

```

```

//Copy constructor
~queueType();
//Destructor
private:
int maxQueueSize; //variable to store the maximum queue size
int count; //variable to store the number of
//elements in the queue
int queueFront; //variable to point to the first
//element of the queue
int queueRear; //variable to point to the last
//element of the queue
Type *list; //pointer to the array that holds
//the queue elements
};

```

Empty Queue and Full Queue

As discussed earlier, the queue is empty if `count == 0`, and the queue is full if `count == maxQueueSize`. So the functions to implement these operations are as follows:

```

template <class Type>
bool queueType<Type>::isEmptyQueue() const
{
return (count == 0);
} //end isEmptyQueue
template <class Type>
bool queueType<Type>::isFullQueue() const
{
return (count == maxQueueSize);
} //end isFullQueue

```

Initialize Queue

This operation initializes a queue to an empty state. The first element is added at the first array position. Therefore, we initialize `queueFront` to 0, `queueRear` to `maxQueueSize - 1`, and `count` to 0.

The definition of the function `initializeQueue` is as follows:

```
template <class Type>
void queueType<Type>::initializeQueue()
{
    queueFront = 0;
    queueRear = maxQueueSize - 1;
    count = 0;
} //end initializeQueue
```

Front

This operation returns the first element of the queue. If the queue is nonempty, the element of the queue indicated by the index `queueFront` is returned; otherwise, the program terminates.

```
template <class Type>
Type queueType<Type>::front() const
{
    assert(!isEmptyQueue());
    return list[queueFront];
} //end front
```

Back

This operation returns the last element of the queue. If the queue is nonempty, the element of the queue indicated by the index `queueRear` is returned; otherwise, the program terminates.

```
template <class Type>
Type queueType<Type>::back() const
{
```

```

assert(!isEmptyQueue());
return list[queueRear];
} //end back

```

Add Queue

Next, we implement the addQueue operation. Because queueRear points to the last element of the queue, to add a new element to the queue, we first advance queueRear to the next array position, and then add the new element to the array position indicated by queueRear. We also increment count by 1. So the function addQueue is as follows:

```

template <class Type>
void queueType<Type>::addQueue(const Type& newElement)
{
    if (!isFullQueue())
    {
        queueRear = (queueRear + 1) % maxQueueSize; //use the
        //mod operator to advance queueRear
        //because the array is circular
        count++;
        list[queueRear] = newElement;
    }
    else
        cout << "Cannot add to a full queue." << endl;
} //end addQueue

```

Delete Queue

To implement the deleteQueue operation, we access the index queueFront. Because queueFront points to the array position containing the first element of the queue, to remove the first queue element, we decrement count by 1 and advance queueFront to the next queue element. So the function deleteQueue is as follows:

```

template <class Type>

```

```

void queueType<Type>::deleteQueue()
{
    if (!isEmptyQueue())
    {
        count- -;
        queueFront = (queueFront + 1) % maxQueueSize; //use the
        //mod operator to advance queueFront
        //because the array is circular
    }
    else
        cout << "Cannot remove from an empty queue" << endl;
} //end deleteQueue

```

Application of Queues: Simulation

A technique in which one system models the behavior of another system is called simulation. For example, physical simulators include wind tunnels used to experiment with the design of car bodies and flight simulators are used to train airline pilots. Simulation techniques are used when it is too expensive or dangerous to experiment with real systems.

You can also design computer models to study the behavior of real systems. (We will describe some real systems modeled by computers shortly.) Simulating the behavior of an expensive or dangerous experiment using a computer model is usually less expensive than using the real system, and a good way to gain insight without putting human life in danger. Moreover, computer simulations are particularly useful for complex systems where it is difficult to construct a mathematical model. For such systems, computer models can retain descriptive accuracy. In mathematical simulations, the steps of a program are used to model the behavior of a real system. Let us consider one such problem.

The manager of a local movie theater is hearing complaints from customers about the time they have to wait in line to buy tickets. The theater currently has only one cashier. Another theater is preparing to open in the neighborhood and the manager is afraid of losing customers. The manager wants to hire enough cashiers so that a customer does not have to wait too long to buy a ticket, but does not want to hire extra cashiers on a trial basis and potentially waste

time and money. One thing that the manager would like to know is the average time a customer has to wait for service. The manager wants someone to write a program to simulate the behavior of the theater.

In computer simulation, the objects being studied are usually represented as data. For the theater problem, some of the objects are the customers and the cashier. The cashier serves the customers and we want to determine a customer's average waiting time. Actions are implemented by writing algorithms, which in a programming language are implemented with the help of functions. Thus, functions are used to implement the actions of the objects. In C++, we can combine the data and the operations on that data into a single unit with the help of classes. Thus, objects can be represented as classes. The member variables of the class describe the properties of the objects, and the function members describe the actions on that data. This change in simulation results can also occur if we change the values of the data or modify the definitions of the functions (that is, modify the algorithms implementing the actions). The main goal of a computer simulation is to either generate results showing the performance of an existing system or predict the performance of a proposed system.

In the theater problem, when the cashier is serving a customer, the other customers must wait. Because customers are served on a first-come, first-served basis and queues are an effective way to implement a First In First Out system, queues are important data structures for use in computer simulations. This section examines computer simulations in which queues are the basic data structure. These simulations model the behavior of systems, called queuing systems, in which queues of objects are waiting to be served by various servers. In other words, a queuing system consists of servers and queues of objects waiting to be served. We deal with a variety of queuing systems on a daily basis. For example, a grocery store and a banking system are both queuing systems. Furthermore, when you send a print request to a networked printer that is shared by many people, your print request goes in a queue. Print requests that arrived before your print requests are usually completed before yours. Thus, the printer acts as the server when a queue of documents is waiting to be printed.

14.5 Evaluation of Expression

The usual notation for writing arithmetic expressions (the notation we learned in elementary school) is called infix notation, in which the operator is written between the operands. For example, in the expression $a + b$, the operator $+$ is between the operands a and b . In infix notation, the operators have precedence. That is, we must evaluate expressions from left to right, and

multiplication and division have higher precedence than addition and subtraction. If we want to evaluate the expression in a different order, we must include parentheses. For example, in the expression $a + b * c$, we first evaluate $*$ using the operands b and c , and then we evaluate $+$ using the operand a and the result of $b * c$.

In the early 1920s, the Polish mathematician Jan Lukasiewicz discovered that if operators were written before the operands (prefix or Polish notation; for example, $+ a b$), the parentheses can be omitted. In the late 1950s, the Australian philosopher and early computer scientist Charles L. Hamblin proposed a scheme in which the operators follow the operands (postfix operators), resulting in the Reverse Polish notation. This has the advantage that the operators appear in the order required for computation.

For example, the expression:

$a + b * c$

in a postfix expression is:

$a b c * +$

The following example shows various infix expressions and their equivalent postfix expressions.

<u>Infix expression</u>	<u>Equivalent postfix expression</u>
$a + b$	$ab +$
$a + b * c$	$ab c * +$
$a * b + c$	$ab * c +$
$(a + b) * c$	$ab + c *$
$(a - b) * (c + d)$	$ab - cd + *$
$(a + b) * (c - d / e) + f$	$ab + cde / - * f +$

Shortly after Lukasiewicz's discovery, it was realized that postfix notation had important applications in computer science. In fact, many compilers use stacks to first translate infix expressions into some form of postfix notation and then translate this postfix expression into machine code. Postfix expressions can be evaluated using the following algorithm:

Scan the expression from left to right. When an operator is found, back up to get the required number of operands, perform the operation, and continue.

Consider the following postfix expression:

$6\ 3\ +\ 2\ * =$

Let us evaluate this expression using a stack and the previous algorithm. Figure 14.12 shows how this expression gets evaluated.

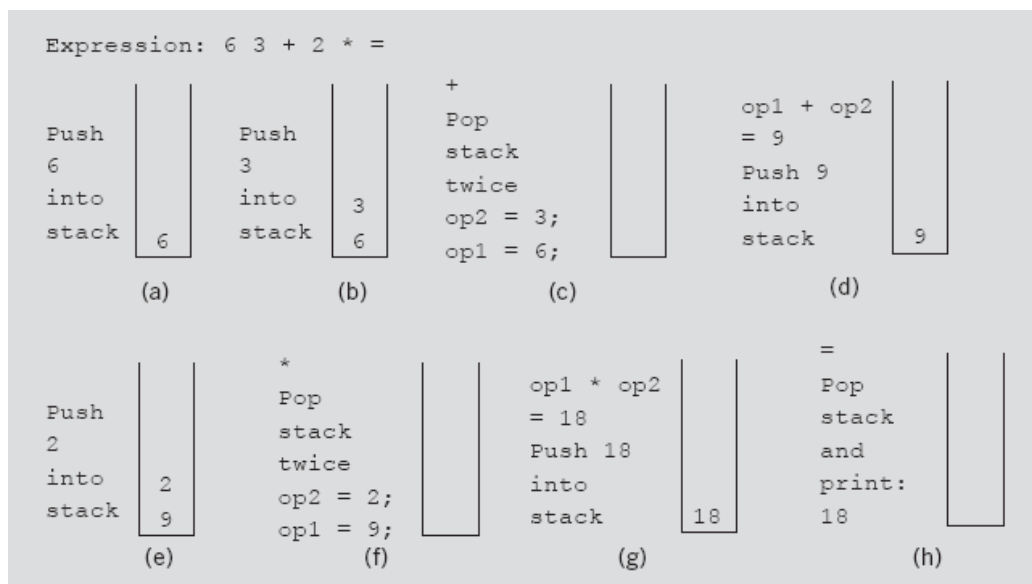


Figure 14.12 Evaluating the postfix expression

Read the first symbol, 6, which is a number. Push the number onto the stack; see Figure 14.12(a). Read the next symbol, 3, which is a number. Push the number onto the stack; see Figure 14.12(b). Read the next symbol, +, which is an operator. Because an operator requires two operands to be evaluated, pop the stack twice; see Figure 14.12(c). Perform the operation and put the result back onto the stack; see Figure 14.12(d). Read the next symbol, 2, which is a number. Push the number onto the stack; see Figure 14.12(e). Read the next symbol, *, which is an operator. Because an operator requires two operands to be evaluated, pop the stack twice; see Figure 14.12(f). Perform the operation and put the result back onto the stack; see Figure 14.12(g).

Scan the next symbol, =, which is the equal sign, indicating the end of the expression.

Therefore, print the result. The result of the expression is in the stack, so pop and print; see Figure 14.12(h).

The value of the expression $6 \ 3 + 2 \ * = 18$.

From this discussion, it is clear that when we read a symbol other than a number, the following cases arise:

1. The symbol we read is one of the following: +, -, *, /, or =.
 - a. If the symbol is +, -, *, or /, the symbol is an operator and so we must evaluate it. Because an operator requires two operands, the stack must have at least two elements; otherwise, the expression has an error.
 - b. If the symbol is = (an equal sign), the expression ends and we must print the answer. At this step, the stack must contain exactly one element; otherwise, the expression has an error.
2. The symbol we read is something other than +, -, *, /, or =. In this case, the expression contains an illegal operator.

It is also clear that when an operand (number) is encountered in an expression, it is pushed onto the stack because the operator comes after the operands.

Consider the following expressions:

i. $7 \ 6 + 3 ; 6 - =$

ii. $14 + 2 \ 3 \ * =$

iii. $14 \ 2 \ 3 + =$

Expression (i) has an illegal operator, expression (ii) does not have enough operands for +, and expression (iii) has too many operands. In the case of expression (iii), when we encounter the equal sign (=), the stack will have two elements and this error cannot be discovered until we are ready to print the value of the expression. To make the input easier to read, we assume that the postfix expressions are in the following form:

#6 #3 + #2 * =

The symbol # precedes each number in the expression. If the symbol scanned is #, the next input is a number (that is, an operand). If the symbol scanned is not #, it is either an operator (might be illegal) or an equal sign (indicating the end of the expression). Furthermore, we assume that each expression contains only the +, -, *, and / operators.

This program outputs the entire postfix expression together with the answer. If the expression has an error, the expression is discarded. In this case, the program outputs the expression together with an appropriate error message. Because an expression might contain an error, we must clear the stack before processing the next expression. Also, the stack must be initialized; that is, the stack must be empty.

MAIN ALGORITHM

Pursuant to the previous discussion, the main algorithm in pseudo code is as follows:

Read the first character

while not the end of input data

{

a. initialize the stack

b. process the expression

c. output result

d. get the next expression

}

To simplify the complexity of the function main, we write four functions—evaluateExpression, evaluateOpr, discardExp, and printResult. The function evaluateExpression, if possible, evaluates the expression and leaves the result in the stack. If the postfix expression is error free, the function printResult outputs the result. The function evaluateOpr evaluates an operator, and the function discardExp discards the current expression if there is any error in the expression.

FUNCTION evaluateExpression The function evaluateExpression evaluates each postfix expression. Each expression ends with the symbol =. The general algorithm in pseudocode is as follows:

```

while (ch is not = '=') //process each expression
//= marks the end of an expression
{
  switch (ch)
  {
    case '#':
      read a number
      output the number;
      push the number onto the stack;
      break;
    default:
      assume that ch is an operation
      evaluate the operation;
  } //end switch
  if no error was found, then
  {
    read next ch;
    output ch;
  }
  else
    Discard the expression
} //end while

```

From this algorithm, it follows that this method has five parameters—a parameter to access the input file, a parameter to access the output file, a parameter to access the stack, a parameter to pass a character of the expression, and a parameter to indicate whether there is an error in the expression. The definition of this function is as follows:

```

void evaluateExpression(ifstream& inpF, ofstream& outF,
stackType<double>& stack,
char& ch, bool& isExpOk)

{
double num;
outF << ch;
while (ch != '=')
{
switch (ch)
{
case '#':
inpF >> num;
outF << num << " ";
if (!stack.isFullStack())
stack.push(num);
else
{
cout << "Stack overflow. "
<< "Program terminates!" << endl;
exit(0); //terminate the program
}
break;
default:
evaluateOpr(outF, stack, ch, isExpOk);
} //end switch
if (isExpOk) //if no error
{

```

```

inpF >> ch;
outF << ch;
if (ch != '#')
outF << " ";
}
else
discardExp(inpF, outF, ch);
} //end while (!= '=')
} //end evaluateExpression

```

Note that the funtion exit terminates the program.

FUNCTION evaluateOpr

This function (if possible) evaluates an expression. Two operands are needed to evaluate an operation and operands are saved in the stack. Therefore, the stack must contain at least two numbers. If the stack contains fewer than two numbers, the expression has an error. In this case, the entire expression is discarded and an appropriate message is printed. This function also checks for any illegal operations. In pseudocode, this function is as follows:

```

if stack is empty
{
error in the expression
set expressionOk to false
}
else
{
retrieve the top element of stack into op2
pop stack
if stack is empty
{
error in the expression

```

```
    set expressionOk to false
  }
  else
  {
    retrieve the top element of stack into op1
    pop stack
    //If the operation is legal, perform the operation and
    //push the result onto the stack;
    //otherwise, report the error.
    switch (ch)
    {
      case '+': //add the operands: op1 + op2
        stack.push(op1 + op2);
        break;
      case '-': //subtract the operands: op1 - op2
        stack.push(op1 - op2);
        break;
      case '*': //multiply the operands: op1 * op2
        stack.push(op1 * op2);
        break;
      case '/': //If (op2 != 0), op1 / op2
        stack.push(op1 / op2);
        break;
      otherwise operation is illegal
      {
        output an appropriate message;
        set expressionOk to false
      }
    }
  }
```



```

} //end switch
}

```

Following this pseudocode, the definition of the function evaluateOpr is as follows:

```

void evaluateOpr(ofstream& out, stackType<double>& stack,
char& ch, bool& isExpOk)
{
double op1, op2;
if (stack.isEmptyStack())
{
out << " (Not enough operands)";
isExpOk = false;
}
else
{
op2 = stack.top();
stack.pop();
if (stack.isEmptyStack())
{
out << " (Not enough operands)";
isExpOk = false;
}
else
{
op1 = stack.top();
stack.pop();
switch (ch)
{
case '+':

```

```

    stack.push(op1 + op2);
    break;
    case '-':
    stack.push(op1 - op2);
    break;
    case '*':
    stack.push(op1 * op2);
    break;
    case '/':
    if (op2 != 0)
    stack.push(op1 / op2);
    else
    {
    out << " (Division by 0)";
    isExpOk = false;
    }
    break;
    default:
    out << " (Illegal operator)";
    isExpOk = false;
} //end switch
} //end else
} //end else
} //end evaluateOpr
FUNCTION discardExp

```

This function is called whenever an error is discovered in the expression. It reads and writes the input data only until the input is '=', the end of the expression. The definition of this function is as follows:

```
void discardExp(ifstream& in, ofstream& out, char& ch)
{
    while (ch != '=')
    {
        in.get(ch);
        out << ch;
    }
} //end discardExp
```

14.6 Summary

A stack is a list of homogenous elements in which the addition and deletion of elements occurs only at one end, called the top of the stack. A data structure in which the elements are added at one end, called the rear, and deleted from the other end, called the front; a First In First Out (FIFO) data structure.

14.7 Review Questions

1. Define linear data structure.
2. What is Stack?
3. Explain the operations on Stack.
4. Explain the Operations on Queue.
5. Explain about Infix to postfix conversion.
6. Explain the Application of Stack.
7. Explain the applications on Queue.

LESSON 15

TREES

Structure

15.1 Introduction

15.2 Learning Objectives

15.3 Binary Tree

15.4 Binary Tree Traversal

15.5 Binary search Tree

15.6 Threaded Binary Trees

15.7 Summary

15.8 Review Questions

15.1 Introduction

A tree is a widely used abstract data type (ADT) or data structure implementing this ADT that simulates a hierarchical tree structure, with a root value and subtrees of children, represented as a set of linked nodes.

A tree data structure can be defined recursively (locally) as a collection of nodes (starting at a root node), where each node is a data structure consisting of a value, together with a list of references to nodes (the “children”), with the constraints that no reference is duplicated, and none points to the root.

Alternatively, a tree can be defined abstractly as a whole (globally) as an ordered tree, with a value assigned to each node. Both these perspectives are useful: while a tree can be analyzed mathematically as a whole, when actually represented as a data structure it is usually represented and worked with separately by node (rather than as a list of nodes and an adjacency list of edges between nodes, as one may represent a digraph, for instance). For example, looking at a tree as a whole, one can talk about “the parent node” of a given node, but in general

as a data structure a given node only contains the list of its children, but does not contain a reference to its parent

15.2 Learning Objectives

At the end of this lesson you can able to

- Tree
- Binary tree
- Binary tree traversal
- Binary search tree

15.3 Binary tree

This lesson introduces one of the most fundamental structures in computer science: binary trees. The use of the word tree here comes from the fact that, when we draw them, the resultant drawing often resembles the trees found in a forest. There are many ways of ways of defining binary trees. Mathematically, a binary tree is a connected, undirected, finite graph with no cycles, and no vertex of degree greater than three.

Definition: A binary tree is a finite set of nodes which is either empty or consists of a root and two disjoint binary trees called the left subtree and the right subtree.

For most computer science applications, binary trees are rooted: A special node, r , of degree at most two is called the root of the tree. For every node, u , r , the second node on the path from u to r is called the parent of u . Each of the other nodes adjacent to u is called a child of u . Most of the binary trees we are interested in are ordered, so we distinguish between the left child and right child of u .

In illustrations, binary trees are usually drawn from the root down-ward, with the root at the top of the drawing and the left and right children respectively given by left and right positions in the drawing (Figure 15.1).

Because binary trees are so important, a certain terminology has developed for them: The depth of a node, u , in a binary tree is the length of the path from u to the root of the tree. If

a node, w , is on the path from u to r , then w is called an ancestor of u and u a descendant of w . The subtree of a node, u , is the binary tree that is rooted at u and contains all of u 's descendants. The height of a node, u , is the length of the longest path from u to one of its descendants. The height of a tree is the height of its root. A node, u , is a leaf if it has no children.

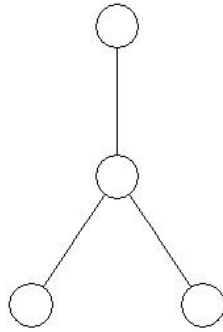


Figure 15.1: The parent, left child, and right child of the node u in a BinaryTree.

The simplest way to represent a node, u , in a binary tree is to explicitly store the (at most three) neighbours of u :

```

BinaryTree
class BTreeNode {
    N    *left;
    N    *right;
    N    *parent;
    BTreeNode() {
        left = right = parent = NULL;
    }
};
  
```

When one of these three neighbours is not present, we set it to nil. In this way, both external nodes of the tree and the parent of the root correspond to the value nil.

The binary tree itself can then be represented by a pointer to its root

node, r :

```

BinaryTree
  
```

```
Node r; // root node
```

We can compute the depth of a node, u , in a binary tree by counting the number of steps on the path from u to the root:

```
int depth(Node *u)
{ int d = 0;
  while (u != r)
  {
    u = u->parent; d++;
  }
  return d;
}
```

15.3.1 Recursive Algorithms

Using recursive algorithms makes it very easy to compute facts about binary trees. For example, to compute the size of (number of nodes in) a binary tree rooted at node u , we recursively compute the sizes of the two subtrees rooted at the children of u , sum up these sizes, and add one:

```
BinaryTree
int size(Node .u) {
  if (u == nil) return 0;
  return 1 + size(u->left) + size(u->right);
}
```

To compute the height of a node u , we can compute the height of u 's two subtrees, take the maximum, and add one:

```
BinaryTree
int height(Node .u) {
  if (u == nil) return -1;
  return 1 + max (height(u->left), height(u->right));
}
```

15.4 Binary Tree Traversal

The item insertion, deletion, and lookup operations require that the binary tree be traversed. Thus, the most common operation performed on a binary tree is to traverse the binary tree, or visit each node of the binary tree. As you can see from the diagram of a binary tree, the traversal must start at the root node because there is a pointer to the root node. For each node, we have two choices:

- Visit the node first.
- Visit the subtrees first.

These choices lead to three different traversals of a binary tree—Inorder, preorder, and postorder.

Inorder Traversal

In an inorder traversal, the binary tree is traversed as follows:

1. Traverse the left subtree.
2. Visit the node.
3. Traverse the right subtree.

Preorder Traversal

In a preorder traversal, the binary tree is traversed as follows:

1. Visit the node.
2. Traverse the left subtree.
3. Traverse the right subtree.

Postorder Traversal

In a postorder traversal, the binary tree is traversed as follows:

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the node.

Clearly, each of these traversal algorithms is recursive.

The listing of the nodes produced by the inorder traversal of a binary tree is called the inorder sequence. The listing of the nodes produced by the preorder traversal of a binary tree is called the preorder sequence. The listing of the nodes produced by the postorder traversal of a binary tree is called the postorder sequence. Before giving the C++ code for each of these traversals, let us illustrate the inorder traversal of the binary tree in Figure 15.2. For simplicity, we assume that visiting a node means to output the data stored in the node. The section, “Binary Tree Traversal and Functions as Parameters,” located later in this chapter, explains how to modify the binary tree traversal algorithms so that by using a function, the user can specify the action to be performed on a node when the node is visited. A pointer to the binary tree in Figure 15.2 is stored in the pointer variable root (which points to the node with info A).

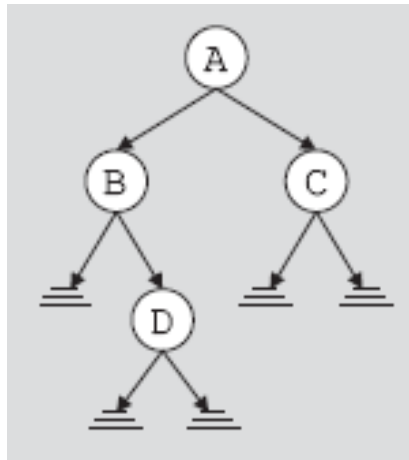


Figure 15.2 Binary Tree for an inorder Traversal

Therefore, we start the traversal at A.

1. Traverse the left subtree of A; that is, traverse $LA = \{B, D\}$.
2. Visit A.
3. Traverse the right subtree of A; that is, traverse $RA = \{C\}$.

Now, we cannot do Step 2 until we have finished Step 1.

1. Traverse the left subtree of A; that is, traverse $LA = \{B, D\}$. Now LA is a binary tree with the root node B. Because LA is a binary tree, we apply the inorder traversal criteria to LA.

1.1. Traverse the left subtree of B; that is, traverse $LB = \text{empty}$.

1.2. Visit B.

1.3. Traverse the right subtree of B; that is, traverse $RB = \{D\}$.

As before, first we complete Step 1.1 before going to Step 1.2.

1.1. Because the left subtree of B is empty, there is nothing to traverse.

Step 1.1 is completed, so we proceed to Step 1.2.

1.2. Visit B. That is, output B on an output device. Clearly, the first node printed is B. This completes Step 1.2, so we proceed to Step 1.3.

1.3. Traverse the right subtree of B; that is, traverse $RB = \{D\}$. Now RB is a binary tree with the root node D. Because RB is a binary tree, we apply the inorder traversal criteria to RB.

1.3.1. Traverse the left subtree of D; that is, traverse $LD = \text{empty}$.

1.3.2. Visit D.

1.3.3. Traverse the right subtree of D; that is, traverse $RD = \text{empty}$.

1.3.1. Because the left subtree of D is empty, there is nothing to traverse. Step

1.3.1 is completed, so we proceed to Step 1.3.2.

1.3.2. Visit D. That is, output D on an output device. This completes Step 1.3.2,

So we proceed to Step 1.3.3.

1.3.3. Because the right subtree of D is empty, there is nothing to traverse. Step

1.3.3 is completed.

This completes Step 1.3. Because Steps 1.1, 1.2, and 1.3 are completed, Step 1 is completed, and so we go to Step 2.

2. Visit A. That is, output A on an output device. This completes Step 2, so we proceed to Step 3.

3. Traverse the right subtree of A; that is, traverse $RA = \{C\}$. Now RA is a binary tree with the root node C. Because RA is a binary tree, we apply the inorder traversal criteria to RA.

- 3.1. Traverse the left subtree of C; that is, traverse LC= empty.
- 3.2. Visit C.
- 3.3. Traverse the right subtree of C; that is, traverse RC= empty.
- 3.1. Because the left subtree of C is empty, there is nothing to traverse.

Step 3.1 is completed.

- 3.2. Visit C. That is, output C on an output device. This completes Step 3.2, so we proceed to Step 3.3.
- 3.3. Because the right subtree of C is empty, there is nothing to traverse. Step 3.3 is completed.

This completes Step 3, which in turn completes the traversal of the binary tree.

Clearly, the inorder traversal of the previous binary tree outputs the nodes in the following order:

Inorder sequence: B D A C

Similarly, the preorder and postorder traversals output the nodes in the following order:

Preorder sequence: A B D C

Postorder sequence: D B C A

15.5 Binary Search Trees

Now that you know the basic operations on a binary tree, this section discusses a special type of binary tree, called a binary search tree.

Consider the binary tree in Figure 15-3.

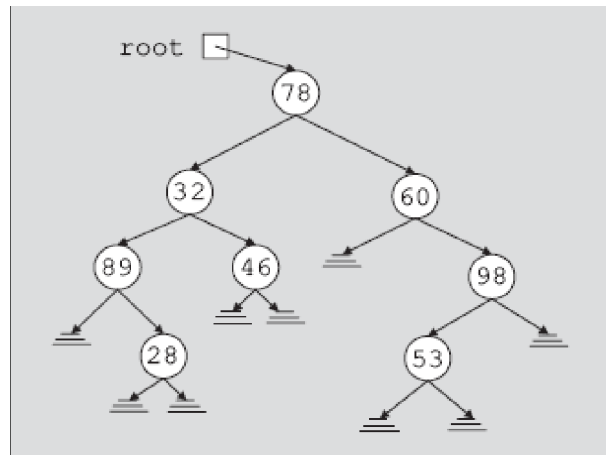


Figure 15.3 Arbitrary Binary Tree

Suppose that we want to determine whether 50 is in the binary tree. To do so, we can use any of the previous traversal algorithms to visit each node and compare the search item with the data stored in the node. However, this could require us to traverse a large part of the binary tree, so the search would be slow. We need to visit each node in the binary tree until either the item is found or we have traversed the entire binary tree because no criteria exist to guide our search. This case is like an arbitrary linked list where we must start our search at the first node, and continue looking at each node until either the item is found or the entire list is searched.

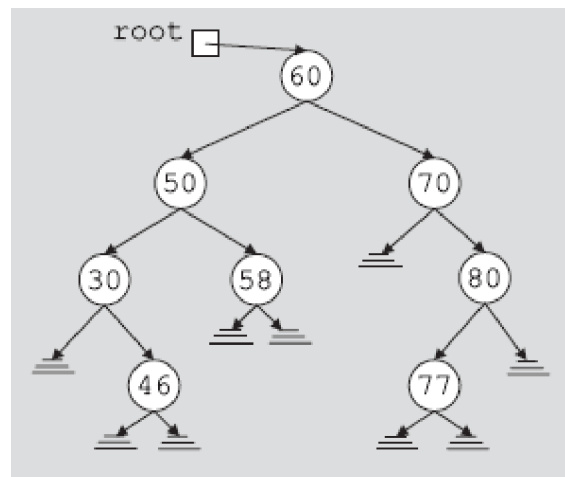


Figure 15.4 Binary Search Tree

On the other hand, consider the binary tree in Figure 15.4.

In the binary tree in Figure 15.4. the data in each node is

- Larger than the data in its left subtree
- Smaller than the data in its right subtree

The binary tree in Figure 15.4 has some structure. Suppose that we want to determine whether 58 are in this binary tree. As before, we must start our search at the root node. We compare 58 with the data in the root node; that is, we compare 58 with 60. Because $58 \leq 60$ and $58 < 60$, it is guaranteed that 58 will not be in the right subtree of the root node. Therefore, if 58 is in the binary tree, it must be in the left subtree of the root node. We follow the left pointer of the root node and go to the node with info 50. We now apply the same criteria at this node. Because $58 > 50$, we must follow the right pointer of this node and go to the node with info 58. At this node we find item 58. This example shows that every time we move down to a child, we eliminate one of the subtrees of the node from our search. If the binary tree is nicely constructed, the search is very similar to the binary search on arrays.

The binary tree given in Figure 15.4 is a special type of binary tree, called a binary search tree. (In the following definition, by the term key of the node we mean the key of the data item that uniquely identifies the item.)

Definition: A binary search tree, T , is either empty or the following is true:

1. T has a special node called the root node.
2. T has two sets of nodes, LT and RT , called the left subtree and right subtree of T , respectively.
3. The key in the root node is larger than every key in the left subtree and smaller than every key in the right subtree.
4. LT and RT are binary search trees.

The following operations are typically performed on a binary search tree:

- Search the binary search tree for a particular item.
- Insert an item in the binary search tree.

- Delete an item from the binary search tree.
- Find the height of the binary search tree.
- Find the number of nodes in the binary search tree.
- Find the number of leaves in the binary search tree.
- Traverse the binary search tree.
- Copy the binary search tree.

Clearly, every binary search tree is a binary tree. The height of a binary search tree is determined the same way as the height of a binary tree. Similarly, the operations to find the number of nodes, to find the number of leaves, and to do inorder, preorder, and postorder traversals of a binary search tree are the same as those for a binary tree. Therefore, we can inherit all of these operations from the binary tree. That is, we can extend the definition of the binary tree by using the principle of inheritance and, hence, define the binary search tree.

The following class defines a binary search tree as an ADT by extending the definition of the binary tree:

```
//*****
// binary search tree.
//*****

template <class elemType>
class bSearchTreeType: public binaryTreeType<elemType>
{
public:
    bool search(const elemType& searchItem) const;
    //Function to determine if searchItem is in the binary search tree.
    //Postcondition: Returns true if searchItem is found in the
    // binary search tree; otherwise, returns false.
    void insert(const elemType& insertItem);
    //Function to insert insertItem in the binary search tree.
```

```

//Postcondition: If no node in the binary search tree has the
// same info as insertItem, a node with the info insertItem
// is created and inserted in the binary search tree.
void deleteNode(const elemType& deleteltem);
//Function to delete deleteltem from the binary search tree.
//Postcondition: If a node with the same info as deleteltem
// is found, it is deleted from the binary search tree.
private:
void deleteFromTree(binaryTreeNode<elemType>* &p);
//Function to delete the node to which p points is deleted
//from the binary search tree.
//Postcondition: The node to which p points is deleted from the binary search tree.
};

```

Next, we describe each of these operations.

Search

The function search searches the binary search tree for a given item. If the item is found in the binary search tree, it returns true; otherwise, it returns false. Because the pointer root points to the root node of the binary search tree, we must begin our search at the root node. Furthermore, because root must always point to the root node, we need a pointer, say current, to traverse the binary search tree. The pointer current is initialized to root.

If the binary search tree is nonempty, we first compare the search item with the info in the root node. If they are the same, we stop the search and return true. Otherwise, if the search item is smaller than the info in the node, we follow llink to go to the left subtree; otherwise, we follow rlink to go to the right subtree. We repeat this process for the next node. If the search item is in the binary search tree, our search ends at the node containing the search item; otherwise, the search ends at an empty subtree. Thus, the general algorithm is as follows:

if root is NULL

Cannot search an empty tree, returns false.

```

else
{
current = root;
while (current is not NULL and not found)
if (current->info is the same as the search item)
set found to true;
else if(current->info is greater than the search item)
follow the llink of current
else
follow the rlink of current
}

```

This pseudocode algorithm translates into the following C++ function:

```

template <class elemType>
bool bSearchTreeType<elemType>::
search(const elemType& searchItem) const
{
binaryTreeNode<elemType> *current;
bool found = false;
if (root == NULL)
cerr << "Cannot search the empty tree." << endl;
else
{
current = root;
while (current != NULL && !found)
{
if (current->info == searchItem)
found = true;
else if (current->info > searchItem)

```



```

current = current->llink;
else
current = current->rlink;
} //end while
} //end else
return found;
} //end search

```

Insert

After inserting an item in a binary search tree, the resulting binary tree must also be a binary search tree. To insert a new item, first we search the binary search tree and find the place where the new item is to be inserted. The search algorithm is similar to the search algorithm of the function search. Here we traverse the binary search tree with two pointers—a pointer, say current, to check the current node and a pointer, say trailCurrent, pointing to the parent of current. Because duplicate items are not allowed, our search must end at an empty subtree. We can then use the pointer trailCurrent to insert the new item at the proper place. The item to be inserted, insertItem, is passed as a parameter to the function insert. The general algorithm is as follows:

- a. Create a new node and copy insertItem into the new node. Also set llink and rlink of the new node to NULL.
- b. if the root is NULL, the tree is empty so make root point to the new node.

```

else
{
current = root;
while (current is not NULL) //search the binary tree
{
trailCurrent = current;
if (current->info is the same as the insertItem)
Error: Cannot insert duplicate

```

```

exit
else if(current->info > insertItem)
    Follow llink of current
else
    Follow rlink of current
}
//insert the new node in the binary tree
if (trailCurrent->info > insertItem)
    make the new node the left child of trailCurrent
else
    make the new node the right child of trailCurrent
}

```

This pseudocode algorithm translates into the following C++ function:

```

template <class elemType>
void bSearchTreeType<elemType>::insert(const elemType& insertItem)
{
    binaryTreeNode<elemType> *current; //pointer to traverse the tree
    binaryTreeNode<elemType> *trailCurrent; //pointer behind current
    binaryTreeNode<elemType> *newNode; //pointer to create the node
    newNode = new binaryTreeNode<elemType>;
    assert(newNode != NULL);
    newNode->info = insertItem;
    newNode->llink = NULL;
    newNode->rlink = NULL;
    if (root == NULL)
        root = newNode;
    else
    {

```

```

current = root;
while (current != NULL)
{
    trailCurrent = current;
    if (current->info == insertItem)
    {
        cerr << "The insert item is already in the list-";
        cerr << "duplicates are not allowed."
        << insertItem << endl;
        return;
    }
    else if (current->info > insertItem)
        current = current->llink;
    else
        current = current->rlink;
} //end while
if (trailCurrent->info > insertItem)
    trailCurrent->llink = newNode;
else
    trailCurrent->rlink = newNode;
}
} //end insert

```

15.6 Threaded Binary Trees

“A binary tree is threaded by making all right child pointers that would normally be null point to the inorder successor of the node, and all left child pointers that would normally be null point to the inorder predecessor of the node.”

A threaded binary tree makes it possible to traverse the values in the binary tree via a linear traversal that is more rapid than a recursive in-order traversal. It is also possible to discover

the parent of a node from a threaded binary tree, without explicit use of parent pointers or a stack, albeit slowly. This can be useful where stack space is limited, or where a stack of parent pointers is unavailable (for finding the parent pointer via DFS).

To see how this is possible, consider a node k that has a right child r . Then the left pointer of r must be either a child or a thread back to k . In the case that r has a left child, that left child must in turn have either a left child of its own or a thread back to k , and so on for all successive left children. So by following the chain of left pointers from r , we will eventually find a thread pointing back to k . The situation is symmetrically similar when q is the left child of p —we can follow q 's right children to a thread pointing ahead to p .

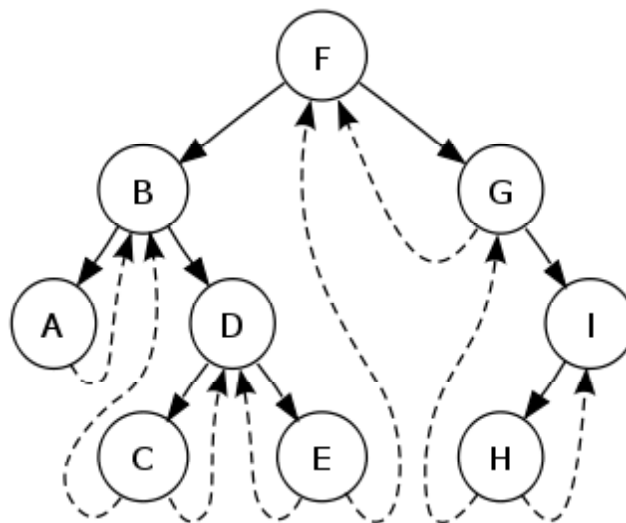


Figure 15.5. Threaded binary tree

If we look carefully at the linked representation of any binary tree, we notice that there are more null links than actual pointers. As we saw before, there are $n + 1$ null links and $2n$ total links. A clever way to make use of these null links has been devised by A. J. Perlis and C. Thornton. Their idea is to replace the null links by pointers, called threads, to other nodes in the tree. If the $RCHILD(P)$ is normally equal to zero, we will replace it by a pointer to the node which would be printed after P when traversing the tree in inorder. A null $LCHILD$ link at node P is replaced by a pointer to the node which immediately precedes node P in inorder. Figure 15.5. shows the binary tree of figure 15.3(b) with its new threads drawn in as dotted lines.

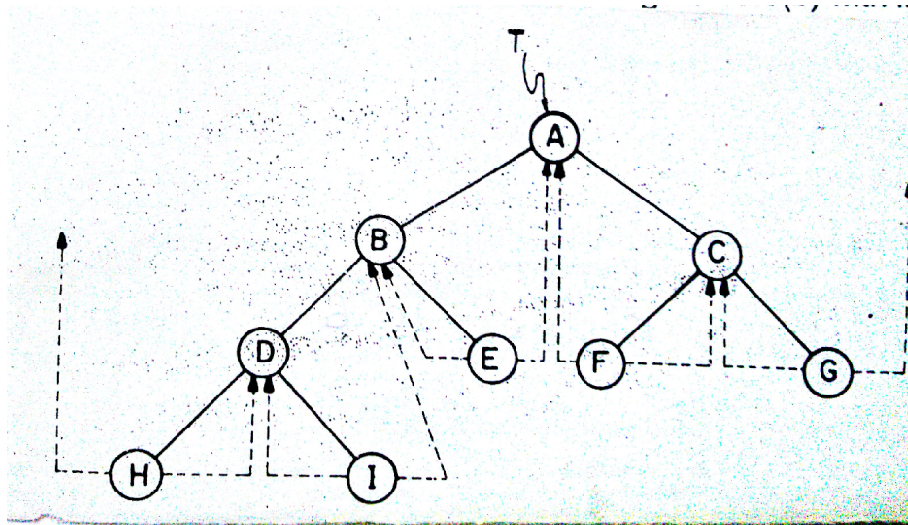


Figure 15.6 Threaded Tree

The tree T has 9 nodes and 10 null links which have been replaced by threads. If we traverse T in inorder the nodes will be visited in the order $H D I B E A F C G$. For example node E has a predecessor thread which points to B and a successor thread which points to A .

In the memory representation we must be able to distinguish between threads and normal pointers. This is done by adding two extra one bit fields $LBIT$ and $RBIT$.

$LBIT(P) = 1$ if $LCHILD(P)$ is a normal pointer

$LBIT(P) = 0$ if $LCHILD(P)$ is a thread

$RBIT(P) = 1$ if $RCHILD(P)$ is a normal pointer

$RBIT(P) = 0$ if $RCHILD(P)$ is a thread

In figure 15.6 we see that two threads have been left dangling in $LCHILD(H)$ and $RCHILD(G)$. In order that we leave no loose threads we will assume a head node for all threaded binary trees. Then the complete memory representation for the tree of figure 15.6 is shown in figure 15.7. The tree T is the left subtree of the head node. We assume that an empty binary tree is represented by its head node as

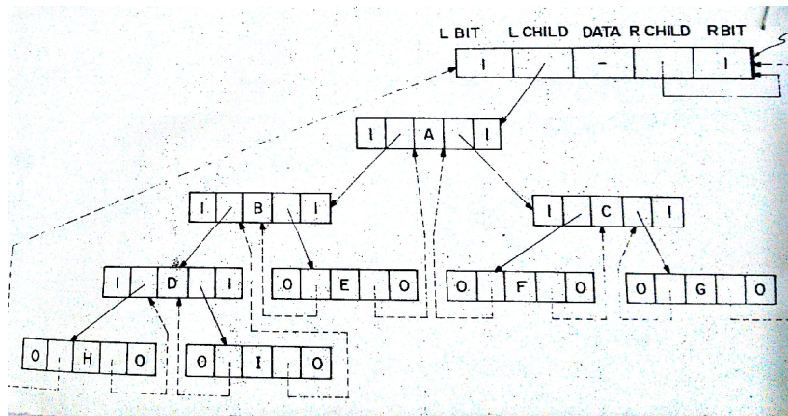


Figure 15.7 Memory Representation of Threaded Tree

This assumption will permit easy algorithm design. Now that we have made use of the old null links we will see that the algorithm for inorder traversal is simplified. First, we observe that for any node X in a binary tree, if $RBIT(X) = 0$, then the inorder successor of X is $RCHILD(X)$ by definition of threads. If $RBIT(X) = 1$, then the inorder successor of X is obtained by following a path of left child links from the right child of X until a node with $LBIT = 0$ is reached. The algorithm *INSUC* finds the inorder successor of any node X in a threaded binary tree

```

procedure INSUC( $X$ )
  //find the inorder succesor of  $X$  in a threaded binary tree//
   $S = RCHILD(X)$  //if  $RBIT(X) = 0$  we are done//
  if  $RBIT(X) = 1$  then [while  $LBIT(S) = 1$  do //follow left//
     $S = LCHILD(S)$  //until a thread//
  end]
  return ( $S$ )
end INSUC.

```

The interesting thing to note about procedure *INSUC* is that it is now possible to find the inorder successor of an arbitrary node in a threaded binary tree without using any information regarding inorder predecessors and also without using an additional stack. If we wish to list in inorder all the nodes in a threaded binary tree, then we can make repeated calls to the procedure *INSUC*. Since the tree is the left subtree of the head node and because of the choice of $RBIT =$

1 for the head node, the inorder sequence of nodes for tree T is obtained by the procedure TINORDER.

```

procedure TINORDER (T)
  //traverse the threaded binary tree, T, in inorder//
  HEAD= T
  loop
    T =INSUC(T)
    if T = HEAD then return
    print(DATA(T))
  forever
end TINORDER

```

The computing time is still $O(n)$ for a binary tree with n nodes. The constant here will be somewhat smaller than for procedure INORDER3.

We have seen how to use the threads of a threaded binary tree for inorder traversal. These threads also simplify the algorithms for preorder and postorder traversal. Before closing this section let us see how to make insertions into a threaded tree. This will give us a procedure for growing threaded trees. We shall study only the case of inserting a node T as the right child of a node S . The case of insertion of a left child is given as an exercise. If S has an empty right subtree, then the insertion is simple and diagrammed in figure 15.8(a). If the right subtree of S is non-empty, then this right subtree is made the right subtree of T after insertion. When this is done, T becomes the inorder predecessor of a node which has a LBIT = 0 and consequently there is a thread which has to be updated to point to T . The node containing this thread was previously the inorder successor of S . Figure 15.8 (b) illustrates the insertion for this case. In both cases S is the inorder predecessor of T . The details are spelled out in algorithm INSERT_RIGHT.

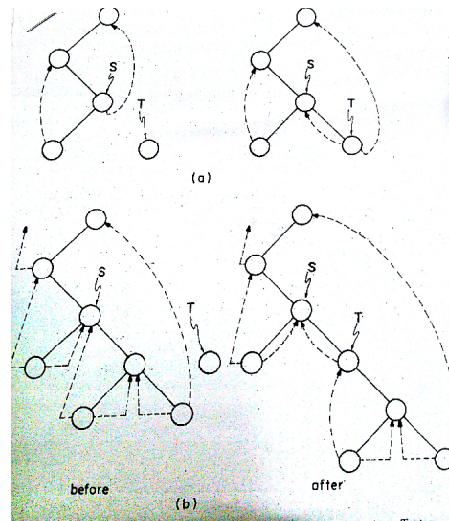


Figure 15.8 Insertion of T as a Right Child of S in a Threaded Binary Tree

```

procedure INSERT_RIGHT (S,T)
  //insert node T as the right child of S in a threaded binary tree//
  RCHILD(T) =RCHILD(S); RBIT(T)= RBIT(S)
  LCHILD(T) =S; LBIT(T)= 0 //LCHILD(T) is a thread//
  RCHILD(S) =T; RBIT(S) =1 //attach node T to S//
  if RBIT (T) = 1 then [W =INSUC(T)//S had a right child//
  LCHILD(W) =T]
  end INSERT_RIGHT.
  
```

15.7 Summary

A tree is a widely used abstract data type (ADT) or data structure implementing this ADT that simulates a hierarchical tree structure, with a root value and subtrees of children, represented as a set of linked nodes.

A tree data structure can be defined recursively (locally) as a collection of nodes (starting at a root node), where each node is a data structure consisting of a value, together with a list of

references to nodes (the “children”), with the constraints that no reference is duplicated, and none points to the root.

“A binary tree is threaded by making all right child pointers that would normally be null point to the inorder successor of the node, and all left child pointers that would normally be null point to the inorder predecessor of the node.”

15.8 Review Questions

1. Define tree.
2. Define Binary tree.
3. Explain the various binary tree traversal methods.
4. Describe threaded binary tree.
5. Discuss about Binary search tree.
6. Explain the representation of binary tree.

LESSON 16

GRAPH

Structure

16.1 Introduction

16.2 Learning Objectives

16.3 Graph Representation

16.4 Graph Traversal

16.5 Shortest Path Problems

16.6 Minimum Cost Spanning Trees

16.7 Summary

16.8 Review Questions

16.1 Introduction

A graph data structure consists of a finite (and possibly mutable) set of ordered pairs, called edges or arcs, of certain entities called nodes or vertices. As in mathematics, an edge (x,y) is said to point or go from x to y . The nodes may be part of the graph structure, or may be external entities represented by integer indices or references. A graph data structure may also associate to each edge some edge value, such as a symbolic label or a numeric attribute (cost, capacity, length, etc.).

16.2 Learning Objectives

At the end of this lesson you can able to know-

- Graph Representation
- Graph Traversal
- Minimum Cost Spanning Trees
- Shortest Path Problems
- Applications of Graph

16.3 Graph Representation

To facilitate and simplify our discussion, we borrow a few definitions and terminology from set theory. Let X be a set. If a is an element of X , we write $a \in X$. (The symbol “ $\in$ ” means “belongs to.”) A set Y is called a subset of X if every element of Y is also an element of X . If Y is a subset of X , we write $Y \subseteq X$. (The symbol “ $\subseteq$ ” means “is a subset of.”) The intersection of sets A and B , written $A \cap B$, is the set of all elements that are in A and B ; that is, $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$. (The symbol “ $\cap$ ” means “intersection.”) The union of sets A and B , written $A \cup B$, is the set of all elements that are in A or in B ; that is, $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$. (The symbol “ $\cup$ ” means “union.”) For sets A and B , the set $A \times B$ is the set of all ordered pairs of elements of A and B ; that is, $A \times B = \{(a, b) \mid a \in A, b \in B\}$. (The symbol “ $\times$ ” means Cartesian product.”).

A graph G is a pair, $G = (V, E)$, where V is a finite nonempty set, called the set of vertices of G and $E \subseteq V \times V$. That is, the elements of E are pairs of elements of V . E is called the set of edges of G . G is called trivial if it has only one vertex.

Let $V(G)$ denote the set of vertices, and $E(G)$ denote the set of edges of a graph G . If the elements of E are ordered pairs, G is called a directed graph or digraph; otherwise, G is called an undirected graph. In an undirected graph, the pairs (u, v) and (v, u) represent the same edge. Let G be a graph. A graph H is called a subgraph of G if $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$; that is, every vertex of H is a vertex of G , and every edge in H is an edge in G .

A graph can be shown pictorially. The vertices are drawn as circles, and a label inside the circle represents the vertex. In an undirected graph, the edges are drawn using lines. In a directed graph, the edges are drawn using arrows.

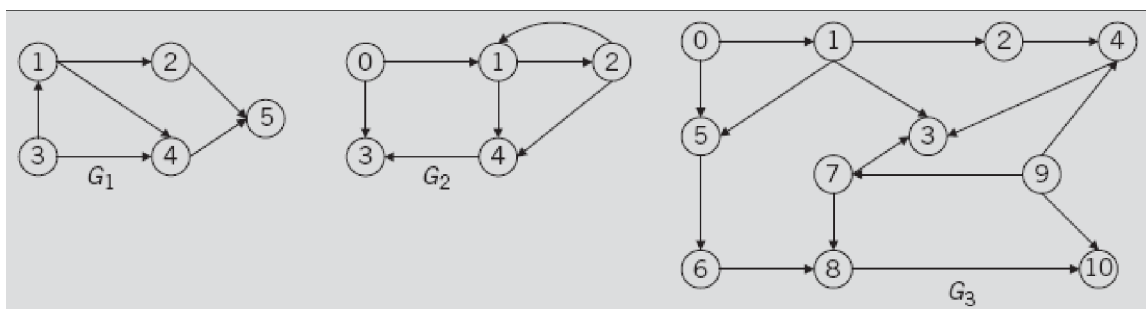


Figure 16.1 Various directed graphs

For the graphs of Figure 16.1, we have

$$V(G_1) = \{1, 2, 3, 4, 5\} \quad E(G_1) = \{(1, 2), (1, 4), (2, 5), (3, 1), (3, 4), (4, 5)\}$$

$$V(G_2) = \{0, 1, 2, 3, 4\} \quad E(G_2) = \{(0, 1), (0, 3), (1, 2), (1, 4), (2, 1), (2, 4), (4, 3)\}$$

$$V(G_3) = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} \quad E(G_3) = \{(0, 1), (0, 5), (1, 2), (1, 3), (1, 5), (2, 4), (4, 3), (5, 6), (6, 8), (7, 3), (7, 8), (8, 10), (9, 4), (9, 7), (9, 10)\}$$

Let G be an undirected graph. Let u and v be two vertices in G . Then u and v are called adjacent if there is an edge from one to the other; that is, $(u, v) \in E$. An edge incident on a single vertex is called a loop. If two edges, e_1 and e_2 , are associated with the same pair of vertices $\{u, v\}$, then e_1 and e_2 are called parallel edges. A graph is called a simple graph if it has no loops and no parallel edges. Let $e = (u, v)$ be an edge in G . We then say that edge e is incident on the vertices u and v . The degree of u , written $\deg(u)$ or $d(u)$, is the number of edges incident with u . We make the convention that each loop on a vertex u contributes 2 to the degree of u . u is called an even (odd) degree vertex if the degree of u is even (odd). There is a path from u to v if there is a sequence of vertices $u_1, u_2, \dots, u_n$ such that $u = u_1$, $u_n = v$ and (u_i, u_{i+1}) is an edge for all $i = 1, 2, \dots, n-1$. Vertices u and v are called connected if there is a path from u to v . A simple path is a path in which all the vertices, except possibly the first and last vertices, are distinct. A cycle in G is a simple path in which the first and last vertices are the same. G is called connected if there is a path from any vertex to any other vertex. A maximal subset of connected vertices is called a component of G .

Let G be a directed graph, and let u and v be two vertices in G . If there is an edge from u to v , that is, $(u, v) \in E$, we say that u is adjacent to v and v is adjacent from u . The definitions of the paths and cycles in G are similar to those for undirected graphs. G is called strongly connected if any two vertices in G are connected. Consider the directed graphs of Figure 16.1. In G_1 , 1-4-5 is a path from vertex 1 to vertex 5. There are no cycles in G_1 . In G_2 , 1-2-1 is a cycle. In G_3 , 0-1-2-4-3 is a path from vertex 0 to vertex 3; 1-5-6-8-10 is a path from vertex 1 to vertex 10. There are no cycles in G_3 .

To write programs that process and manipulate graphs, the graphs must be stored—that is, represented—in computer memory. A graph can be represented (in computer memory) in several ways. We now discuss two commonly used ways: adjacency matrices and adjacency lists.

16.3.1 Adjacency Matrices

Let G be a graph with n vertices, where $n > 0$. Let $V(G) = \{v_1, v_2, \dots, v_n\}$. The adjacency matrix A_G is a two-dimensional $n \times n$ matrix such that the $(i, j)^{\text{th}}$ entry of A_G is 1 if there is an edge from v_i to v_j ; otherwise, the $(i, j)^{\text{th}}$ entry is 0. That is,

$$A_G(i, j) = \begin{cases} 1 & \text{if } (v_i, v_j) \in E(G) \\ 0 & \text{otherwise} \end{cases}$$

In an undirected graph, if $(v_i, v_j) \in E(G)$, then $(v_j, v_i) \in E(G)$, so $A_G(i, j) = 1 = A_G(j, i)$. It follows that the adjacency matrix of an undirected graph is symmetric.

Consider the directed graphs of Figure 16.1. The adjacency matrices of the directed graphs G_1 and G_2 are as follows:

$$A_{G_1} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A_{G_2} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

16.3.2 Adjacency Lists

Let G be a graph with n vertices, where $n > 0$. Let $V(G) = \{v_1, v_2, \dots, v_n\}$. In the adjacency list representation, corresponding to each vertex, v , there is a linked list such that each node of the linked list contains the vertex, u , such that $(v, u) \in E(G)$. Because there are n nodes, we use an array, A , of size n , such that $A[i]$ is a reference variable pointing to the first node of the linked list containing the vertices to which v_i is adjacent. Clearly, each node has two components, say vertex and link. The component vertex contains the index of the vertex adjacent to vertex i .

16.3.3 Operations on Graphs

Now that you know how to represent graphs in computer memory, the next obvious step is to learn the basic operations on a graph. The operations commonly performed on a graph are as follows:

1. Create the graph. That is, store the graph in computer memory using a particular graph representation.
2. Clear the graph. This operation makes the graph empty.
3. Determine whether the graph is empty.
4. Traverse the graph.
5. Print the graph.

How a graph is represented in computer memory depends on the specific application. For illustration purposes, we use the adjacency list (linked list) representation of graphs. Therefore, for each vertex v the vertices adjacent to v (in a directed graph, also called the immediate successors) are stored in the linked list associated with v .

The labeling of the vertices of a graph depends on a specific application. If you are dealing with the graph of cities, you could label the vertices by the names of the cities. However, to write algorithms to manipulate a graph as well as to simplify the algorithm, there must be some ordering to the vertices. That is, we must specify the first vertex, the second vertex, and so on. Therefore, for simplicity, throughout this lesson we assume that the n vertices of the graphs are numbered $0, 1, \dots, n - 1$. Moreover, it follows that the class we will design to implement the graph algorithm will not be a template.

Graphs as ADTs

In this section, we describe the class to implement graphs as an abstract data type (ADT) and provide the definitions of the functions to implement the operations on a graph. The following class defines a graph as an ADT:

```
class graphType
{
public:
```

```
bool isEmpty() const;
//Function to determine whether the graph is empty.
//Postcondition: Returns true if the graph is empty; otherwise, returns false.
void createGraph();
//Function to create a graph.
//Postcondition: The graph is created using the adjacency list representation.
void clearGraph();
//Function to clear graph.
//Postcondition: The memory occupied by each vertex is deallocated.
void printGraph() const;
//Function to print graph.
//Postcondition: The graph is printed.
void depthFirstTraversal();
//Function to perform the depth first traversal of the entire graph.
//Postcondition: The vertices of the graph are printed
// using the depth first traversal algorithm.
void dftAtVertex(int vertex);
//Function to perform the depth first traversal of
//the graph at a node specified by the parameter vertex.
//Postcondition: Starting at vertex, the vertices are
// printed using the depth first traversal algorithm.
void breadthFirstTraversal();
//Function to perform the breadth first traversal of the entire graph.
//Postcondition: The vertices of the graph are printed
// using the breadth first traversal algorithm.
graphType(int size = 0);
//Constructor
//Postcondition: gSize = 0; maxSize = size;
```

```

// graph is an array of pointers to linked lists.
~graphType();
//Destructor
//The storage occupied by the vertices is deallocated.
protected:
int maxSize; //maximum number of vertices
int gSize; //current number of vertices
unorderedLinkedList<int> *graph; //array to create
//adjacency lists
private:
void dft(int v, bool visited[]);
//Function to perform the depth first traversal of
//the graph at a node specified by the parameter vertex.
//This function is used by the public member functions
//depthFirstTraversal and dftAtVertex.
//Postcondition: Starting at vertex, the vertices are
// printed using the depth first traversal algorithm.
};

```

A graph is empty if the number of vertices is 0—that is, if gSize is 0. Therefore, the definition of the function isEmpty is as follows:

```

bool graphType::isEmpty() const
{
return (gSize == 0);
}

```

The definition of the function createGraph depends on how the data is input into the program. For illustration purposes, we assume that the data to the program is input from a file. The user is prompted for the input file. The data in the file appears in the following form:


```

5
0 2 4 ... -999
1 3 6 8 ... -999
...

```

The first line of input specifies the number of vertices in the graph. The first entry in the remaining lines specifies the vertex, and all of the remaining entries in the line (except the last) specify the vertices that are adjacent to the vertex. Each line ends with the number –999.

Using these conventions, the definition of the function createGraph is as follows:

```

void graphType::createGraph()
{
    ifstream infile;
    char fileName[50];
    int vertex;
    int adjacentVertex;
    if (gSize != 0) //if the graph is not empty, make it empty
        clearGraph();
    cout << "Enter input file name: ";
    cin >> fileName;
    cout << endl;
    infile.open(fileName);
    if (!infile)
    {
        cout << "Cannot open input file." << endl;
        return;
    }
    infile >> gSize; //get the number of vertices
    for (int index = 0; index < gSize; index++)
    {

```

```

infile >> vertex;
infile >> adjacentVertex;
while (adjacentVertex != -999)
{
graph[vertex].insertLast(adjacentVertex);
infile >> adjacentVertex;
} //end while
} // end for
infile.close();
} //end createGraph

```

The function `clearGraph` empties the graph by deallocating the storage occupied by each linked list and then setting the number of vertices to 0.

```

void graphType::clearGraph()
{
for (int index = 0; index < gSize; index++)
graph[index].destroyList();
gSize = 0;
} //end clearGraph

```

The definition of the function `printGraph` is given next:

```

void graphType::printGraph() const
{
for (int index = 0; index < gSize; index++)
{
cout << index << " ";
graph[index].print();
cout << endl;
}
cout << endl;

```

```
} //end printGraph
```

The definitions of the constructor and the destructor are as follows:

```
//Constructor
graphType::graphType(int size)
{
    maxSize = size;
    gSize = 0;
    graph = new unorderedLinkedList<int>[size];
}

//Destructor
graphType::~~graphType()
{
    clearGraph();
}
```

16.4 Graph Traversals

Traversing a graph is similar to traversing a binary tree, except that traversing a graph is a bit more complicated. Now a binary tree has no cycles and starting at the root node we can traverse the entire tree. On the other hand, a graph might have cycles and we might not be able to traverse the entire graph from a single vertex (for example, if the graph is not connected). Therefore, we must keep track of the vertices that have been visited. We must also traverse the graph from each vertex (that has not been visited) of the graph. This ensures that the entire graph is traversed.

The two most common graph traversal algorithms are the depth-first traversal and breadth-first traversal, which are described next. For simplicity, we assume that when a vertex is visited, its index is output. Moreover, each vertex is visited only once. We use the bool array visited to keep track of the visited vertices.

16.4.1 Depth-First Traversal

The depth-first traversal is similar to the preorder traversal of a binary tree. The general algorithm is as follows:

for each vertex, v , in the graph
 if v is not visited
 start the depth first traversal at v

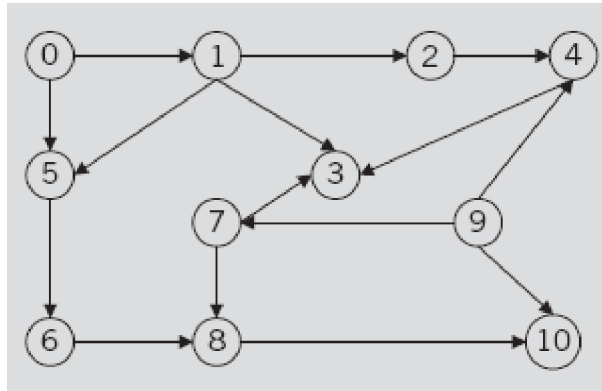


Figure 16.2 Directed Graph G3

The depth-first ordering of the vertices of graph G3 in Figure 16.2 is as follows:

0 1 2 4 3 5 6 8 10 7 9

For the graph of Figure 16.2, the depth-first search starts at the vertex 0. After visiting all the vertices that can be reached starting at the vertex 0, the depth-first search starts at the next vertex that is not visited. There is a path from the vertex 0 to every other vertex except the vertices 7 and 9. Therefore, when the depth-first search starts at the vertex 0, all the vertices except 7 and 9 are visited before these vertices. After completing the depth-first search that started at the vertex 0, the depth-first search starts at the vertex 7 and then at the vertex 9. Note that there is no path from the vertex 7 to the vertex 9. Therefore, after completing the depth-first search that started at the vertex 7, the depth-first search starts at the vertex 9.

The general algorithm to do a depth-first traversal at a given node, v , is as follows:

1. mark node v as visited
2. visit the node
3. for each vertex u adjacent to v
 if u is not visited
 start the depth first traversal at u

Clearly, this is a recursive algorithm. We use a recursive function, `dft`, to implement this algorithm. The vertex at which the depth-first traversal is to be started, and the bool array `visited`, are passed as parameters to this function.

```
void graphType::dft(int v, bool visited[])
{
    visited[v] = true;
    cout << " " << v << " "; //visit the vertex
    linkedListIterator<int> graphIt;
    //for each vertex adjacent to v
    for (graphIt = graph[v].begin(); graphIt != graph[v].end();
        ++graphIt)
    {
        int w = *graphIt;
        if (!visited[w])
            dft(w, visited);
    } //end while
} //end dft
```

In the preceding code, note that the statement

`linkedListIterator<int> graphIt;` declares `graphIt` to be an iterator. In the for loop, we use it to traverse a linked list (adjacency list) to which the pointer `graph[v]` points. Next, let us look at the statement

```
int w = *graphIt;
```

The expression `*graphIt` returns the label of the vertex, adjacent to the vertex `v`, to which `graphIt` points.

Next, we give the definition of the function `depthFirstTraversal` to implement the depth-first traversal of the graph.

```
void graphType::depthFirstTraversal()
{
```

```

bool *visited; //pointer to create the array to keep
//track of the visited vertices
visited = new bool[gSize];
for (int index = 0; index < gSize; index++)
    visited[index] = false;
//For each vertex that is not visited, do a depth
//first traverssal
for (int index = 0; index < gSize; index++)
    if (!visited[index])
        dft(index,visited);
delete [] visited;
} //end depthFirstTraversal

```

The function `depthFirstTraversal` performs a depth-first traversal of the entire graph. The definition of the function `dftAtVertex`, which performs a depth-first traversal at a given vertex, is as follows:

```

void graphType::dftAtVertex(int vertex)
{
    bool *visited;
    visited = new bool[gSize];
    for (int index = 0; index < gSize; index++)
        visited[index] = false;
    dft(vertex, visited);
    delete [] visited;
} // end dftAtVertex

```

16.4.2 Breadth-First Traversal

The breadth-first traversal of a graph is similar to traversing a binary tree level-by-level (the nodes at each level are visited from left to right). All the nodes at any level, i , are visited before visiting the nodes at level $i + 1$. The breadth-first ordering of the vertices of the graph G_3 in Figure 16.2 is as follows:

0 1 5 2 3 6 4 8 10 7 9

For the graph G3, we start the breadth traversal at vertex 0. After visiting the vertex 0, we visit the vertices that are directly connected to it and are not visited, which are 1 and 5. Next, we visit the vertices that are directly connected to 1 and are not visited, which are 2 and 3. After this, we visit the vertices that are directly connected to 5 and are not visited, which is 6. After this, we visit the vertices that are directly connected to 2 and are not visited, and so on.

As in the case of the depth-first traversal, because it might not be possible to traverse the entire graph from a single vertex, the breadth-first traversal also traverses the graph from each vertex that is not visited. Starting at the first vertex, the graph is traversed as much as possible; we then go to the next vertex that has not been visited. To implement the breadth-first search algorithm, we use a queue. The general algorithm is as follows:

1. for each vertex v in the graph
 - if v is not visited
 - add v to the queue //start the breadth first search at v
2. Mark v as visited
3. while the queue is not empty
 - 3.1. Remove vertex u from the queue
 - 3.2. Retrieve the vertices adjacent to u
 - 3.3. for each vertex w that is adjacent to u
 - if w is not visited
 - 3.3.1. Add w to the queue
 - 3.3.2. Mark w as visited

The following C++ function, `breadthFirstTraversal`, implements this algorithm:

```
void graphType::breadthFirstTraversal()
{
    linkedQueueType<int> queue;
    bool *visited;
```

```

visited = new bool[gSize];
for (int ind = 0; ind < gSize; ind++)
    visited[ind] = false; //initialize the array
//visited to false
linkedListIterator<int> graphIt;
for (int index = 0; index < gSize; index++)
    if (!visited[index])
    {
        queue.addQueue(index);
        visited[index] = true;
        cout << " " << index << " ";
        while (!queue.isEmptyQueue())
        {
            int u = queue.front();
            queue.deleteQueue();
            for (graphIt = graph[u].begin();
                graphIt != graph[u].end(); ++graphIt)
            {
                int w = *graphIt;
                if (!visited[w])
                {
                    queue.addQueue(w);
                    visited[w] = true;
                    cout << " " << w << " ";
                }
            }
        } //end while
    }

```



```

delete [] visited;
} //end breadthFirstTraversal

```

16.5 Shortest Path Algorithm

Graph theory has many applications. For example, we can use graphs to show how different chemicals are related or to show airline routes. Graphs also be used to show the highway structure of a city, state, or country. The edges connecting two vertices can be assigned a nonnegative real number, called the weight of the edge. If the graph represents a highway structure, the weight can represent the distance between two places or the travel time from one place to another. Such graphs are called weighted graphs.

Let G be a weighted graph. Let u and v be two vertices in G , and let P be a path in G from u to v . The weight of the path P is the sum of the weights of all the edges on the path P , which is also called the weight of v from u via P .

Let G be a weighted graph representing a highway structure. Suppose that the weight of an edge represents the travel time. For example, to plan monthly business trips, a salesperson wants to find the shortest path (that is, the path with the smallest weight) from her or his city to every other city in the graph. Many such problems exist in which we want to find the shortest path from a given vertex, called the source, to every other vertex in the graph. This section describes the shortest path algorithm, also called a greedy algorithm, developed by Dijkstra.

Let G be a graph with n vertices, where $n \geq 0$. Let $V(G) = \{v_1, v_2, \dots, v_n\}$. Let W be a two-dimensional $n \times n$ matrix such that

$$W(i,j) = \begin{cases} w_{ij} & \text{if } (v_i, v_j) \text{ is an edge in } G \text{ and } w_{ij} \text{ is the weight of the edge } (v_i, v_j) \\ \infty & \text{if there is no edge from } v_i \text{ to } v_j \end{cases}$$

The input to the program is the graph and the weight matrix associated with the graph. To make inputting the data easier, we extend the definition of the class `graphType` (using inheritance), and add the function `createWeightedGraph` to create the graph and the weight matrix associated with the graph. Let us call this class `weightedGraphType`. The functions to implement the shortest path algorithm will also be added to this class.

```

class weightedGraphType: public graphType
{
public:

```

```

void createWeightedGraph();
//Function to create the graph and the weight matrix.
//Postcondition: The graph using adjacency lists and
// its weight matrix is created.
void shortestPath(int vertex);
//Function to determine the weight of a shortest path
//from vertex, that is, source, to every other vertex
//in the graph.
//Postcondition: The weight of the shortest path from vertex
// to every other vertex in the graph is determined.
void printShortestDistance(int vertex);
//Function to print the shortest weight from the vertex
//specified by the parameter vertex to every other vertex in
//the graph.
//Postcondition: The weight of the shortest path from vertex
// to every other vertex in the graph is printed.
weightedGraphType(int size = 0);
//Constructor
//Postcondition: gSize = 0; maxSize = size;
// graph is an array of pointers to linked lists.
// weights is a two-dimensional array to store the weights
// of the edges.
// smallestWeight is an array to store the smallest weight
// from source to vertices.
~weightedGraphType();
//Destructor
//The storage occupied by the vertices and the arrays

```

```
//weights and smallestWeight is deallocated.
protected:
double **weights; //pointer to create weight matrix
double *smallestWeight; //pointer to create the array to store
//the smallest weight from source to vertices
};
```

Given a vertex, say vertex (that is, a source), this section describes the shortest path algorithm.

The general algorithm is as follows:

1. Initialize the array smallestWeight so that $\text{smallestWeight}[u] = \text{weights}[\text{vertex}, u]$
2. Set $\text{smallestWeight}[\text{vertex}] = 0$.
3. Find the vertex v that is closest to vertex for which the shortest path has not been determined.
4. Mark v as the (next) vertex for which the smallest weight is found.
5. For each vertex w in G , such that the shortest path from vertex to w has not been determined and an edge (v, w) exists, if the weight of the path to w via v is smaller than its current weight, update the weight of w to the weight of v + the weight of the edge (v, w) . Because there are n vertices, Steps 3 through 5 are repeated $n - 1$ times.

16.6 Minimum Spanning Tree

Consider the graph of Figure 16.3, which represents the airline connections of a company, between seven cities. The number on each edge represents some cost factor of maintaining the connection between cities.

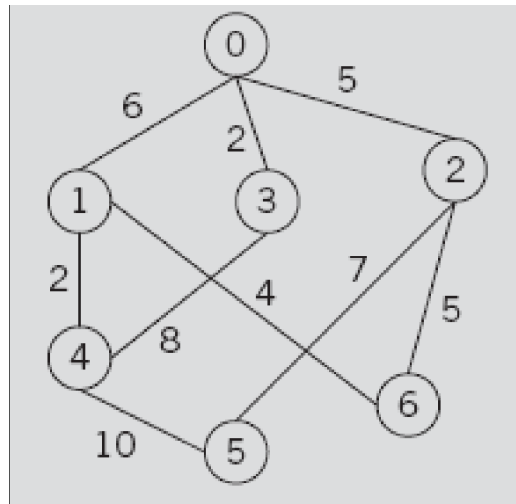


Figure 16.3 Airline connections between cities and the cost factor of maintaining the connections

Due to financial hardship, the company needs to shut down the maximum number of connections and still be able to fly (may be not directly) from one city to another. The graphs of Figure 16.4- (a), (b), and (c) show three different solutions.

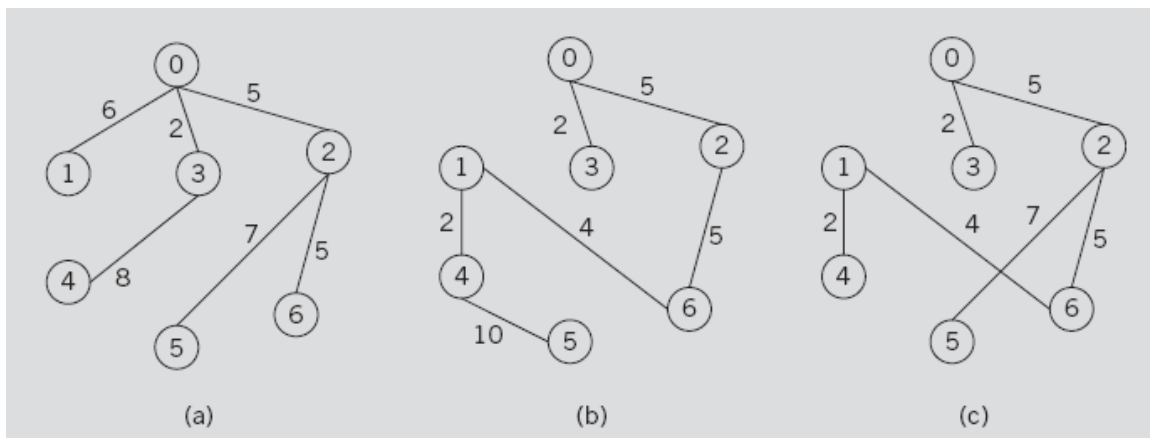


Figure 16.4 Possible solutions to the graph of Figure 16.3

The total cost factor of maintaining the remaining connections in Figure 16.4(a) is 33, in Figure 16.4(b) it is 28, and in Figure 16.4(c) it is 25. Out of these three solutions, obviously, the desired solution is the one shown by the graph of Figure 16.4(c) because it gives the lowest

cost factor. Graphs of Figure 16.4 are called spanning trees of the graph of Figure 16.4. Let us note the following from the graphs of Figure 16.4. Each of the graphs of Figure 16.4 is a subgraph of the graph of Figure 16.3, and there is a unique path from a node to any other node. Such graphs are called trees. There are many other situations, where given a weighted graph, we need to determine a graph such as in Figure 16.4 with the smallest weight. In this section, we give an algorithm to determine such graphs. However, first we introduce some terminology.

A (free) tree T is a simple graph such that if u and v are two vertices in T , there is a unique path from u to v . A tree in which a particular vertex is designated as a root is called a rooted tree. If a weight is assigned to the edges in T , T is called a weighted tree. If T is a weighted tree, the weight of T , denoted by $W(T)$, is the sum of the weights of all the edges in T .

A tree T is called a spanning tree of graph G if T is a subgraph of G such that $V(T) = V(G)$, that is, all the vertices of G are in T . Suppose G denotes the graph of Figure 16.3. Then the graphs of Figure 16.4 show three spanning trees of G . Let us note the following theorem.

Theorem 16-1: A graph G has a spanning tree if and only if G is connected.

From this theorem, it follows that to determine a spanning tree of a graph, the graph must be connected.

Definition: Let G be a weighted graph. A minimum (minimal) spanning tree of G is a spanning tree with the minimum weight.

There are two well-known algorithms, Prim's algorithm and Kruskal's algorithm, for finding a minimum spanning tree of a graph. This section discusses Prim's algorithm to find a minimum spanning tree. If interested, you can find Kruskal's algorithm in the discrete structures book or a data structures book listed in Appendix H. Prim's algorithm builds the tree iteratively by adding edges until a minimum spanning tree is obtained. We start with a designated vertex, which we call the source vertex. At each iteration, a new edge that does not form a cycle is added to the tree.

Let G be a weighted graph such that $V(G) = \{v_0, v_1, \dots, v_{n-1}\}$, where n , the number of vertices, is nonnegative. Let v_0 be the source vertex. Let T be the partially built tree. Initially $V(T)$ contains the source vertex and $E(T)$ is empty. At the next iteration, a new vertex that is not in

$V(T)$ is added to $V(T)$, such that an edge exists from a vertex in T to the new vertex so that the corresponding edge has the smallest weight. The corresponding edge is added to $E(T)$. The general form of Prim's algorithm is as follows. (Let n be the number of vertices in G .)

```

1. Set  $V(T) = \{\text{source}\}$ 

2. Set  $E(T) = \text{empty}$ 

3. for  $i = 1$  to  $n$ 

  3.1.  $\text{minWeight} = \text{infinity}$ ;

  3.2. for  $j = 1$  to  $n$ 

    if  $v_j$  is in  $V(T)$ 

      for  $k = 1$  to  $n$ 

        if  $v_k$  is not in  $T$  and  $\text{weight}[v_j, v_k] < \text{minWeight}$ 

          {

             $\text{endVertex} = v_k$ ;

             $\text{edge} = (v_j, v_k)$ ;

             $\text{minWeight} = \text{weight}[v_j, v_k]$ ;

          }

  3.3.  $V(T) = V(T) \cup \{\text{endVertex}\}$ ;

  3.4.  $E(T) = E(T) \cup \{\text{edge}\}$ ;

```

Let N denote the set of vertices of G that are not in T . Suppose that the source vertex is 0. Figure 16.5 shows how Prim's algorithm works.

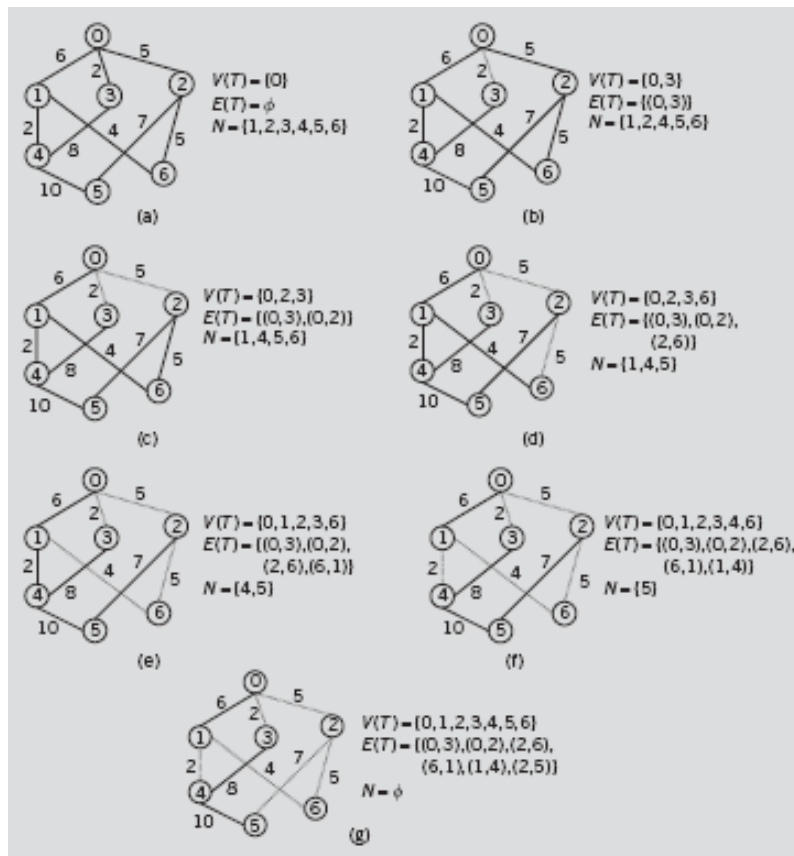


Figure 16.5

After Steps 1 and 2 execute, $V(T)$, $E(T)$, and N are as shown in Figure 16.5(a). Step 3 checks the following edges: $(0,1)$, $(0,2)$, and $(0,3)$. The weight of the edge $(0,1)$ is 6, the weight of the edge $(0,2)$ is 5, and the weight of the edge $(0,3)$ is 2. Clearly, the edge $(0,3)$ has the smallest weight; see Figure 16.5(b). Therefore, vertex 3 is added to $V(T)$ and the edge $(0,3)$ is added to $E(T)$. Figure 16.5(b) shows the resulting graph, $V(T)$, $E(T)$, and N . (The dotted line shows the edge in T .)

Next, Step 3 checks the following edges: $(0,1)$, $(0,2)$, and $(3,4)$. The weight of the edge $(0,1)$ is 6, the weight of the edge $(0,2)$ is 5, and the weight of the edge $(3,4)$ is 8. Clearly, the edge $(0,2)$ has the smallest weight. Therefore, vertex 2 is added to $V(T)$ and the edge $(0,2)$ is added to $E(T)$. Figure 16.5(c) shows the resulting graph, $V(T)$, $E(T)$, and N .

At the next iteration, Step 3 checks the following edges: (0,1), (2,5), (2,6), and (3,4). The weight of the edge (0,1) is 6, the weight of the edge (2,5) is 7, the weight of the edge (2,6) is 5, and the weight of the edge (3,4) is 8. Clearly, the edge (2,6) has the smallest weight.

Therefore, vertex 6 is added to $V(T)$ and the edge (2,6) is added to $E(T)$. Figure 16.5(d) shows the resulting graph, $V(T)$, $E(T)$, and N . (The dotted lines show the edges in T .) At the next iteration, Step 3 checks the following edges: (0,1), (2,5), (3,4), and (6,1). The weight of the edge (0,1) is 6, the weight of the edge (2,5) is 7, the weight of the edge (3,4) is 8, and the weight of the edge (6,1) is 4. Clearly, the edge (6,1) has the smallest weight. Therefore, vertex 1 is added to $V(T)$ and the edge (6,1) is added to $E(T)$. Figure 16.5(e) shows the resulting graph, $V(T)$, $E(T)$, and N . (The dotted lines show the edges in T .) At the next iteration, Step 3 checks the following edges: (1,4), (2,5), and (3,4). The weight of the edge (1,4) is 2, the weight of the edge (2,5) is 7, and the weight of the edge (3,4) is 8. Clearly, the edge (1,4) has the smallest weight. Therefore, vertex 4 is added to $V(T)$ and the edge (1,4) is added to $E(T)$. Figure 16.5(f) shows the resulting graph, $V(T)$, $E(T)$, and N . (The dotted lines show the edges in T .)

At the next iteration, Step 3 checks the following edges: (2,5) and (4,5). The weight of the edge (2,5) is 7 and the weight of the edge (4,5) is 10. Clearly, the edge (2,5) has the smallest weight. Therefore, vertex 5 is added to $V(T)$ and the edge (2,5) is added to $E(T)$. Figure 16.5(g) shows the resulting graph, $V(T)$, $E(T)$, and N . (The dotted lines show the edges in T .)

In Figure 16.5(g), the dotted lines show a minimum spanning tree of G of weight 25.

Before we give the definition of the function to implement Prim's algorithm, let us first define a spanning tree as an ADT.

Let $mstv$ be a bool array such that $mstv[j]$ is true if the vertex v_i is in T , and false otherwise. Let $edges$ be an array such that $edges[j] = k$, if there is an edge connecting vertices v_j and v_k . Suppose that the edge (v_i, v_j) is in the minimum spanning tree. Let $edgeWeights$ be an array such that $edgeWeights[j]$ is the weight of the edge (v_i, v_j) .


```

class msTreeType: public graphType
{
public:
void createSpanningGraph();
//Function to create the graph and the weight matrix.
//Postcondition: The graph using adjacency lists and
// its weight matrix is created.
void minimumSpanning(int sVertex);
//Function to create a minimum spanning tree with
//root as sVertex.
// Postcondition: A minimum spanning tree is created.
// The weight of the edges is also saved in the array
// edgeWeights.
void printTreeAndWeight();
//Function to output the edges of the minimum spanning tree
//and the weight of the minimum spanning tree.
//Postcondition: The edges of a minimum spanning tree
// and their weights are printed.
msTreeType(int size = 0);
//Constructor
//Postcondition: gSize = 0; maxSize = size;
// graph is an array of pointers to linked lists.
// weights is a two-dimensional array to store the weights
// of the edges.
// edges is an array to store the edges of a minimum

```

```

// spanning tree.

// edgeWeights is an array to store the weights of the
// edges of a minimum spanning tree.

~msTreeType();

//Destructor

//The storage occupied by the vertices and the arrays
//weights, edges, and edgeWeights is deallocated.

protected:

int source;

double **weights;

int *edges;

double *edgeWeights;

};

```

16.7 Summary

A graph data structure consists of a finite (and possibly mutable) set of ordered pairs, called edges or arcs, of certain entities called nodes or vertices. As in mathematics, an edge (x,y) is said to point or go from x to y . The nodes may be part of the graph structure, or may be external entities represented by integer indices or references. A graph data structure may also associate to each edge some edge value, such as a symbolic label or a numeric attribute (cost, capacity, length, etc.).

Graph algorithms are a significant field of interest within computer science. Typical higher-level operations associated with graphs are: finding a path between two nodes, like depth-first search and breadth-first search and finding the shortest path from one node to another, like Dijkstra's algorithm. A solution to finding the shortest path from each node to every other node also exists in the form of the Floyd–Warshall algorithm.

16.8 Review Questions

1. Define Graph.
2. Explain the types of graph.
3. What do you mean by adjacency matrix?
4. Explain the Graph traversal methods with example.
5. Explain minimum cost spanning tree.
6. Discuss about shortest path algorithm.

MODEL QUESTION PAPER

PART A - (10 * 2 = 20 marks)

Answer any TEN questions.

All questions carry equal marks.

1. How to define a function in C++?
2. What is operator overloading?
3. What do you mean virtual function ?
4. What are derived class exception?
5. Define namespace.
6. Define a binary tree.
7. Translate infix expression into its equivalent post fix expression: $(A-B)*(D/E)$
8. What is a linked list?
9. What are the different types of traversing?
10. What is AVL tree?
11. What do you mean by overflow and underflow in Queue?
12. What is a splay operation?

PART B - (5 * 6= 30 marks)

Answer any FIVE questions.

All questions carry equal marks.

13. What are the feature of Object oriented Programming ?Give any two applications
14. Explain binary operator overloading using friend function with example.
15. Explain multilevel and hierarchical inheritance.
16. Write short notes on file pointers.
17. Discuss about implementation of stack?

18. Discuss the evaluation of expressions along with the algorithm for infix to postfix conversion.
19. Explain in detail about B tree.

PART C - (3 * 10 = 30 marks)

Answer any THREE questions.

All questions carry equal marks.

20. Explain exception handling with an example.
21. Explain threaded binary tree.
22. Write insertion and deletion algorithm for the doubly linked lists.
23. Explain the minimum cost of spanning tree? With example.
24. Explain an appropriate data structure to represent a polynomial and write suitable algorithm to multiply two polynomials.